

Оглавление

1 Вводные данные	4
1.1 Данные стенда	4
1.2 Учётные данные хостов домена.....	4
2 Создание виртуальных машин (STEP-0).....	6
2.1 Создание виртуальных сетевых коммутаторов	6
2.2 Создание локального DNS-резолвера (на основе hosts).....	7
2.3 Устанавливаем утилиту Packer	7
2.4 Подготавливаем рабочую директорию и собираем данные для работы Packer	7
2.5 Создаём описание установки cloud-init для Ubuntu	8
2.6 Создаём описание виртуальных машин на языке HCL для Packer ..	9
2.7 Запускаем создание виртуальных машин через Packer	11
2.7.1 Подготовка.....	11
2.7.2 Сервер kube-gateway	11
2.7.3 Сервер kube-combo-1	13
2.7.4 Сервер kube-combo-2	13
2.7.5 Сервер kube-combo-3	14
2.7.6 Результат	14
3 Настройка виртуальных машин	15
3.1 Установка CoreDNS	15
3.2 Создание центра сертификации SSL Step CA.....	19
3.3 Установка Ansible	22
3.4 Настройка хостов через Ansible	24
3.4.1 Роль для Debug	24
3.4.2 Настройка Bash.....	25
3.4.3 Установка утилит Linux	25
3.4.4 Установка ip_forwarding.....	27
3.4.5 Установка Sysdig	29
3.4.6 Установка Chrony	29
3.4.7 Настройка Timesyncd	30
3.4.8 Установка Docker	31
3.4.9 Установка Ctop	33
3.4.10 Копирование корневого сертификата stepca	33
3.4.11 Установка Nmapoxy	34
3.4.12 Установка Keepalived	36
3.4.13 Установка Kubernetes	39
3.4.14 Установка Etcctl	43

3.4.15 Установка репозитория gitlab-runner	44
3.4.16 Завершаем работу с Ansible.....	45
3.5 Подключение kubectl на gateway.dev.lan в управление Kubernetes .45	
3.5.1 Снимаем taint с кластера для разрешение запуска Pod-с на Control Plane	46
3.6 Установка Traefik	47
3.7 Настройка мониторинга.....	49
3.7.1 Установка Prometheus.....	49
3.7.2 Установка Grafana.....	50
3.7.3 Установка node_exporter для нод Kubernetes	51
3.7.4 Установка node_exporter на сервер gateway.....	54
3.7.5 Установка Loki	55
3.7.6 Установка Promtail для нод Kubernetes	56
3.7.7 Установка Promtail на сервер gateway.....	60
3.8 Установка Hashicorp Vault (опционально).....	62
3.9 Установка Helm.....	65
3.9.1 Установка утилиты kube-ops-view	66
3.10 Установка GitLab	68
3.10.1 Установка	68
3.10.2 Добавление пользователя.....	70
3.10.3 Создание токена доступа для Git	71
3.10.4 Добавление Runner	73
3.10.5 Добавление внешних Runner	74
3.10.6 Наполняем локальный Docker контейнерами.....	75
3.10.7 Создаём тестовый проект в GitLab	75
3.10.8 Устанавливаем Golang.....	76
3.10.9 Создаём тестовый проект на Go	76
3.11 Установка MinIO	83
3.11.1 Добавляем отдельный диск для объектного хранилища	83
3.11.2 Устанавливаем MinIO.....	85
3.11.3 Настройка Keeralived.....	86
3.11.4 Проверка работоспособности	87
4 Kubernetes — отработка.....	89
4.1 Namespace	89
4.2 DNS.....	89
4.3 Node.....	90
4.4 События в Kubernetes	91
4.5 Pod.....	91
4.5.1 Простой под	91
4.5.2 Два контейнера в поде.....	92
4.5.3 Init Container в поде	93

4.6 ReplicaSet	95
4.6.1 Простой пример ReplicaSet	95
4.7 Deployment	97
4.7.1 Простой deploy	97
4.7.2 Deploy со стратегией	98
4.7.3 Создание через kubectl	100
4.7.4 Изменение числа реплик через kubectl	100
4.8 DaemonSet	100
4.9 Service	102
4.9.1 Простой service	103
4.9.2 Создание через kubectl для Deployment	104
4.9.3 Kubectl port-forward	106
4.10 Задачи на выполнение	106
4.10.1 Job	106
4.10.1.1 Простой job	107
4.10.2 CronJob	107
4.10.2.1 Простой CronJob	107
4.11 Admission Controller	108
4.11.1 ValidatingAdmissionWebhook	109
4.12 Передача данных в Pod	115
4.12.1 ENV	115
4.12.2 ConfigMap	116
4.12.2.1 Простейший ConfigMap	116
4.12.3 Secret	117
4.12.3.1 Простейший Secret	117
4.13 Аутентификация и авторизация	119
4.13.1 Аутентификация	119
4.13.1.1 Плагины аутентификации	119
4.13.1.2 Получение токена ServiceAccount и использование в прямом запросе к API	119
4.13.1.3 Создание сертификата X.509 для входа пользователем ...	122
4.13.1.4 ServiceAccount	122
4.13.2 Авторизация	123
4.13.2.1 RBAC	123
4.13.2.2 Определение текущего пользователя и контекста kubectl	123
4.13.2.3 Проверка прав для пользователя	124
4.13.2.4 Простой RBAC	124
4.13.2.5 Добавление сертификата пользователя в контекст kubectl	127
4.13.2.6 Встроенные кластерные роли	127

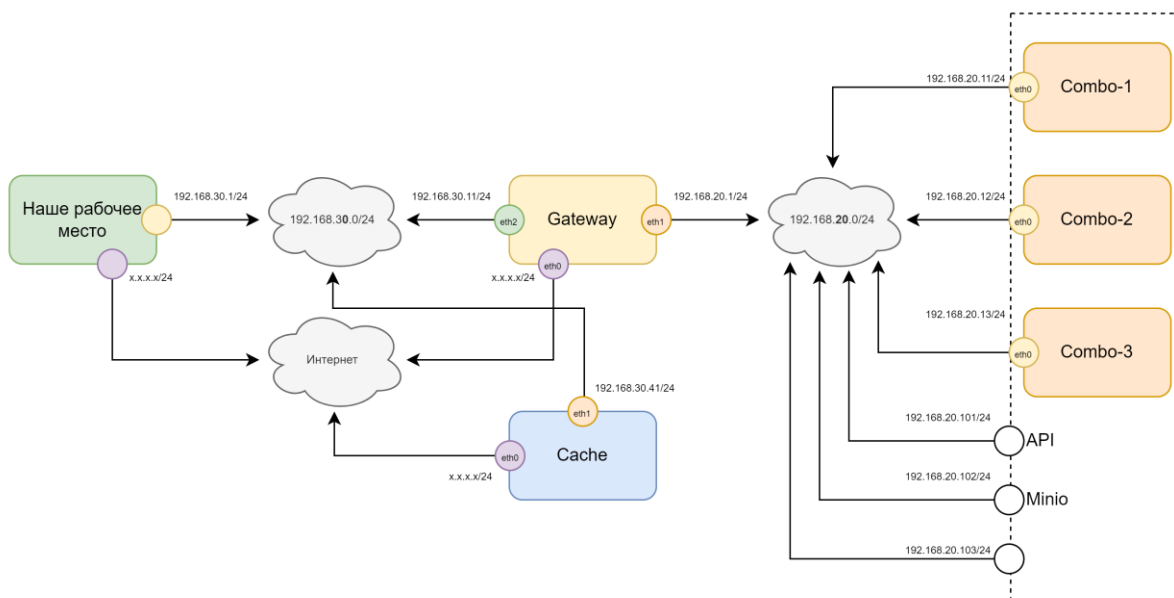
4.14 Linux Capabilities.....	128
4.14.1 Пример Capabilities.....	128
4.14.2 SecurityContext.....	129
4.14.3 Хакерский Pod.....	129
4.15 CNI.....	130
4.15.1 Исследование Flannel.....	131
4.15.2 NetworkPolicy.....	132
4.15.3 Сеть Docker.....	133
4.16 Ingress.....	134
4.16.1 Простой Ingress.....	134
4.16.2 IngressController.....	135
5 Дополнительный материал.....	139
5.1 Docker, ContainerD, CRI-O.....	139
5.2 Диагностике неисправностей в Kubernetes.....	139
5.3 Балансировка нагрузки.....	140
5.4 Helm.....	141

1 Вводные данные

1.1 Данные стенда

- хостовая ОС — **Windows 11 24H2** 26100.6899
- Нурег-V — версия **Hyper-V 10.0.26100.5074**
 - Виртуальные сетевые коммутаторы:
 - **Default Switch** — NAT-коммутатор по умолчанию, только для доступа в Интернет из виртуальной сети;
 - **External** — "Внешняя сеть" на основе установленного в компьютере физического LAN;
 - **Internal** — "Внутренняя сеть", которая доступна только хосту и виртуальной машине;
 - **Private** — "Частная сеть", которая доступна только виртуальным машинам между собой.
- ISO-образы:
 - ОС - **Ubuntu Live Server 24.04.3**
 - ISO образ — **ubuntu-24.04.3-live-server-amd64.iso**
 - MD5-хэш — 19fe5793f7411cfe124d9fa0c0f4d449
- SSH — **OpenSSH_for_Windows_9.5p1**, LibreSSL 3.8.2

1.2 Учётные данные хостов домена



- имя пользователя Linux — **administrator**
- пароль пользователя Linux — **Pa\$\$w0rd**
- имя домена — **dev.lan**
- наружная сеть **Internal** (с хостовой машиной) — **192.169.30.0/24**
- приватная сеть **Private** (с машинами Kubernetes) — **192.168.20.0/24**
- шлюз — **gateway** (gateway.dev.lan)
 - Имя виртуальной машины — **kube-gateway**
 - IP (Private) — **eth0, 192.168.20.1**
 - Шлюз
 - DNS
 - IP (Internal) — **eth1, 192.168.30.11**
 - SSH
 - IP (Default Switch) — **eth2, DHCP**
- кластер Kubernetes API для Control-plane (виртуальный IP для kube-apiserver):
 - IP — **192.168.20.101**
 - DNS — **k8s-control.dev.lan**
- комбинированный 1:
 - Имя виртуальной машины — **kube-combo-1**
 - IP — **eth0, 192.168.20.11**
 - DNS — **combo-1.dev.lan**
 - Назначение —

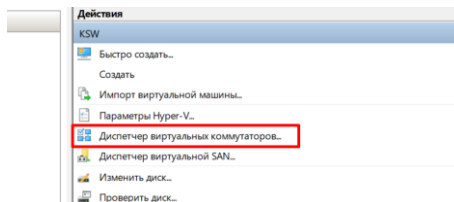
- комбинированный 2:
 - Имя виртуальной машины — **kube-combo-2**
 - IP — **eth0, 192.168.20.12**
 - DNS — **combo-2.dev.lan**
- комбинированный 3:
 - Имя виртуальной машины — **kube-combo-3**
 - IP — **eth0, 192.168.20.13**
 - DNS — **combo-3.dev.lan**

2 Создание виртуальных машин (STEP-0)

Под каждый из серверов создаём виртуальную машину. Создаём их автоматически, через инструмент Packer.

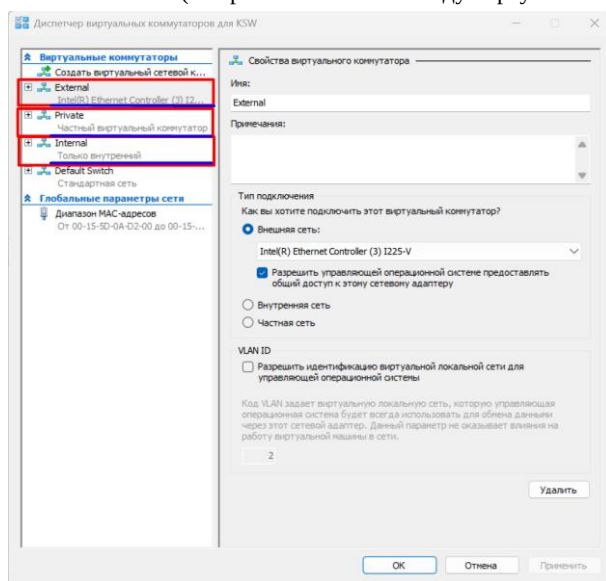
2.1 Создание виртуальных сетевых коммутаторов

Создаём набор виртуальных сетевых коммутаторов. Для этого в правой области **Диспетчера Hyper-V** нажимаем на пункт **Диспетчер виртуальных коммутаторов**:



Создаём 3 виртуальных сетевых коммутатора (помимо уже существующего **Default Switch**), назвав их соответственно:

- **External** — который подключаем к физическому оборудованию **Внешняя сеть** (указав LAN или WiFi адаптер);
- **Internal** — который подключаем к **Внутренней сети** (она не выходит за пределы хоста);
- **Private** — который подключён к **Частной сети** (она работает только между виртуальных машин).



2.2 Создание локального DNS-резолвера (на основе hosts)

Чтобы начать работать по доменным именам, а не по IP-адресам, вводим первоначальные FQDN имена виртуальных машин в хостовую машину. Для этого **открываем hosts файл** и в конец файла добавляем все имена хостов, указанных в CoreDNS:

```
notepad C:\Windows\System32\drivers\etc\hosts

192.168.30.11 gateway.dev.lan
192.168.20.11 combo-1.dev.lan
192.168.20.12 combo-2.dev.lan
192.168.20.13 combo-3.dev.lan
```

2.3 Устанавливаем утилиту Packer

Скачиваем Packer с зеркала https://hashicorp-releases.mcs.mail.ru/packer/1.14.2/packer_1.14.2_windows_amd64.zip (т.к. официальный сайт заблокирован для РФ);

Создаём директорию C:\packer и распаковываем утилиту packer.exe в неё. Добавляем путь к директории C:\packer в переменную окружения PATH, чтобы программу можно было запустить из любого места;

Проверяем работу программы, вызвав консоль CMD и запустив:

```
packer --version

Packer v1.14.2
```

Устанавливаем в Packer плагин для работы с Hyper-V:

```
packer plugins install github.com/hashicorp/hyperv

C:\Users\kirill\AppData\Roaming\packer.d\plugins\github.com\hashicorp\hyperv\packer-plugin-hyperv_v1.1.4_x5.0_windows_amd64.exe
```

Проверяем установленные плагины:

```
packer plugins installed

C:\Users\kirill\AppData\Roaming\packer.d\plugins\github.com\hashicorp\hyperv\packer-plugin-hyperv_v1.1.5_x5.0_windows_amd64.exe
```

2.4 Подготавливаем рабочую директорию и собираем данные для работы Packer

Создаём рабочую директорию E:\Packer.

Скачиваем с официального сайта <https://ubuntu.com/download/server> ISO-образ дистрибутива ubuntu-24.04.3-live-server-amd64.iso. Помещаем его в директорию E:\Packer.

Заходим в консоль **PowerShell** под **Администратором** и получаем SHA256 хэш со скачанного дистрибутива (он понадобится для конфигурации Packer):

```
Get-FileHash E:\Packer\ubuntu-24.04.3-live-server-amd64.iso

Algorithm Hash Path
-----
SHA256 C3514BF0056180D09376462A7A1B4F213C1D6E8EA67FAE5C25099C6FD3D8274B E:\Packer\ubuntu-24.04.3-live-server-amd64.iso
```

Проверяем список виртуальный сетевых коммутаторов Hyper-V. Если он отличается от указанного ниже (в столбцах Name и SwitchType), корректируем их:

```
Get-VMSwitch | Format-Table

Name SwitchType NetAdapterInterfaceDescription
----
Internal Internal
Private Private
External External Realtek PCIe GbE Family Controller
Default Switch Internal
```

Генерируем хэш от пароля **Pa\$\$w0rd** для будущего пользователя (или использовать уже готовый, который в этом примере ниже). Для этого нужно воспользоваться WSL2 в Windows, установив утилиту Whois и запустив команду генерации хэша от пароля:

```
wsl
sudo apt install whois
mkpasswd --method=SHA-512

Password: Pa$$w0rd
$6$unxT/qvU8DxvVwtt$FowKUjwoCIEHKd1qXYQh0Yo4ohKPeqNx0HGHa/bTCuAVsZYJ.RmjY4QZrRxwt2t/8pVHgaPZc5L3jLz4Fnkfc.
```

Где в результате:

- **\$6\$** — метод SHA-512
- **unxT/qvU8DxvVwtt** — соль (до следующего \$)
- **FowKUjwoCIEHKd1qXYQh0Yo4ohKPeqNx0HGHa/bTCuAVsZYJ.RmjY4QZrRxwt2t/8pVHgaPZc5L3jLz4Fnkfc.** — хэш пароля (после \$).

Проверяем, что пароль верный, повторив процедуру с указанием той же самой "соли". Мы должны получить тот же самый хэш:

```
mkpasswd --method=SHA-512 --salt=unxT/qvU8DxvVwtt

Password: Pa$$w0rd
$6$unxT/qvU8DxvVwtt$FowKUjwoCIEHKd1qXYQh0Yo4ohKPeqNx0HGHa/bTCuAVsZYJ.RmjY4QZrRxwt2t/8pVHgaPZc5L3jLz4Fnkfc.
```

Получаем текущую рабочую директорию для размещения виртуальных машин Hyper-V:

```
$vmRoot = (Get-CimInstance -Namespace "root\virtualization\v2" -ClassName "Msvm_VirtualSystemManagementServiceSettingData").DefaultExternalDataRoot
Echo $vmRoot

E:\VirtualMachines\Hyper-V\
```

2.5 Создаём описание установки cloud-init для Ubuntu

Ubuntu будет устанавливаться через получение специального YAML сценария из файла `user-data`. В нём будут описаны все основные настройки. Дальнейшие, индивидуальные настройки для каждого хоста будут установлены в Packer.

Принцип установки Ubuntu такой:

Packer создаёт HTTP-сервер, через который раздаёт файл `user-data`. Затем он создаёт виртуальную машину через API Hyper-V PowerShell, подключает ISO-образ и запускает её. Через API Hyper-V (через передачу «нажатий клавиш») в загрузчике указывает свой HTTP-сервер (через единственный созданный интерфейс) и запускает процесс установки. Сам он переходит в режим ожидания (через API Hyper-V), когда Ubuntu установится, перезагрузится, получит по внутреннему DHCP IP-адрес. Как только Packer это увидел, он заходит на сервер по SSH, создаёт во временной папке `sh`-файл с теми командами, которыми нужно добить настройку сервера и выполняют этот скрипт. После этого виртуальная машина завершает работу и файлы, из которых она состоит переносятся в директорию, где установлены все виртуальные машины.

В папке `E:\Packer` создаём поддиректорию `http`, в котором создаём файл `user-data`. Структура должна получиться такая:

```
Packer
├── http
│   └── user-data
├── kube-combo-1.pkrvars.hcl
├── kube-combo-2.pkrvars.hcl
├── kube-combo-3.pkrvars.hcl
├── kube-gateway.pkrvars.hcl
├── ubuntu-hyperv.pkr.hcl
└── ubuntu-24.04.3-live-server-amd64.iso
```

Заполняем файл `user-data`:

notepad E:\Packer\http\user-data

```
#cloud-config
# See the autoinstall documentation at:
# https://canonical-subiquity.readthedocs-hosted.com/en/latest/reference/autoinstall-reference.html
autoinstall:
  version: 1
  apt:
    disable_components: [] # не отключать никакие компоненты
    preserve_sources_list: true # не перезаписывать sources.list
  packages:
    # Сервер SSH
    - openssh-server
    # Эти пакеты обязательны, чтобы Packer через API Hyper-V определил IP-адрес созданной
    # виртуальной машины через интерфейсы KVP или VMBus (внутренний канал Hyper-V между гостевой ОС и хостом):
    - linux-cloud-tools-common
    - linux-cloud-tools-generic
    # Утилиты для манипуляции конфигурационными файлами формата JSON и YAML
    - jq
    - yq
  late_commands:
    # Обязателен, чтобы sudo не требовал пароля при автоматическим использованием Packer:
    - echo 'administrator ALL=(ALL) NOPASSWD:ALL' > /target/etc/sudoers.d/administrator
  codecs:
    install: false
  drivers:
    install: false
  identity:
    hostname: ubuntu
    # Зашифрованный пароль `Pa$$w0rd`
    password: $6$unxT/qVU8DxvVWtt$FowKujwoCIEHKd1qYQh0Yo4ohKPeqNx0HGHa/bTCuAVsZYJ.RmjY4QZrXwt2t/8pVHgaPZc5L3jlz4Fnkfc.
    realname: Administrator
    username: administrator
  kernel:
    package: linux-generic
  keyboard:
    layout: us,ru
    toggle: alt_shift_toggle
    variant: ", "
  locale: ru_RU.UTF-8
  network:
    version: 2
    ethernets:
      eth0:
        dhcp4: true
  oem:
    install: auto
  source:
    id: ubuntu-server
    search_drivers: false
  ssh:
    allow-pw: true
    authorized-keys: []
    install-server: true
# Разметить диск автоматически в LVM.
storage:
  layout:
    name: lvm
updates: security
```

2.6 Создаём описание виртуальных машин на языке HCL для Packer

В директории `E:\Packer` создаём конфигурационный файл `ubuntu-hyperv.pkr.hcl` с общим описанием виртуальной машины. К нему мы будем применять другие файлы конфигурации с конкретными настройками для конкретных типов машин:

При работе из VSCode можно установить плагин с ID: `4ops.packer`

notepad E:\Packer\ubuntu-hyperv.pkr.hcl

```
packer {
  required_plugins {
    hyperv = {
      version = ">= 1.1.5" # Версия плагина Hyper-V (смотреть через `packer plugins installed`)
      source  = "github.com/hashicorp/hyperv"
    }
  }
}
```

```

# Не изменяемые локальные переменные. Доступен через local.
locals {
  iso_url = "ubuntu-24.04.3-live-server-amd64.iso" # Путь и имя ISO
  iso_checksum = "sha256:C3514BF0056180D09376462A7A1B4F213C1D6E8EA67FAE5C25099C6FD3D8274B" # SHA-256 ISO
  vm_dir = "E:\VirtualMachines\Hyper-V" # Путь к директории с виртуальными машинами Hyper-V
}

# Изменяемая переменная, доступная через var. Имя хоста
variable "vm_hostname" {
  type = string
  default = "kube" # Значение по умолчанию. Изменяется в pkrvars.hcl файлах.
}

# Изменяемая переменная, доступная через var. Размер диска в мегабайтах
variable "disk_size" {
  type = number
  default = 10000 # Значение по умолчанию. Изменяется в pkrvars.hcl файлах.
}

# Изменяемая переменная, доступная через var. Колличество CPU
variable "cpus" {
  type = number
  default = 1 # Значение по умолчанию. Изменяется в pkrvars.hcl файлах.
}

# Изменяемая переменная, доступная через var. Количество памяти в мегабайтах
variable "memory" {
  type = number
  default = 1024 # Значение по умолчанию. Изменяется в pkrvars.hcl файлах.
}

# Список индивидуальных команд для каждого сервера. По умолчанию заполнен пустышкой, т.к. нельзя оставлять пустым список.
variable "inline" {
  type = list(string)
  default = [
    "uname -a",
  ]
}

source "hyperv-iso" "ubuntu_server" {
  # Настройки ISO
  iso_url = local.iso_url
  iso_checksum = local.iso_checksum

  # Настройки виртуальной машины
  disk_size = var.disk_size # Размер диска
  cpus = var.cpus # CPU
  memory = var.memory # RAM
  generation = 2 # Hyper-V Gen 2 (UEFI)
  boot_wait = "5s"
  shutdown_command = "sudo shutdown -P now" # Вариант без пароля

  http_directory = "http" # Папка с user-data для автоматической установки Ubuntu

  switch_name = "Default Switch" # Создать подключение к NAT сетевому коммутатору

  ssh_username = "administrator" # Имя пользователя (должно совпадать с user-data)
  ssh_password = "Pa$$w0rd" # Пароль (можно заменить на свой)
  ssh_timeout = "30m" # Таймаут подключения
  ssh_port = 22

  # Настройки нажатий клавиш для передачи в загрузчик (для автоматической установки)
  boot_command = [
    "c<wait>", # Переход в командную строку GRUB
    "linux /casper/vmlinuz autoinstall cloud-config-url=http://{{.HTTPIP}}:{{.HTTTPort}}/user-data ds=nocloud-net;s=/cdrom/",
    " ---<enter>",
    "initrd /casper/initrd<enter>",
    "boot<enter>"
  ]

  # Настройка образа на выходе
  vm_name = "${var.vm_hostname}" # Имя будущей виртуальной машины
  output_directory = "${local.vm_dir}/${var.vm_hostname}" # Папка с виртуальной машины (диск, описание)
  guest_additions_mode = "disable" # Не использовать Guest Addon
}

build {
  sources = ["source.hyperv-iso.ubuntu_server"]

  # Общие настройки для всех создаваемых виртуальных машин.
  # Провижининг (установка пакетов, настройка, после создания и включения виртуальной машины)
  # После его отработки, виртуальная машина будет записана как результат на диск в output_directory
  provisioner "shell" {
    # Вариант без пароля (т.к. обёрнут в sudo), где:
    # {{.Path }} – Полный путь к временному скрипту на целевой машине (например, /tmp/script_12345.sh)
    # {{.Vars }} – строка вида VAR1=value1 VAR2=value2 ..., содержащая переменные окружения, которые Packer хочет передать в скрипт.
    # В результате будет что то типа: sudo sh -c 'PACKER_BUILDER_TYPE=hyperv-iso PACKER_BUILD_NAME=ubuntu_server /tmp/packer-shell12345.sh'
    # В файл /tmp/packer-shell12345.sh будут завернуты все команды из inline.
    # execute_command = "sudo sh -c '{{.Vars }} {{.Path }}'"
    execute_command = "sh -c '{{.Vars }} {{.Path }}'"
    inline = [
      "sudo apt update",
      "sudo apt purge -y unattended-upgrades", # Удаляем сервис обновления дистрибутива
      "sudo rm -rf /var/log/unattended-upgrades",
      # Дополнительные утилиты
      "sudo apt install -y bash-completion vim iputils-ping dnsutils traceroute curl tree htop gdu jq yq unzip tmux lsof html2text python3.12-venv",
      "echo 'set -g mouse on' > ~/.tmux.conf", # Настройка tmux
      "mkdir -p ~/.config/htop",
      "echo 'fields=0 48 50 17 18 38 39 40 2 46 47 49 1\nhide_userland_threads=1\nntree_view=1\nscreen:Main=PID USER NLWP PRIORITY NICE M_VIRT M_RESIDENT M_SHARE STATE\nPERCENT_CPU PERCENT_MEM TIME Command\n.tree_view=1' > ~/.config/htop/htoprc", # Настройка htop
      # Пользовательские alias ls
      "echo 'alias sudo='sudo 'alias l='ls -ahF --group-directories-first'\nalias ll='ls -lhF --group-directories-first'\nalias la='ls -lhF --group-directories-first'" > ~/.bash_aliases",
      "sudo cp ~/.bash_aliases /etc/skel/",
      # Установка Docker
      "curl -fsSL https://download.docker.com/linux/debian/gpg | sudo tee /etc/apt/keyrings/docker.asc", # Добавление репозитория (ключ и адрес)
      "echo 'deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian bullseye stable' | sudo tee /etc/apt/sources.list.d/docker.list",
      "sudo apt update",
      "sudo apt install -y docker-ce=5:28.* docker-ce-cli=5:28.* docker-buildx-plugin=0.28.* docker-compose-plugin=2.40.*", # Установка дистрибутива
      "sudo systemctl restart docker",
      "sudo usermod -aG docker administrator", # Добавляем пользователя administrator в группу Docker
      "echo '{\"features\": {\"buildkit\": true}}' | sudo tee /etc/docker/daemon.json", # Устанавливаем работу через buildx по умолчанию
      # Часовой пояс
      "sudo timedatectl set-timezone Europe/Moscow",
      # Удаление swap
      "sudo swapoff /swap.img",
      "sudo rm /swap.img",
      "sudo sed -i 's|/swap.img/d|' /etc/fstab",
      # Установка iptables
      "echo 'iptables-persistent iptables-persistent/autosave_v4 boolean true' | sudo debconf-set-selections", # Ответы для установки
      "echo 'iptables-persistent iptables-persistent/autosave_v6 boolean true' | sudo debconf-set-selections", # Ответы для установки
      "sudo apt install -y iptables iptables-persistent",
    ]
  }

  # Скрипты из внешнего файла pkrvars
  provisioner "shell" {
    execute_command = "sh -c '{{.Vars }} {{.Path }}'"
    inline = var.inline
  }
}

```

```
# Завершающие скрипты
provisioner "shell" {
  execute_command = "sh -c '{{ .Vars }} {{ .Path }}'"
  inline = [
    # Очищаем кэш apt
    "sudo apt autoremove -y",
    "sudo apt clean"
  ]
}
```

Создаём файлы настроек для создаваемых серверов. Создаём настройки для сервера **kube-gateway**:

```
notepad E:\Packer\kube-gateway.pkrvars.hcl
```

```
vm_hostname = "kube-gateway" # Имя виртуальной машины Hyper-V
disk_size = 200000 # Общий размер диска (в мегабайтах)
cpus = 4 # Колличество CPU
memory = 9216 # Размер памяти (в мегабайтах)
inline = [
    # Изменение конфигурации сетевых интерфейсов:
    "sudo yq -i -y '.network.ethernets *='",
    "{\"eth1\":{\"addresses\":[\"192.168.20.1/24\"],\"dhcp4\":false},\"eth2\":{\"addresses\":[\"192.168.30.11/24\"],\"dhcp4\":false}}",
    "/etc/netplan/50-cloud-init.yaml",
    # Смена имени хоста:
    "echo 'gateway' | sudo tee /etc/hostname",
    "sudo hostnamectl set-hostname gateway",
    "sudo sed -i 's/127.0.1.1.*127.0.1.1 gateway/' /etc/hosts",
]
```

Создаём настройки для сервера **kube-combo-1**:

```
notepad E:\Packer\kube-combo-1.pkrvars.hcl
```

```
vm_hostname = "kube-combo-1" # Имя виртуальной машины Hyper-V
disk_size = 100000 # Общий размер диска (в мегабайтах)
cpus = 2 # Количество CPU
memory = 3076 # Размер памяти (в мегабайтах)
inline = [
    # Изменение конфигурации сетевых интерфейсов:
    "sudo yq -i -y '.network.etherternets.eth0 = {\"dhcp4\": false, \"addresses\": [\"192.168.20.11/24\"], \"routes\": [{\"to\": \"default\", \"via\": \"192.168.20.1\"}], \"nameservers\": {\"addresses\": [\"192.168.20.1\"], \"search\": [\"dev.lan\"]}}' /etc/netplan/50-cloud-init.yaml",
    # Смена имени хоста:
    "echo 'combo-1' | sudo tee /etc/hostname",
    "sudo hostnamectl set-hostname combo-1",
    "sudo sed -i 's/127.0.1.1.*127.0.1.1 combo-1/' /etc/hosts",
    # Изменение адреса кша deb-пакетов:
    "#echo 'Acquire::http:Proxy \"http://192.168.20.41:3142\";' | sudo tee /etc/apt/apt.conf.d/90curtin-aptproxy > /dev/null",
]
```

Создаём настройки для сервера **kube-combo-2**:

```
notepad E:\Packer\kube-combo-2.pkrvars.hcl
```

```
vm_hostname = "kube-combo-2" # Имя виртуальной машины Hyper-V
disk_size = 100000 # Общий размер диска (в мегабайтах)
cpus = 2 # Количество CPU
memory = 3076 # Размер памяти (в мегабайтах)
inline = [
    # Изменение конфигурации сетевых интерфейсов:
    "sudo yq -i -y '.network.ethernets.eth0 = {\"dhcp4\": false, \"addresses\": [\"192.168.20.12/24\"], \"routes\": [{\"to\": \"default\", \"via\": \"192.168.20.1\"}], \"nameservers\": {\"addresses\": [\"192.168.20.1\"], \"search\": [\"dev.lan\"]}}' /etc/netplan/50-cloud-init.yaml",
    # Смена имени хоста:
    "echo 'combo-2' | sudo tee /etc/hostname",
    "sudo hostnamectl set-hostname combo-2",
    "sudo sed -i 's/127.0.1.1.*127.0.1.1 combo-2/' /etc/hosts",
    # Изменение адреса кша deb-пакетов:
    "#echo 'Acquire::http:Proxy \"http://192.168.20.41:3142\";' | sudo tee /etc/apt/apt.conf.d/90curtin-aptproxy > /dev/null",
]
```

Создаём настройки для сервера **kube-combo-3**:

```
notepad E:\Packer\kube-combo-3.pkrvars.hcl
```

```
vm_hostname = "kube-combo-3" # Имя виртуальной машины Hyper-V
disk_size = 100000 # Общий размер диска (в мегабайтах)
cpus = 2 # Колличество CPU
memory = 3076 # Размер памяти (в мегабайтах)
inline = [
    # Изменение конфигурации сетевых интерфейсов:
    "sudo yq -i -y '.network.ethernets.eth0 = {\"dhcp4\": false, \"addresses\": [\"192.168.20.13/24\"], \"routes\": [{\"to\": \"default\", \"via\": \"192.168.20.1\"}], \"nameservers\": {\"addresses\": [\"192.168.20.1\"], \"search\": [\"dev.lan\"]}}' /etc/netplan/50-cloud-init.yaml",
    # Смена имени хоста:
    "echo 'combo-3' | sudo tee /etc/hostname",
    "sudo hostnamectl set-hostname combo-3",
    "sudo sed -i 's/127.0.1.1.* /127.0.1.1 combo-3/' /etc/hosts",
    # Изменение адреса кэша deb-пакетов:
    "# echo 'Acquire::http::Proxy \"http://192.168.20.41:3142\";' | sudo tee /etc/apt/apt.conf.d/90curtin-aptproxy > /dev/null",
]
```

2.7 Запускаем создание виртуальных машин через Packer

2.7.1 Подготовка

Запускаем терминал **PowerShell** в режиме **Администратора** и заходим в рабочую директорию:

```
e:
cd Packer
```

Запускаем процесс инициализации проекта:

```
packer init .
```

Запускаем валидацию конфигурации (не обращаем внимание на предупреждение):

```
packer validate .
```

```
Warning: Hyper-V might fail to create a VM if there is not enough free memory in the system.

on ubuntu-hyperv.pkr.hcl line 52:
(source code not available)

The configuration is valid.
```

2.7.2 Сервер kube-gateway

Запускаем сборку сервера **kube-gateway** в **PowerShell** под **Администратором**:

```
# B CMD:
# packer build -var-file=kube-gateway.pkrvars.hcl ubuntu-hyperv.pkr.hcl
# B PowerShell:
cmd /c "packer build -var-file=kube-gateway.pkrvars.hcl ubuntu-hyperv.pkr.hcl"

Warning: Hyper-V might fail to create a VM if there is not enough free memory in the system.

on ubuntu-hyperv.pkr.hcl line 41:
(source code not available)

hyperv-iso.ubuntu_server: output will be in this color.

==> hyperv-iso.ubuntu_server: Creating build directory...
==> hyperv-iso.ubuntu_server: Retrieving ISO
==> hyperv-iso.ubuntu_server: Trying ubuntu-24.04.3-live-server-amd64.iso
==> hyperv-iso.ubuntu_server: Trying ubuntu-24.04.3-live-server-amd64.iso?checksum=sha256%3Ac3514bf0056180d09376462a7a1b4f213c1d6e8ea67fae5c25099c6fd3d8274b
==> hyperv-iso.ubuntu_server: ubuntu-24.04.3-live-server-amd64.iso?checksum=sha256%3Ac3514bf0056180d09376462a7a1b4f213c1d6e8ea67fae5c25099c6fd3d8274b => E:/Packer/ubuntu-24.04.3-live-server-amd64.iso
==> hyperv-iso.ubuntu_server: Starting HTTP server on port 8486
==> hyperv-iso.ubuntu_server: Creating switch 'Default Switch' if required...
==> hyperv-iso.ubuntu_server: Switch 'Default Switch' already exists. Will not delete on cleanup...
==> hyperv-iso.ubuntu_server: Creating virtual machine...
==> hyperv-iso.ubuntu_server: Enabling Integration Service...
==> hyperv-iso.ubuntu_server: Setting boot drive to os dvd drive E:/Packer/ubuntu-24.04.3-live-server-amd64.iso ...
==> hyperv-iso.ubuntu_server: Mounting os dvd drive E:/Packer/ubuntu-24.04.3-live-server-amd64.iso ...
==> hyperv-iso.ubuntu_server: Skipping mounting Integration Services Setup Disk...
==> hyperv-iso.ubuntu_server: Mounting secondary DVD images...
==> hyperv-iso.ubuntu_server: Configuring vlan...
==> hyperv-iso.ubuntu_server: Determine Host IP for HyperV machine...
==> hyperv-iso.ubuntu_server: Host IP for the HyperV machine: 172.20.192.1
==> hyperv-iso.ubuntu_server: Attempting to connect with vmconnect...
==> hyperv-iso.ubuntu_server: Starting the virtual machine...
==> hyperv-iso.ubuntu_server: Waiting 5s for boot...
==> hyperv-iso.ubuntu_server: Typing the boot command...
==> hyperv-iso.ubuntu_server: Waiting for SSH to become available...
==> hyperv-iso.ubuntu_server: Connected to SSH!
...
==> hyperv-iso.ubuntu_server: kube-gateway
==> hyperv-iso.ubuntu_server: Gracefully halting virtual machine...
==> hyperv-iso.ubuntu_server: Waiting for vm to be powered down...
==> hyperv-iso.ubuntu_server: Unmount/delete secondary dvd drives...
==> hyperv-iso.ubuntu_server: Unmount/delete Integration Services dvd drive...
==> hyperv-iso.ubuntu_server: Unmount/delete os dvd drive...
==> hyperv-iso.ubuntu_server: Delete os dvd drives controller 0 location 1 ...
==> hyperv-iso.ubuntu_server: Compacting disks...
==> hyperv-iso.ubuntu_server: Compacting disk: kube-gateway.vhdx
==> hyperv-iso.ubuntu_server: Disk size is unchanged
==> hyperv-iso.ubuntu_server: Exporting virtual machine...
==> hyperv-iso.ubuntu_server: Collating build artifacts...
==> hyperv-iso.ubuntu_server: Disconnecting from vmconnect...
==> hyperv-iso.ubuntu_server: Unregistering and deleting virtual machine...
==> hyperv-iso.ubuntu_server: Deleting build directory...
Build 'hyperv-iso.ubuntu_server' finished after 4 minutes 58 seconds.

==> Wait completed after 4 minutes 58 seconds

==> Builds finished. The artifacts of successful builds are:
--> hyperv-iso.ubuntu_server: VM files in directory: E:/VirtualMachines/Hyper-V/kube-gateway
```

Регистрируем виртуальную машину в **Hyper-V** (т.к. созданы только файлы, сам **Hyper-V** ещё об этой машине ничего не знает):

```
$vmName = "kube-gateway" # Имя виртуальной машины (не имя хоста)

# Получаем абсолютный путь к файлу *.vmcx` (он всегда имеет произвольное имя, например D48F167D-453D-45E4-9A7B-398C56AF3995.vmcx)
$vmcxFile = Get-ChildItem -Path "E:/VirtualMachines/Hyper-V/$vmName/Virtual Machines" -Filter *.vmcx | Select-Object -First 1
# Импортируем виртуальную машину
Import-VM -Path $vmcxFile.FullName -Register

Name State CPUUsage(%) MemoryAssigned(M) Uptime Status Version
----
kube-gateway Off 0 0 00:00:00 Работает нормально 12.0
```

Добавляем новые сетевые адаптеры и перенастраиваем уже существующий:

```
$vmName = "kube-gateway" # Имя виртуальной машины (не имя хоста)

# Создаём ещё 2 адаптера:
Add-VMNetworkAdapter -VMName $vmName -SwitchName "Private"
Add-VMNetworkAdapter -VMName $vmName -SwitchName "Internal"
```

Включаем виртуальную машину, заходим на неё по **SSH** командой:

```
$vmName = "kube-gateway" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

Для дальнейшей настройки по **SSH** заходим на сервер командой:

```
# Удаляем ключи, если хост пересоздавался: ssh-keygen -R gateway.dev.lan
ssh-copy-id administrator@gateway.dev.lan
ssh administrator@gateway.dev.lan

sudo shutdown -h now
```

Делаем снимок виртуальной машины с именем «**STEP-0 (Создан через Packer)**».

Снова включаем виртуальную машину, чтобы через неё подключаться к серверам kube-combo-1, kube-combo-2, kube-combo-3.

```
$vmName = "kube-gateway" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

Для удобства входа через на сервера kube-combo-1, kube-combo-2, kube-combo-3 через сервер kube-gateway без его упоминания в команде ssh через параметр -J, можно создать config файл по пути C:\Users\<Пользователь>\.ssh\config следующего содержания:

```
Host gateway.dev.lan
```

```
HostName gateway.dev.lan
User administrator

Host combo-1.dev.lan
  HostName combo-1.dev.lan
  User administrator
  ProxyJump gateway.dev.lan

Host combo-2.dev.lan
  HostName combo-2.dev.lan
  User administrator
  ProxyJump gateway.dev.lan

Host combo-3.dev.lan
  HostName combo-3.dev.lan
  User administrator
  ProxyJump gateway.dev.lan
```

Тогда к хостам можно обращаться таким образом:

```
ssh gateway.dev.lan
ssh combo-1.dev.lan
ssh combo-2.dev.lan
ssh combo-3.dev.lan
```

2.7.3 Сервер kube-combo-1

Запускаем сборку сервера **kube-combo-1** в **PowerShell** под **Администратором**:

```
# В CMD:
# packer build -var-file=kube-combo-1.pkrvars.hcl ubuntu-hyperv.pkr.hcl
# В PowerShell:
cmd /c "packer build -var-file=kube-combo-1.pkrvars.hcl ubuntu-hyperv.pkr.hcl"
```

Регистрируем сервер в Hyper-V:

```
$vmName = "kube-combo-1" # Имя виртуальной машины (не имя хоста)

# Получаем абсолютный путь к файлу `*.vmcx` (он всегда имеет произвольное имя, например D48F167D-453D-45E4-9A7B-398C56AF3995.vmcx)
$vmcxFile = Get-ChildItem -Path "E:\VirtualMachines\Hyper-V\$vmName\Virtual Machines" -Filter *.vmcx | Select-Object -First 1
# Импортируем виртуальную машину
Import-VM -Path $vmcxFile.FullName -Register
```

Name	State	CPUUsage(%)	MemoryAssigned(M)	Uptime	Status	Version
kube-combo-1	Off	0	0	00:00:00	Работает нормально	12.0

Перенастраиваем существующий сетевой адаптер на сеть Private:

```
$vmName = "kube-combo-1" # Имя виртуальной машины (не имя хоста)

$adapters = Get-VMNetworkAdapter -VMName $vmName # Список сетевых адаптеров виртуальной машины
Connect-VMNetworkAdapter -VMNetworkAdapter $adapters[0] -SwitchName "Private" # Подключаем адаптер к коммутатору Private
```

Включаем виртуальную машину, заходим на неё по SSH командой:

```
$vmName = "kube-combo-1" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

Для дальнейшей настройки по SSH заходим на сервер командой:

```
# Удаляем ключи, если хост пересоздавался: ssh-keygen -R combo-1.dev.lan
ssh-copy-id administrator@combo-1.dev.lan administrator@gateway.dev.lan
ssh administrator@combo-1.dev.lan -J administrator@gateway.dev.lan

sudo shutdown -h now
```

Делаем снимок виртуальной машины с именем «STEP-0 (Создан через Packer)».

Снова включаем виртуальную машину:

```
$vmName = "kube-combo-1" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

2.7.4 Сервер kube-combo-2

Запускаем сборку сервера **kube-combo-2** в **PowerShell** под **Администратором**:

```
# В CMD:
# packer build -var-file=kube-combo-2.pkrvars.hcl ubuntu-hyperv.pkr.hcl
# В PowerShell:
cmd /c "packer build -var-file=kube-combo-2.pkrvars.hcl ubuntu-hyperv.pkr.hcl"
```

Регистрируем сервер в Hyper-V:

```
$vmName = "kube-combo-2" # Имя виртуальной машины (не имя хоста)

# Получаем абсолютный путь к файлу `*.vmcx` (он всегда имеет произвольное имя, например D48F167D-453D-45E4-9A7B-398C56AF3995.vmcx)
$vmcxFile = Get-ChildItem -Path "E:\VirtualMachines\Hyper-V\$vmName\Virtual Machines" -Filter *.vmcx | Select-Object -First 1
# Импортируем виртуальную машину
Import-VM -Path $vmcxFile.FullName -Register
```

Name	State	CPUUsage(%)	MemoryAssigned(M)	Uptime	Status	Version
kube-combo-2	Off	0	0	00:00:00	Работает нормально	12.0

Перенастраиваем существующий сетевой адаптер на сеть Private:

```
$vmName = "kube-combo-2" # Имя виртуальной машины (не имя хоста)

$adapters = Get-VMNetworkAdapter -VMName $vmName # Список сетевых адаптеров виртуальной машины
Connect-VMNetworkAdapter -VMNetworkAdapter $adapters[0] -SwitchName "Private" # Подключаем адаптер к коммутатору Private
```

Включаем виртуальную машину, заходим на неё по SSH командой:

```
$vmName = "kube-combo-2" # Имя виртуальной машины (не имя хоста)
```

```
Start-VM -Name $vmName # Включаем виртуальную машину
```

Для дальнейшей настройки по SSH заходим на сервер командой:

```
# Удаляем ключи, если хост пересоздавался: ssh-keygen -R combo-2.dev.lan
ssh-copy-id administrator@combo-2.dev.lan administrator@gateway.dev.lan
ssh administrator@combo-2.dev.lan -J administrator@gateway.dev.lan

sudo shutdown -h now
```

Делаем снимок виртуальной машины с именем «STEP-0 (Создан через Packer)».

Снова включаем виртуальную машину:

```
$vmName = "kube-combo-2" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

2.7.5 Сервер kube-combo-3

Запускаем сборку сервера kube-combo-3 в PowerShell под Администратором:

```
# В CMD:
# packer build -var-file=kube-combo-3.pkrvars.hcl ubuntu-hyperv.pkr.hcl
# В PowerShell:
cmd /c "packer build -var-file=kube-combo-3.pkrvars.hcl ubuntu-hyperv.pkr.hcl"
```

Регистрируем сервер в Hyper-V:

```
$vmName = "kube-combo-3" # Имя виртуальной машины (не имя хоста)

# Получаем абсолютный путь к файлу `*.vmcx` (он всегда имеет произвольное имя, например D48F167D-453D-45E4-9A7B-398C56AF3995.vmcx)
$vmcxFile = Get-ChildItem -Path "E:/VirtualMachines/Hyper-V/$vmName/Virtual Machines" -Filter *.vmcx | Select-Object -First 1
# Импортируем виртуальную машину
Import-VM -Path $vmcxFile.FullName -Register
```

Name	State	CPUUsage(%)	MemoryAssigned(M)	Uptime	Status	Version
kube-combo-3	Off	0	0	00:00:00	Работает нормально	12.0

Перенастраиваем существующий сетевой адаптер на сеть Private:

```
$vmName = "kube-combo-3" # Имя виртуальной машины (не имя хоста)

$adapters = Get-VMNetworkAdapter -VMName $vmName # Список сетевых адаптеров виртуальной машины
Connect-VMNetworkAdapter -VMNetworkAdapter $adapters[0] -SwitchName "Private" # Подключаем адаптер к коммутатору Private
```

Включаем виртуальную машину, заходим на неё по SSH командой:

```
$vmName = "kube-combo-3" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

Для дальнейшей настройки по SSH заходим на сервер командой:

```
# Удаляем ключи, если хост пересоздавался: ssh-keygen -R combo-3.dev.lan
ssh-copy-id administrator@combo-3.dev.lan administrator@gateway.dev.lan
ssh administrator@combo-3.dev.lan -J administrator@gateway.dev.lan

sudo shutdown -h now
```

Делаем снимок виртуальной машины с именем «STEP-0 (Создан через Packer)».

Снова включаем виртуальную машину:

```
$vmName = "kube-combo-3" # Имя виртуальной машины (не имя хоста)

Start-VM -Name $vmName # Включаем виртуальную машину
```

2.7.6 Результат

Все сервера должны появиться в Диспетчере Hyper-V:

 kube-combo-1	Работает	0%
 kube-combo-2	Работает	0%
 kube-combo-3	Работает	0%
 kube-gateway	Работает	0%

3 Настройка виртуальных машин

3.1 Установка CoreDNS

GitHub — <https://github.com/coredns/coredns>

Устанавливаем DNS сервер первым, чтобы начинать общаться в локальной сети только по DNS именам. **Устанавливаем CoreDNS:**

```
cd ~
wget -P /tmp https://github.com/coredns/coredns/releases/download/v1.13.1/coredns_1.13.1_linux_amd64.tgz
sudo tar -xzf /tmp/coredns_1.13.1_linux_amd64.tgz -C /usr/local/bin/ coredns
```

Проверяем работу:

```
| coredns -version
```

```
CoreDNS-1.13.1
linux/amd64, go1.25.2, 1db4568
```

Отключаем встроенный в Ubuntu Server systemd-resolved. Systemd-resolved уже занимает 53-й порт, предназначенный для DNS, что можно проверить:

```
| sudo ss -tulnp | grep ':53'
```

```
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:* users:((("systemd-resolve",pid=913,fd=16))
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=913,fd=14))
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:((("systemd-resolve",pid=913,fd=17))
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=913,fd=15))
```

Выгружаем Systemd-resolved и запрещаем загружаться автоматически:

```
sudo systemctl stop systemd-resolved
sudo systemctl disable systemd-resolved
```

Проверяем, что порт освободился (вывод должен быть пуст):

```
| sudo ss -tulnp | grep ':53'
```

Создаём конфигурацию CoreDNS, создав файл:

```
sudo mkdir -p /etc/coredns
sudo tee /etc/coredns/Corefile << 'EOF' > /dev/null
.:53 {
    # Обработать DNS-запросы на порту 53 для любой зоны (.)

    errors                # Включить вывод ошибок DNS
    health {              # Включить healthcheck-эндпоинт
        lameduck 5s       # При завершении работы ждать 5 сек перед остановкой
    }
    ready                 # Включить эндпоинт /ready для проверки готовности

    # Локальные записи для домена dev.lan
    hosts {               # Использовать статические записи из файла hosts
        192.168.20.1 gateway.dev.lan # Сопоставление IP с именами. Шлюз
        192.168.20.11 combo-1.dev.lan
        192.168.20.12 combo-2.dev.lan
        192.168.20.13 combo-3.dev.lan
        192.168.20.1 traefik.dev.lan # Traefik Dashboard
        192.168.20.1 whoami.dev.lan  # Whoami Test Container
        192.168.20.1 registry.dev.lan # Peetr Docker
        192.168.20.1 portainer.dev.lan # Portainer
        192.168.20.1 gitlab.dev.lan   # Gitlab
        192.168.20.1 ca.dev.lan       # Step CA, центр сертификации
        192.168.20.1 vault.dev.lan    # HashiCorp Vault
        192.168.20.1 code.dev.lan     # Code (VSCode server)
        192.168.20.1 prometheus.dev.lan # БД Prometheus
        192.168.20.1 grafana.dev.lan  # Grafana
        192.168.20.1 hello.dev.lan   # Демо-сайт
        192.168.20.101 control.dev.lan # Балансировщик keepalived для combo-1,2,3.dev.lan
        192.168.20.102 minio.dev.lan  # Сервер MinIO для combo-1,2,3.dev.lan
        fallthrough # Если имя не найдено, передать запрос следующему плагину
    }

    # Форвардинг всех остальных запросов к вышестоящему DNS (8.8.8.8)
    forward . 8.8.8.8 # Перенаправлять все запросы (.) на внешний DNS

    cache 30          # Кэшировать ответы в течение 30 секунд
    reload            # Позволить автоматическую перезагрузку конфига при изменениях
    log              # Включить логирование DNS-запросов
}
EOF
```

Создаём Linux пользователя **coredns**:

```
| sudo useradd -r -s /bin/false -M -d /etc/coredns coredns
```

где:

- **-r** — создать системного пользователя (без домашней директории).
- **-s /bin/false** — запретить вход в оболочку.
- **-M** — не создавать домашнюю директорию.
- **-d /etc/coredns** — рабочая директория (если нужна).
- **coredns** — имя пользователя и группы (группа создаётся автоматически).

Проверяем:

```
| id coredns
```

```
uid=999(coredns) gid=990(coredns) groups=990(coredns)
```

Устанавливаем права на запускной файл CoreDNS:

```
sudo chown root:coredns /usr/local/bin/coredns
sudo chmod 750 /usr/local/bin/coredns
```

Устанавливаем права на конфигурацию:

```
sudo chown -R coredns:coredns /etc/coredns
sudo chmod 640 /etc/coredns/Corefile
```

Создаём службу запуска CoreDNS:

```
sudo tee /etc/systemd/system/coredns.service << 'EOF' > /dev/null
[Unit]
# Описание сервиса (отображается в systemctl status)
Description=CoreDNS DNS Server

# Запускать CoreDNS только после того, как поднята сеть
After=network.target

[Service]
# Запускать процесс от имени непривилегированного пользователя "coredns"
User=coredns
Group=coredns

# Разрешить привязку к порту 53 без root
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE

# Команда для запуска CoreDNS с указанием конфигурационного файла
ExecStart=/usr/local/bin/coredns -conf /etc/coredns/Corefile

# Автоматически перезапускать сервис при падении
Restart=always

# Логирование в systemd-journal
StandardOutput=journal
StandardError=journal

[Install]
# Указывает, что сервис должен запускаться при загрузке системы (в многопользовательском режиме)
WantedBy=multi-user.target
EOF
```

Запускаем службу и устанавливаем её в автозагрузку:

```
sudo systemctl daemon-reload
sudo systemctl enable coredns
sudo systemctl restart coredns
sudo systemctl status coredns
```

```
• coredns.service - CoreDNS DNS Server
   Loaded: loaded (/etc/systemd/system/coredns.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-04-07 14:25:21 UTC; 927ms ago
 Invocation: 234659df4637416eaa62d8cd91ee6582
    Main PID: 1979 (coredns)
      Tasks: 11 (limit: 1789)
     Memory: 10.8M (peak: 12M)
        CPU: 22ms
    CGroup: /system.slice/coredns.service
            └─1979 /usr/local/bin/coredns -conf /etc/coredns/Corefile
```

Проверяем локальный резолв:

```
dig gateway.dev.lan

; <<> DiG 9.20.0-2ubuntu3.1-Ubuntu <<> gateway.dev.lan
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 4268
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: a65bad0cf1b37821 (echoed)
;; QUESTION SECTION:
;gateway.dev.lan.                IN      A

;; ANSWER SECTION:
gateway.dev.lan.                30      IN      A      192.168.20.1

;; Query time: 0 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Fri May 16 15:15:28 MSK 2025
;; MSG SIZE rcvd: 89
```

Проверяем резолв адресов из Интернета:

```
dig ya.ru

; <<> DiG 9.20.0-2ubuntu3.1-Ubuntu <<> ya.ru
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 26312
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: dcfa8ae299635826 (echoed)
;; QUESTION SECTION:
;ya.ru.                          IN      A

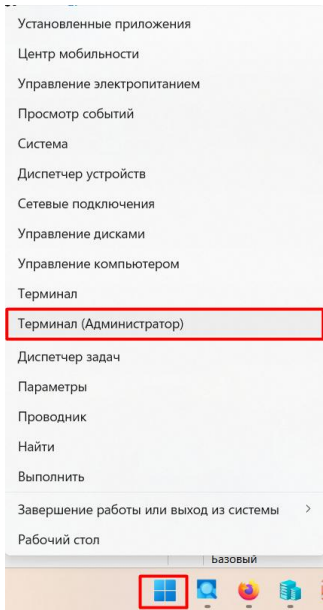
;; ANSWER SECTION:
ya.ru.                30      IN      A      77.88.44.242
ya.ru.                30      IN      A      5.255.255.242
ya.ru.                30      IN      A      77.88.55.242

;; Query time: 21 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Fri May 16 15:20:15 MSK 2025
;; MSG SIZE rcvd: 109
```

Просмотр лога работы CoreDNS:

```
sudo journalctl -u coredns -f
```

Чтобы из рабочего окружения в **Windows** заходить на WEB-интерфейсы устанавливаемых программ, необходимо, чтобы в эти программы поступала информация об запрашиваемом `Host`: в запросе WEB-браузера. Этого можно добиться, создав локальную DNS таблицу. Для этого в **Windows** заходим в консоль под Администратором, нажав правой клавишей мыши на кнопку **Пуск** и выбрав **Терминал (Администратор)**:



Открываем `hosts` файл и в конец файла добавляем все имена хостов, указанных в CoreDNS:

```
| notepad C:\Windows\System32\drivers\etc\hosts

192.168.30.11 gateway.dev.lan
192.168.20.11 combo-1.dev.lan
192.168.20.12 combo-2.dev.lan
192.168.20.13 combo-3.dev.lan
192.168.30.11 traefik.dev.lan
192.168.30.11 whoami.dev.lan
192.168.30.11 registry.dev.lan
192.168.30.11 portainer.dev.lan
192.168.30.11 gitlab.dev.lan
192.168.30.11 ca.dev.lan
192.168.30.11 vault.dev.lan
192.168.30.11 code.dev.lan
192.168.30.11 prometheus.dev.lan
192.168.30.11 grafana.dev.lan
192.168.30.11 hello.dev.lan
```

Проверяем в терминале резон:

```
| ping gateway.dev.lan

Обмен пакетами с gateway.dev.lan [192.168.30.11] с 32 байтами данных:
Ответ от 192.168.30.11: число байт=32 время<1мс TTL=64
Ответ от 192.168.30.11: число байт=32 время<1мс TTL=64
Ответ от 192.168.30.11: число байт=32 время<1мс TTL=64
Ответ от 192.168.30.11: число байт=32 время<1мс TTL=64

Статистика Ping для 192.168.30.11:
  Пакетов: отправлено = 4, получено = 4, потеряно = 0
  (0% потерь)
Приблизительное время приема-передачи в мс:
  Минимальное = 0мсек, Максимальное = 0 мсек, Среднее = 0 мсек
```

3.2 Создание центра сертификации SSL Step CA

GitHub CA — <https://github.com/smallstep/certificates>

GitHub CLI — <https://github.com/smallstep/cli>

Для того, чтобы всё общение между хостом и внутренними сервисами через TLS (HTTPS), устанавливаем центр сертификации, поддерживающий протокол ACME (Let's Encrypt). Для этого устанавливаем центр сертификации Step CA, который эмитирует работу Let's Encrypt. В CoreDNS уже прописана для него запись `ca.dev.lan`

```
Проверяем, что в системе не занят порт :9443 на котором будет работать Step CA командой (вывод команды должен быть пустой):  
| sudo ss -tulnp | grep 9443
```

Скачиваем установочные файлы:

```
wget -P /tmp https://github.com/smallstep/certificates/releases/download/v0.28.4/step-ca_0.28.4-1_amd64.deb  
wget -P /tmp https://github.com/smallstep/cli/releases/download/v0.28.7/step-cli_0.28.7-1_amd64.deb
```

Устанавливаем:

```
| sudo apt install /tmp/step-ca_0.28.4-1_amd64.deb /tmp/step-cli_0.28.7-1_amd64.deb
```

Создаём файл с паролем, который затем будет передаваться подключаться службой step-ca автоматически:

```
sudo mkdir /etc/step-ca  
echo 'Pa$$word' | sudo tee /etc/step-ca/password > /dev/null  
sudo chmod 600 /etc/step-ca/password
```

Инициализируем Step CA командой (необходимо будет выбрать Standalone и указать пароль **Pa\$\$word**). При этом будет выпущен корневой сертификат сроком на 10 лет:

```
sudo step ca init \  
--deployment-type=standalone \  
--name "My Dev CA" \  
--dns "ca.dev.lan" \  
--address ":9443" \  
--provisioner "administrator@mail.dev.lan" \  
--password-file /etc/step-ca/password
```

```
Use the arrow keys to navigate: ↑ ↓ → ←  
? What deployment type would you like to configure?:  
  Standalone - step-ca instance you run yourself  
  Linked - standalone, plus cloud configuration, reporting & alerting  
  Hosted - fully-managed step-ca cloud instance run for you by smallstep
```

```
Generating root certificate... done!
```

```
Generating intermediate certificate... done!
```

```
✓ Root certificate: /root/.step/certs/root_ca.crt  
✓ Root private key: /root/.step/secrets/root_ca_key  
✓ Root fingerprint: 3f35dace13a926151d808e96f366a989e9cf095fabf73ece6ed42dde7790d1a2  
✓ Intermediate certificate: /root/.step/certs/intermediate_ca.crt  
✓ Intermediate private key: /root/.step/secrets/intermediate_ca_key  
✓ Database folder: /root/.step/db  
✓ Default configuration: /root/.step/config/defaults.json  
✓ Certificate Authority configuration: /root/.step/config/ca.json
```

```
Your PKI is ready to go. To generate certificates for individual services see 'step help ca'.
```

```
FEEDBACK □ □  
The step utility is not instrumented for usage statistics. It does not phone home. But your feedback is extremely valuable. Any information you can provide regarding how you're using `step` helps. Please send us a sentence or two, good or bad at feedback@smallstep.com or join GitHub Discussions https://github.com/smallstep/certificates/discussions and our Discord https://u.step.sm/discord.
```

В результате будет выстроена структура дерева сертификатов:

```
| sudo tree /root/.step  
  
/root/.step  
├── certs                # Сертификаты CA (публичные ключи)  
│   ├── intermediate_ca.crt # Промежуточный сертификат (подписан root_ca)  
│   └── root_ca.crt        # Корневой сертификат (самоподписанный)  
├── config               # Конфигурационные файлы  
│   ├── ca.json           # Основной конфиг CA (настройки, provisioners)  
│   └── defaults.json      # Параметры по умолчанию  
├── db                   # База данных выданных сертификатов (SQLite)  
├── secrets              # Приватные ключи (защищённые)  
│   ├── intermediate_ca_key # Приватный ключ промежуточного CA  
│   └── root_ca_key        # Приватный ключ корневого CA  
└── templates            # Шаблоны для генерации сертификатов (опционально)
```

Добавляем "провизιονера" ACME, чтобы можно было работать с Step CA как с Let's Encrypt. Эта команда изменяет файл конфигурации `/root/.step/config/ca.json` необходимую настройку:

```
| sudo step ca provisioner add acme --type ACME
```

```
✓ CA Configuration: /root/.step/config/ca.json
```

```
Success! Your `step-ca` config has been updated. To pick up the new configuration SIGHUP (kill -1 <pid>) or restart the step-ca process.
```

Создаём службу для запуска step-ca:

```
sudo tee /etc/systemd/system/step-ca.service << 'EOF' > /dev/null  
[Unit]  
Description=Step Certificate Authority  
Documentation=https://smallstep.com/docs/step-ca  
After=network.target  
  
[Service]  
User=root  
Group=root
```

```
ExecStart=/usr/bin/step-ca /root/.step/config/ca.json --password-file=/etc/step-ca/password
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

Запускаем сервис:

```
sudo systemctl daemon-reload
sudo systemctl enable step-ca
sudo systemctl restart step-ca
sudo systemctl status step-ca
```

```
• step-ca.service - Step Certificate Authority
   Loaded: loaded (/etc/systemd/system/step-ca.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-05-31 15:29:39 MSK; 3min 11s ago
  Invocation: e5b866479f234376bbdea5a052f9673a
     Docs: https://smallstep.com/docs/step-ca
    Main PID: 55358 (step-ca)
      Tasks: 7 (limit: 6390)
    Memory: 13M (peak: 13.3M)
       CPU: 560ms
    CGroup: /system.slice/step-ca.service
            └─55358 /usr/bin/step-ca /root/.step/config/ca.json --password-file=/etc/step-ca/password
```

Проверяем, что сервер Step CA занимает нужный порт:

```
sudo ss -tlnp | grep 9443

tcp    LISTEN 0      4096      *:9443      *: *      users:(( "step-ca", pid=55358, fd=10))
```

Проверяем, что Step CA жив:

```
sudo step ca health --ca-url=https://ca.dev.lan:9443

ok
```

Установка сертификата в систему Linux

Копируем корневой сертификат в систему, чтобы ему могли доверять все программы и сервисы системы:

```
sudo cp /root/.step/certs/root_ca.crt /usr/local/share/ca-certificates/
sudo chmod 644 /usr/local/share/ca-certificates/root_ca.crt
sudo update-ca-certificates
```

```
Updating certificates in /etc/ssl/certs...
rehash: warning: skipping ca-certificates.crt, it does not contain exactly one certificate or CRL
1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
```

Сертификат помещается в папку /etc/ssl/certs/. Проверяем, что сертификат действует в системе:

```
curl -s https://ca.dev.lan:9443/acme/acme/directory | jq

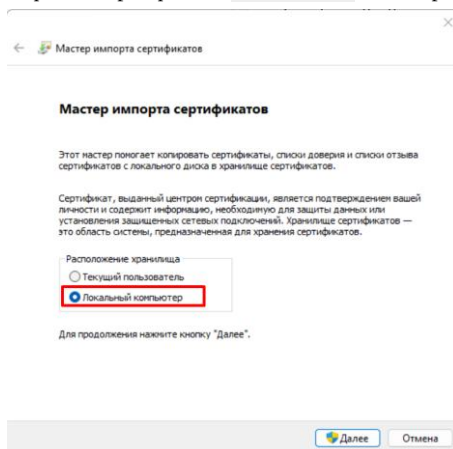
{
  "newNonce": "https://ca.dev.lan:9443/acme/acme/new-nonce",
  "newAccount": "https://ca.dev.lan:9443/acme/acme/new-account",
  "newOrder": "https://ca.dev.lan:9443/acme/acme/new-order",
  "revokeCert": "https://ca.dev.lan:9443/acme/acme/revoke-cert",
  "keyChange": "https://ca.dev.lan:9443/acme/acme/key-change"
}
```

Установка сертификата в браузер Firefox Windows

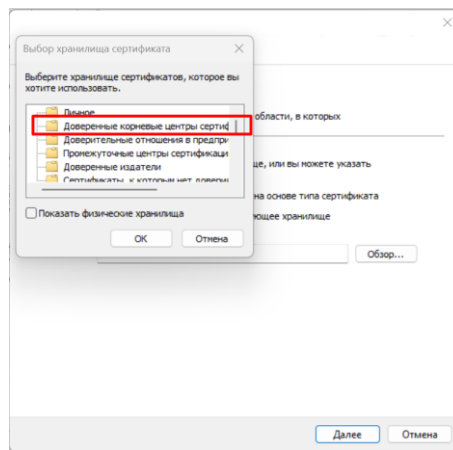
Копируем с хостовой машины Windows сертификат с сервера gateway:

```
scp administrator@gateway.dev.lan:/usr/local/share/ca-certificates/root_ca.crt .
```

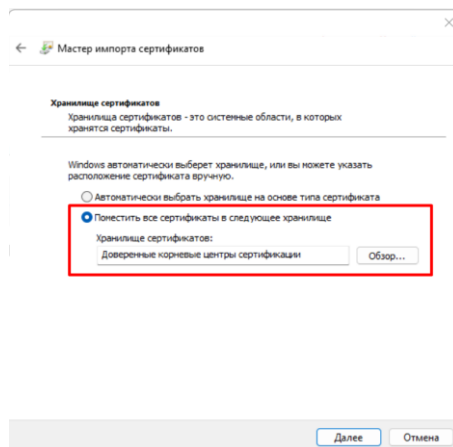
Устанавливаем в систему через нажатие ПКМ на файле сертификата root_ca.crt и выбирая пункт Установить сертификат.



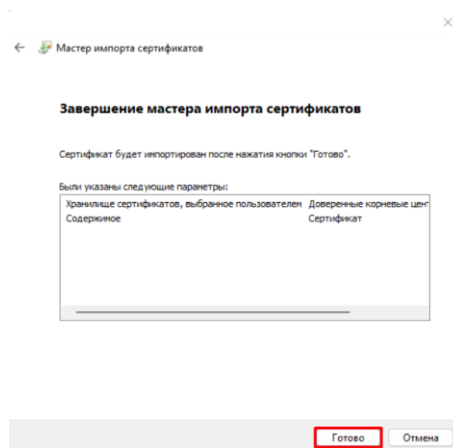
Выбираем пункт Поместить все сертификаты в следующее хранилище и нажать кнопку Обзор и выбираем Доверенные корневые центры сертификации:



После выбора нажимаем **Далее**:



Нажимаем **Готово**:



Устанавливаем в браузер **Firefox**, т.к. Firefox имеет своё хранилище сертификатов. Для этого открываем Firefox:

- вбиваем во вкладке `about:preferences#privacy` → Сертификаты → Просмотр сертификатов → Центры сертификации → Импорт → Указываем сертификат `root_ca.crt`
- устанавливаем галки:
 - [V] — Доверять при идентификации веб-сайтов
 - [V] — Доверять при идентификации пользователей почты
- Нажимаем **Ок**

3.3 Установка Ansible

Ansible устанавливаем в виртуальном окружении. Создаём директорию для проекта:

```
mkdir -p ~/ansible
cd ~/ansible/
```

Создаём виртуальное окружение и активируем его:

```
python3 -m venv .venv
source .venv/bin/activate

(.venv) administrator@gateway:~/ansible$
```

Устанавливаем Ansible:

```
pip install --upgrade pip
```

```
pip install ansible
```

Создаём структуру папок и файлов Ansible:

```
mkdir -p {group_vars,inventory,roles,templates,artifacts}
touch {ansible.cfg,gateways.yml,combos.yml,inventory/hosts.yml}
tree --dirsfirst
```

```
.
├── group_vars
├── inventory
│   └── hosts.yml
├── roles
├── templates
├── ansible.cfg
├── combos.yml
└── gateways.yml
```

... где:

- `group_vars` — групповые переменные;
- `inventory` — описывает инфраструктуру;
- `roles` — все роли;
- `templates` — шаблоны.
- `combos.yml` — сценарий с ролями для хостов combo-*
- `gateways.yml` — сценарий с ролями для хоста gateway

Заполняем данные о хостах в `hosts.yml`:

```
tee inventory/hosts.yml << 'EOF' > /dev/null
[gateways]
gateway ansible_host=gateway.dev.lan ansible_python_interpreter=/usr/bin/python3.12

[combos]
combo-1 ansible_host=combo-1.dev.lan ansible_python_interpreter=/usr/bin/python3.12
combo-2 ansible_host=combo-2.dev.lan ansible_python_interpreter=/usr/bin/python3.12
combo-3 ansible_host=combo-3.dev.lan ansible_python_interpreter=/usr/bin/python3.12

[all:children]
gateways
combos
EOF
```

где:

- `ansible_host=` — указывает FQDN имя хоста, транслируемого через DNS;
- `ansible_python_interpreter=` — указываем путь к интерпретатору на хосте, чтобы не выскакивало предупреждение `[WARNING]: Host 'combo-1' is using the discovered Python interpreter ...`;

Создаём структуру файлов сценариев `gateways.yml`:

```
tee gateways.yml << 'EOF' > /dev/null
---
- name: Настройка шлюзового хоста
  hosts: gateways
  roles:
EOF
```

Создаём структуру файлов сценариев `combos.yml`:

```
tee combos.yml << 'EOF' > /dev/null
---
- name: Настройка рабочих хостов
  hosts: combos
  roles:
EOF
```

Проверяем, как видит Ansible созданные файлы:

```
ansible-inventory --list -i inventory/hosts.yml
```

```
{
  "_meta": {
    "hostvars": {
      "combo-1": {
        "ansible_host": "combo-1.dev.lan",
        "ansible_python_interpreter": "/usr/bin/python3.12"
      },
      "combo-2": {
        "ansible_host": "combo-2.dev.lan",
        "ansible_python_interpreter": "/usr/bin/python3.12"
      },
      "combo-3": {
        "ansible_host": "combo-3.dev.lan",
        "ansible_python_interpreter": "/usr/bin/python3.12"
      },
      "gateway": {
        "ansible_host": "gateway.dev.lan",
        "ansible_python_interpreter": "/usr/bin/python3.12"
      }
    }
  },
  "profile": "inventory_legacy"
},
{
  "all": {
    "children": [
      "ungrouped",
      "gateways",
      "combos"
    ]
  },
  "combos": {
    "hosts": [
      "combo-1",
      "combo-2",
      "combo-3"
    ]
  }
}
```

```

    },
    "gateways": {
      "hosts": [
        "gateway"
      ]
    }
  }
}

```

Создаём ключ на сервере gateway. Все поля оставляем пустыми:

```

ssh-keygen -t rsa

Generating public/private rsa key pair.
Enter file in which to save the key (/home/administrator/.ssh/id_rsa): <ENTER>
Enter passphrase (empty for no passphrase): <ENTER>
Enter same passphrase again: <ENTER>
Your identification has been saved in /home/administrator/.ssh/id_rsa
Your public key has been saved in /home/administrator/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:UfdmW15LAKcrendf4vmEScre4u/oTMxbRUwzsMiLXfI administrator@gateway
The key's randomart image is:
+---[RSA 3072]-----+
|      . +.+. .o. |
|      *. =...oo |
|      o +o+o= o  |
|      oo==* .    |
|      . S. oOE. . |
|      . . * B .   |
|      . . . + B +  |
|      . . = o =    |
|      o +..       |
+----[SHA256]-----+

```

Распространяем ключи по серверам, делая доступ без пароля по IP (не DNS):

```

# Устанавливаем ключ на локальный хост, чтобы его единообразно обрабатывать совместно с другими
ssh-copy-id administrator@gateway.dev.lan
# Устанавливаем ключи на соседние хосты
ssh-copy-id administrator@combo-1.dev.lan
ssh-copy-id administrator@combo-2.dev.lan
ssh-copy-id administrator@combo-3.dev.lan

```

Проверяем работу Ansible:

```

ansible -i inventory/hosts.yaml -m ping combos

combo-1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
combo-3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
combo-2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}

```

Получить максимальные данные о хостах:

```

ansible -i inventory/hosts.yaml -m setup combos
# С фильтрацией данных — ansible -i inventory/hosts.yaml -m setup combos -a 'filter=ansible_distribution_version'
# Запустить команду — ansible -i inventory/hosts.yaml -m shell -a "hostname" combos

```

Деактивируем виртуальное окружение:

```

deactivate

```

3.4 Настройка хостов через Ansible

3.4.1 Роль для Debug

Создаём роль, которая нужна только для тестирования внутренних переменных Ansible (которые меняются от версии к версии). Создаём роль:

```

cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/debug

```

Редактируем задание:

```

vim roles/debug/tasks/main.yml

---
- name: Вывести основные magic variables
  ansible.builtin.debug:
    msg:
      - "inventory_hostname: {{ inventory_hostname }}"
      - "inventory_hostname_short: {{ inventory_hostname_short }}"
      - "inventory_dir: {{ inventory_dir }}"
      - "inventory_file: {{ inventory_file }}"
      - "group_names: {{ group_names }}"
      - "groups: {{ groups.keys() | list }}"
      - "ansible_facts['hostname']: {{ ansible_facts['hostname'] }}"
      - "ansible_facts['env']: {{ ansible_facts['env'] }}"
      - "playbook_dir: {{ playbook_dir }}"
      - "role_name: {{ role_name }}"
      - "role_names: {{ role_names }}"
      - "role_path: {{ role_path }}"
      - "ansible_play_hosts_all: {{ ansible_play_hosts_all }}"
      - "ansible_facts['user_dir']: {{ ansible_facts['user_dir'] }}"
      - "ansible_facts['user_id']: {{ ansible_facts['user_id'] }}"
      - "ansible_facts['real_user_id']: {{ ansible_facts['real_user_id'] }}"

- name: Полный список hostvars
  ansible.builtin.debug:
    msg: "{{ hostvars[inventory_hostname] }}"

```

Создаём playbook для запуска дебага:

```
vim debug-playbook.yml
```

```
---
- hosts: gateway
  roles:
    - debug
```

Запускаем для вывода внутренних переменных:

```
ansible-playbook -i inventory/hosts.yml debug-playbook.yml
```

3.4.2 Настройка Bash

Настраиваем командную оболочку Bash. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/configure_bash
```

Редактируем задание:

```
vim roles/configure_bash/tasks/main.yml
```

```
---
- name: Установка bash-completion
  become: true
  ansible.builtin.apt:
    name: bash-completion
    state: present

- name: Создание .bash_aliases для administrator
  ansible.builtin.blockinfile:
    path: "/home/{{ ansible_facts['user_id'] }}/.bash_aliases"
    block: |
      alias sudo='sudo '
      alias ll='ls -ahF --group-directories-first'
      alias ll='ls -lhF --group-directories-first'
      alias la='ls -lahF --group-directories-first'
    create: true
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"
    mode: "0644"

- name: Проверка наличия /etc/skel/.bash_aliases
  ansible.builtin.stat:
    path: /etc/skel/.bash_aliases
    register: skel_exist

- name: Копирование .bash_aliases to /etc/skel
  become: true
  ansible.builtin.copy:
    src: "/home/{{ ansible_facts['user_id'] }}/.bash_aliases"
    dest: /etc/skel/.bash_aliases
    remote_src: true
    owner: root
    group: root
    mode: "0644"
  when: not skel_exist.stat.exists
```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml
```

```
# ...
roles:
  # ...
  - role: configure_bash
    tags: [configure_bash]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
ansible-playbook -i inventory/hosts.yml gateways.yml --tags configure_bash
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml
```

```
# ...
roles:
  # ...
  - role: configure_bash
    tags: [configure_bash]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yml combos.yml --tags configure_bash
```

3.4.3 Установка утилит Linux

Установка различных утилит Linux для комфортной работы и диагностики. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_utils
```

Создаём шаблон настройки для утилиты htop:

```
vim roles/install_utils/templates/htoprc.j2
```

```
# Beware! This file is rewritten by htop when settings are changed in the interface.
# The parser is also very primitive, and not human-friendly.
htop_version=3.3.0
config_reader_min_version=3
fields=0 48 50 17 18 38 39 40 2 46 47 49 1
hide_kernel_threads=1
hide_userland_threads=1
hide_running_in_container=0
shadow_other_users=0
show_thread_names=0
show_program_path=1
highlight_base_name=0
highlight_deleted_exe=1
shadow_distribution_path_prefix=0
```

```

highlight_megabytes=1
highlight_threads=1
highlight_changes=0
highlight_changes_delay_secs=5
find_comm_in_cmdline=1
strip_exe_from_cmdline=1
show_merged_command=0
header_margin=1
screen_tabs=1
detailed_cpu_time=0
cpu_count_from_one=0
show_cpu_usage=1
show_cpu_frequency=0
show_cpu_temperature=0
degree_fahrenheit=0
update_process_names=0
account_guest_in_cpu_meter=0
color_scheme=0
enable_mouse=1
delay=15
hide_function_bar=0
header_layout=two_50_50
column_meters_0=AllCPUs Memory Swap
column_meter_modes_0=1 1 1
column_meters_1=Tasks LoadAverage Uptime
column_meter_modes_1=2 2 2
tree_view=1
sort_key=46
tree_sort_key=0
sort_direction=-1
tree_sort_direction=1
tree_view_always_by_pid=0
all_branches_collapsed=0
screen:Main=PID USER NLWP PRIORITY NICE M_VIRT M_RESIDENT M_SHARE STATE PERCENT_CPU PERCENT_MEM TIME Command
.sort_key=PERCENT_CPU
.tree_sort_key=PID
.tree_view_always_by_pid=0
.tree_view=1
.sort_direction=-1
.tree_sort_direction=1
.all_branches_collapsed=0
screen:I/O=PID USER IO_PRIORITY IO_RATE IO_READ_RATE IO_WRITE_RATE PERCENT_SWAP_DELAY PERCENT_IO_DELAY Command
.sort_key=IO_RATE
.tree_sort_key=PID
.tree_view_always_by_pid=0
.tree_view=0
.sort_direction=-1
.tree_sort_direction=1
.all_branches_collapsed=0

```

Редактируем задание:

```
| vim roles/install_utils/tasks/main.yml
```

```

---
- name: Установка утилит
  become: true
  ansible.builtin.apt:
    name:
      - bash-completion
      - vim
      - nano
      - iputils-ping
      - dnsutils
      - traceroute
      - curl
      - wget
      - tree
      - htop
      - gdu
      - jq
      - yq
      - unzip
      - tmux
      - lsof
      - html2text
      - yamllint
      - python3.12-venv
    state: present
    update_cache: true
    cache_valid_time: 3600 # Перезапрашивать индекс только если прошло > 1 часа

- name: Конфигурация .tmux.conf
  ansible.builtin.blockinfile:
    path: "/home/{{ ansible_facts['user_id'] }}/.tmux.conf"
    block: |
      # Мышь прокручивает историю терминала, а не историю команд
      set -g mouse on
    create: true
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"
    mode: "0644"

- name: Создание директории под конфигурацию htop
  ansible.builtin.file:
    path: "/home/{{ ansible_facts['user_id'] }}/.config/htop"
    state: directory
    mode: "0755"

- name: Конфигурация htop
  ansible.builtin.template:
    src: htoprc.j2
    dest: "/home/{{ ansible_facts['user_id'] }}/.config/htop/htoprc"
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"
    mode: "0644"
    force: true

```

Добавляем роль в сценарий запуска для gateways:

```
| vim gateways.yml
```

```
# ...
roles:
# ...
- role: install_utils
tags: [install_utils]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
| ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_utils
```

Добавляем роль в сценарий запуска для **combos**:

```
| vim combos.yml
```

```
# ...
roles:
# ...
- role: install_utils
tags: [install_utils]
```

Запускаем обработку конкретно этой команды для **combos**:

```
| ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_utils
```

3.4.4 Установка ip_forwarding

Создаём правило создания ip forwarding для доступа серверов kube-combo-1,2,3 через kube-gateway в сеть Интернет, т.к. сейчас он ничего скачать из сети не сможет. Создаём роль:

```
| cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/ip_forward
```

Редактируем переменные по умолчанию:

```
| vim roles/ip_forward/defaults/main.yml
```

```
---
iptables_forward: false # Установить мост в iptables
iptables_forward_eth_lan: "eth1" # Интерфейс в приватную сеть
iptables_forward_eth_wan: "eth0" # Интерфейс в WAN
```

Редактируем задание:

```
| vim roles/ip_forward/tasks/main.yml
```

```
---
- name: Установка ip_forward
  become: true # Запускать через sudo
  ansible.builtin.lineinfile:
    path: /etc/sysctl.d/99-forward.conf
    regexp: "^net.ipv4.ip_forward=" # Заменяем строку, если там стоит не нужное значение 1 (может кто в ручную отключил)
    line: "net.ipv4.ip_forward=1"
    insertafter: EOF # Добавлять строку в конец файла
    state: present # Строка должна присутствовать
    create: true # Создать файл /etc/sysctl.d/99-forward.conf если его нет (обязательны параметры файла при этом)
    owner: root
    group: root
    mode: "0644"
  notify: Apply sysctl # При изменении запустить handler

# Этот блок только для хостов, который входит в группу gateways
- name: Обработка только для gateway
  when: "'gateways' in group_names" # Ставим зависимость выполнения блока от вхождения хоста в группу gateways
  block:

    # Команда: sudo iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT -m comment --comment 'WAN'
    - name: Разрешить IP-форвардинга из LAN в WAN
      become: true # Запускать через sudo
      ansible.builtin.iptables:
        chain: FORWARD
        in_interface: "{{ iptable_forward_eth_lan }}"
        out_interface: "{{ iptable_forward_eth_wan }}"
        jump: ACCEPT
        comment: "WAN"
        state: present
      notify: Save iptables # При изменении запустить handler

    # Команда: sudo iptables -A FORWARD -i eth0 -o eth1 -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT -m comment --comment 'WAN'
    - name: Разрешить поддерживать состоявшиеся соединения из WAN в LAN
      become: true # Запускать через sudo
      ansible.builtin.iptables:
        chain: FORWARD
        in_interface: "{{ iptable_forward_eth_wan }}"
        out_interface: "{{ iptable_forward_eth_lan }}"
        jump: ACCEPT
        comment: "WAN"
        ctstate: ESTABLISHED,RELATED
        state: present
      notify: Save iptables # При изменении запустить handler

    # Команда: sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE -m comment --comment 'WAN'
    - name: Добавление правил MASQUERADE в iptables
      become: true # Запускать через sudo
      ansible.builtin.iptables:
        table: nat
        chain: POSTROUTING
        out_interface: "{{ iptable_forward_eth_wan }}"
        jump: MASQUERADE
        comment: "WAN"
        state: present
      notify: Save iptables # При изменении запустить handler

# Принудительно запустить всех активированных в этой роли handlers
- name: Flush handlers
  ansible.builtin.meta: flush_handlers
```

Редактируем реакции:

```
| vim roles/ip_forward/handlers/main.yml
```

```

- name: Apply sysctl
  become: true # Запускать через sudo
  # Применить настройки в файле
  ansible.builtin.command: sysctl -p /etc/sysctl.d/99-forward.conf
  changed_when: true # Принудительно будет установлен флаг changed для задачи

- name: Save iptables
  become: true # Запускать через sudo
  ansible.builtin.command: netfilter-persistent save
  changed_when: true # Принудительно будет установлен флаг changed для задачи

```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml
```

```

# ...
roles:
  # ...
  - role: ip_forward
    tags: [ip_forward]
    vars:
      iptable_forward: false # Установить мост в iptables
      iptable_forward_eth_lan: "eth1" # Интерфейс в приватную сеть
      iptable_forward_eth_wan: "eth0" # Интерфейс в WAN

```

Запускаем обработку конкретно этой команды для **gateways**:

```
ansible-playbook -i inventory/hosts.yaml gateways.yml --tags ip_forward
```

Проверить созданные таблицы можно командой:

```
sudo iptables -L -v -n -x --line-numbers
```

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts  bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts  bytes target     prot opt in     out     source               destination
1    0      0 ACCEPT 0 -- eth1    eth0    0.0.0.0/0            0.0.0.0/0            /* WAN */
2    0      0 ACCEPT 0 -- eth0    eth1    0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED /* WAN */

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts  bytes target     prot opt in     out     source               destination
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml
```

```

# ...
roles:
  # ...
  - role: ip_forward
    tags: [ip_forward]
    vars:
      iptable_forward: false # Установить мост в iptables

```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yaml combos.yml --tags ip_forward
```

3.4.5 Установка Sysdig

Устанавливаем диагностическую утилиту sysdig. Для этого устанавливаем заголовочные файлы ядра, для компиляции модулей подключения к событиям системой sysdig и устанавливаем саму утилиту из GitHub:

Создаём роль:

```

cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_sysdig

```

Редактируем задание:

```
vim roles/install_sysdig/tasks/main.yml
```

```

- name: Проверяем наличие sysdig
  ansible.builtin.command: dpkg-query -W sysdig
  register: sysdig_check
  changed_when: false
  failed_when: false # не падаем, если пакет не найден (rc=1)

- name: Если sysdig ещё не установлен, устанавливаем его
  when: sysdig_check.rc != 0
  block:

  - name: Устанавливаем заголовочные файлы ядра
    become: true
    ansible.builtin.apt:
      name: "linux-headers-{{ ansible_facts['kernel'] }}"
      state: present

  - name: Скачиваем sysdig .deb
    ansible.builtin.get_url:
      url: https://github.com/draios/sysdig/releases/download/0.41.3/sysdig-0.41.3-x86_64.deb
      dest: /tmp/sysdig-0.41.3-x86_64.deb # Куда скачивать пакет (установку будем делать из /tmp директории
      mode: '0644'
      checksum: sha256:594f39eb3beb14e5131d2650ec7c758a5ae6582c67772c7f1a048f659a81c333 # Контрольная сумма пакета sysdig-0.41.3-x86_64.deb

  - name: Устанавливаем sysdig .deb
    become: yes
    # apt модуль автоматически вызовет dpkg с зависимостями и сделает apt --fix-broken install при необходимости
    ansible.builtin.apt:
      deb: /tmp/sysdig-0.41.3-x86_64.deb
      state: present

```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml
```

```
# ...
```

```
roles:
# ...
- role: install_sysdig
tags: [install_sysdig]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
| ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_sysdig
```

Добавляем роль в сценарий запуска для **combos**:

```
| vim combos.yml
```

```
# ...
roles:
# ...
- role: install_sysdig
tags: [install_sysdig]
```

Запускаем обработку конкретно этой команды для **combos**:

```
| ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_sysdig
```

3.4.6 Установка Chrony

Устанавливаем сервер синхронизации времени.

Создаём роль:

```
| cd ~/ansible/
| source .venv/bin/activate
| ansible-galaxy init roles/install_chrony
```

Устанавливаем значение по умолчанию:

```
| vim roles/install_chrony/defaults/main.yml
```

```
---
# Сеть, для которой данные сервер будет сервером синхронизации
chrony_network: 192.160.20.0/24
# Интерфейс, через который Chrony принимать клиентов для синхронизации
chrony_interface: eth1
```

Редактируем задание:

```
| vim roles/install_chrony/tasks/main.yml
```

```
---
- name: Установка chrony
  become: true
  ansible.builtin.apt:
    name: chrony
    state: present

- name: Настройка chrony.conf (добавляем allow и local stratum)
  become: true
  ansible.builtin.lineinfile:
    path: /etc/chrony/chrony.conf
    line: "{{ item }}"
    insertafter: EOF
  loop:
    - "allow {{ chrony_network }}"
    - "local stratum 2"
  notify: Restart chrony

- name: Разрешение NTP-трафика (UDP 123) в iptables
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    in_interface: "{{ chrony_interface }}"
    protocol: udp
    destination_port: 123
    jump: ACCEPT
    comment: "NTP"
  notify: Save iptables

- name: Flush handlers
  ansible.builtin.meta: flush_handlers
```

Редактируем реакцию:

```
| vim roles/install_chrony/handlers/main.yml
```

```
---
- name: Restart chrony
  become: true
  ansible.builtin.systemd:
    name: chrony
    state: restarted
    enabled: true

- name: Save iptables
  become: true
  ansible.builtin.command: netfilter-persistent save
  changed_when: false
```

Добавляем роль в сценарий запуска для **gateways**:

```
| vim gateways.yml
```

```
# ...
roles:
# ...
- role: install_chrony
tags: [install_chrony]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
| ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_chrony
```

3.4.7 Настройка Timesyncd

Настраиваем timesyncd на клиентах для синхронизации времени с сервером.

Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/configure_timesyncd
```

Устанавливаем значение по умолчанию:

```
vim roles/configure_timesyncd/defaults/main.yml
```

```
---
# Адрес сервера для синхронизации времени
time_server: gateway.dev.lan
```

Редактируем задание:

```
vim roles/configure_timesyncd/tasks/main.yml
```

```
---
- name: Добавляем NTP-серверы
  become: true
  ansible.builtin.blockinfile:
    path: /etc/systemd/timesyncd.conf
    block: |
      NTP={{ time_server }}
      FallbackNTP={{ time_server }}
    insertafter: '^\[Time\]'
    state: present
  notify: Restart systemd-timesyncd

- name: Flush handlers
  ansible.builtin.meta: flush_handlers
```

Редактируем реакции:

```
vim roles/configure_timesyncd/handlers/main.yml
```

```
---
- name: Restart systemd-timesyncd
  become: true
  ansible.builtin.systemd:
    name: systemd-timesyncd
    state: restarted
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml
```

```
# ...
roles:
# ...
- role: configure_timesyncd
  tags: [configure_timesyncd]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yml combos.yml --tags configure_timesyncd
```

3.4.8 Установка Docker

Устанавливаем систему контейнеризации Docker:

Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_docker
```

Создаём шаблон для описания репозитория:

```
vim roles/install_docker/templates/docker.list.j2
```

```
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu noble stable
```

```
vim roles/install_docker/templates/daemon.json.j2
```

```
{
  "features": {
    "buildkit": true
  }
}
```

Редактируем задание:

```
vim roles/install_docker/tasks/main.yml
```

```
---
- name: Установка зависимостей для GPG и curl
  become: true
  ansible.builtin.apt:
    name:
      - ca-certificates
      - curl
      - gnupg
    state: present
    update_cache: true

- name: Скачать Docker GPG ключ
  become: true
  ansible.builtin.get_url:
    url: "https://download.docker.com/linux/ubuntu/gpg"
    dest: "/etc/apt/keyrings/docker.asc"
    mode: '0644'

- name: Создать Docker репозиторий
  become: true
  ansible.builtin.template:
    src: docker.list.j2
    dest: /etc/apt/sources.list.d/docker.list
    mode: '0644'
```

```
- name: Создание директории под конфигурацию Docker
  become: true
  ansible.builtin.file:
    path: /etc/docker
    state: directory
    mode: '0755'

- name: Создать конфигурацию Docker
  become: true
  ansible.builtin.template:
    src: daemon.json.j2
    dest: /etc/docker/daemon.json
    mode: '0644'

- name: Установить Docker пакеты
  become: true
  ansible.builtin.apt:
    name:
      # На текущий момент косяк в репозитории, нет последней версии пакета. Приходится указывать существующую версию
      - containerd.io=2.2.0-2~ubuntu.24.04~noble
      # Устанавливаем версию 28, которая совместима с устанавливаемой версией Kubernetes
      - docker-ce=5:28.5.2-1~ubuntu.24.04~noble
      - docker-ce-cli=5:28.5.2-1~ubuntu.24.04~noble
      - docker-buildx-plugin
      - docker-compose
      - docker-compose-plugin
    state: present
    update_cache: true

- name: Добавить группу docker
  become: true
  ansible.builtin.group:
    name: docker
    state: present

- name: Добавить пользователя в группу docker
  become: true
  ansible.builtin.user:
    name: "{{ ansible_facts['user_id'] }}"
    groups: docker
    append: true

- name: Запустить и включить Docker сервис
  become: true
  ansible.builtin.systemd:
    name: docker
    state: started
    enabled: true
```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml

# ...
roles:
  # ...
  - role: install_docker
    tags: [install_docker]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_docker
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml

# ...
roles:
  # ...
  - role: install_docker
    tags: [install_docker]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_docker
```

Чтобы ввод в группу сработал, необходимо на всех активных сессиях перезагрузит оболочку bash.

3.4.9 Установка Ctop

Устанавливаем программу просмотра состояния контейнеров по команде `ctop` в Bash. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/docker_ctop
```

Редактируем задание:

```
vim roles/docker_ctop/tasks/main.yml

---
- name: Загрузка ctop в Docker
  become: true
  community.docker.docker_image_pull:
    name: quay.io/vektorlab/ctop:0.7.7
    platform: amd64

- name: Добавление алиаса ctop в .bash_aliases
  ansible.builtin.lineinfile:
    path: "{{ ansible_facts['user_dir'] }}/.bash_aliases"
    regexp: "^alias ctop="
    line: "alias ctop='sudo docker run --rm -ti --name=ctop --volume /var/run/docker.sock:/var/run/docker.sock:ro quay.io/vektorlab/ctop:0.7.7'"
    state: present
  insertafter: EOF
```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml
```

```
# ...
roles:
# ...
- role: docker_ctop
tags: [docker_ctop]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
| ansible-playbook -i inventory/hosts.yaml gateways.yaml --tags docker_ctop
```

Добавляем роль в сценарий запуска для **combos**:

```
| vim combos.yaml
```

```
# ...
roles:
# ...
- role: docker_ctop
tags: [docker_ctop]
```

Запускаем обработку конкретно этой команды для **combos**:

```
| ansible-playbook -i inventory/hosts.yaml combos.yaml --tags docker_ctop
```

3.4.10 Копирование корневого сертификата stepca

Копируем корневой сертификат, созданный StepCA на все хосты стенда.

Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/copy_cert
```

Редактируем задание:

```
| vim roles/copy_cert/tasks/main.yaml
```

```
---
- name: Копирование сертификата домена GitLab на сервера
  become: true
  ansible.builtin.copy:
    src: /usr/local/share/ca-certificates/root_ca.crt
    dest: /usr/local/share/ca-certificates/root_ca.crt
    owner: root
    group: root
    mode: "0644"
    notify: Update certs
- name: Flush handlers
  ansible.builtin.meta: flush_handlers
```

Редактируем реакции:

```
| vim roles/copy_cert/handlers/main.yaml
```

```
---
- name: Update certs
  become: true
  ansible.builtin.command:
    cmd: update-ca-certificates
    changed_when: false
```

Добавляем роль в сценарий запуска для **combos**:

```
| vim combos.yaml
```

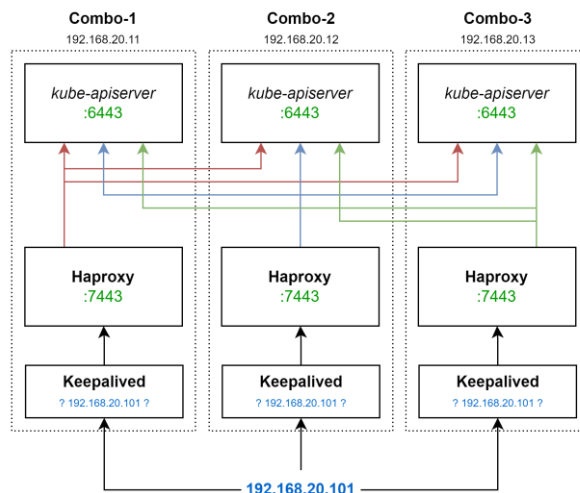
```
# ...
roles:
# ...
- role: copy_cert
tags: [copy_cert]
```

Запускаем обработку конкретно этой команды для **combos**:

```
| ansible-playbook -i inventory/hosts.yaml combos.yaml --tags copy_cert
```

3.4.11 Установка Harpoxu

Создаём плавающий IP, через который будет осуществляться доступ модуля **kubelet** и **kubectl** к управляющим узлам через **7443** порт на реальный порт kube-apiserver — **6443**, Kubelet и kubectl не умеют обращаться к нескольким управляющим узлам и в их конфигурации забит только один адрес. Балансировка нагрузки будет производиться в 2 слоя, через keepralived и harpoxu, как того требует официальный мануал, т.к. через keepralived будет распространяться виртуальный IP, а harpoxu будет понимать, какой из управляющих узлов сейчас ведущий и будет перенаправлять туда трафик kubelet.



Здесь Нароуху выполняет роль балансировщика между 3-мя control-plane нодами. Он открывает у себя порт **7443**, который, в зависимости от того, на каких из control-plane нод жив и работает kube-apiserver (у которого рабочий порт **6443**), перенаправляет порт 7443 на него.

На момент создания сервиса kube-apiserver ещё не существует, поэтому сервис корректно работать не будет.

Устанавливаем и настраиваем Нароуху. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_haproxy
```

Создаём шаблон конфигурации:

```
vim roles/install_haproxy/templates/haproxy.cfg.j2
```

```
# Глобальные настройки HAProxy
global
    # Логирование: отправка логов уровня local0 в /dev/log
    log /dev/log local0
    # Логирование: отправка логов уровня local1 с приоритетом notice в /dev/log
    log /dev/log local1 notice

    # Изоляция процесса HAProxy в chroot-окружении (/var/lib/haproxy)
    chroot /var/lib/haproxy

    # Создание UNIX-сокета для управления HAProxy (доступ с правами admin)
    stats socket /run/haproxy/admin.sock mode 660 level admin

    # Таймаут для операций со статистикой (30 секунд)
    stats timeout 30s

    # Запуск HAProxy от имени пользователя и группы haproxy
    user haproxy
    group haproxy

    # Запуск HAProxy в фоновом режиме (демонизация)
    daemon

    # Базовые пути для SSL-сертификатов:
    # Каталог с CA-сертификатами
    ca-base /etc/ssl/certs
    # Каталог с приватными ключами
    crt-base /etc/ssl/private

    # Настройки SSL по умолчанию (рекомендации Mozilla):
    # Список поддерживаемых шифров для TLS
    ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDSA-AES128-GCM-SHA256:ECDSA-AES256-GCM-SHA384:ECDSA-AES256-GCM-SHA384:ECDSA-CHACHA20-POLY1305:ECDSA-RSA-CHACHA20-POLY1305
    # Современные шифры для TLS 1.3
    ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
    # Минимальная версия TLS 1.2 и отключение TLS-билетов
    ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

# Настройки по умолчанию для всех прокси
defaults
    # Использование глобальных настроек логирования
    log global

    # Режим работы по умолчанию - HTTP
    mode http

    # Включение расширенного логирования HTTP-запросов
    option httplog

    # Не логировать пустые запросы
    option dontlognull

    # Закрывать соединение на стороне сервера после ответа
    option http-server-close

    # Добавлять заголовок X-Forwarded-For, кроме запросов из localhost
    option forwardfor except 127.0.0.0/8

    # Повторная диспетчеризация запросов при отказе сервера
    option redispatch

    # Количество попыток повторного соединения
    retries 1

    # Таймаут соединения с сервером (5 секунд)
    timeout connect 5000
```

```

# Таймаут ожидания данных от клиента (20 секунд)
timeout client 20s

# Таймаут ожидания данных от сервера (20 секунд)
timeout server 20s

# Таймаут обработки HTTP-запроса (10 секунд)
timeout http-request 10s

# Таймаут ожидания в очереди (20 секунд)
timeout queue 20s

# Таймаут keep-alive соединения (10 секунд)
timeout http-keep-alive 10s

# Таймаут для health-check (10 секунд)
timeout check 10s

# Файлы с кастомными страницами ошибок
errorfile 400 /etc/haproxy/errors/400.http
errorfile 403 /etc/haproxy/errors/403.http
errorfile 408 /etc/haproxy/errors/408.http
errorfile 500 /etc/haproxy/errors/500.http
errorfile 502 /etc/haproxy/errors/502.http
errorfile 503 /etc/haproxy/errors/503.http
errorfile 504 /etc/haproxy/errors/504.http

#-----
# Фронтенд для API-сервера Kubernetes
#-----
frontend apiserver
# Слушать порт 7443 на всех интерфейсах
bind *:7443

# Режим работы - TCP (L4 балансировка)
mode tcp

# Включить логирование TCP-соединений
option tcplog

# Использовать бэкенд apiserver по умолчанию
default_backend apiserver

#-----
# Бэкенд с балансировкой между control-plane нодами Kubernetes
#-----
backend apiserver
# Метод проверки работоспособности - HTTP GET /healthz
option httpchk GET /healthz

# Ожидать HTTP-статус 200 в ответе на health-check
http-check expect status 200

# Режим работы - TCP
mode tcp

# Проверка SSL-рукопожатия
option ssl-hello-chk

# Алгоритм балансировки - round-robin
balance roundrobin

# Серверы бэкенда (control-plane ноды Kubernetes)
server combo-1.dev.lan combo-1.dev.lan:6443 check # Нода 1 с проверкой здоровья
server combo-2.dev.lan combo-2.dev.lan:6443 check # Нода 2 с проверкой здоровья
server combo-3.dev.lan combo-3.dev.lan:6443 check # Нода 3 с проверкой здоровья

```

Редактируем задание:

```
| vim roles/install_haproxy/tasks/main.yml
```

```

---
- name: Установка Haproxy
  become: true
  ansible.builtin.apt:
    name: haproxy
    state: present
  notify: Restart Haproxy

- name: Создание скрипта проверки доступности
  become: true
  ansible.builtin.template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
    owner: root
    group: root
    mode: "0544"
  notify: Restart Haproxy

- name: Разрешаем порты
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 8888
    jump: ACCEPT
    comment: "Haproxy"
  notify: Save iptables

```

Редактируем реакции:

```
| vim roles/install_haproxy/handlers/main.yml
```

```

---
- name: Restart Haproxy
  become: true
  ansible.builtin.systemd:
    name: haproxy
    state: restarted
    enabled: true

```

```

daemon_reload: true

- name: Save iptables
  become: true
  ansible.builtin.command: netfilter-persistent save
  changed_when: true

```

Добавляем роль в сценарий запуска для **combos**:

```

vim combos.yml

# ...
roles:
# ...
- role: install_haproxy
  tags: [install_haproxy]

```

Запускаем обработку конкретно этой команды для **combos**:

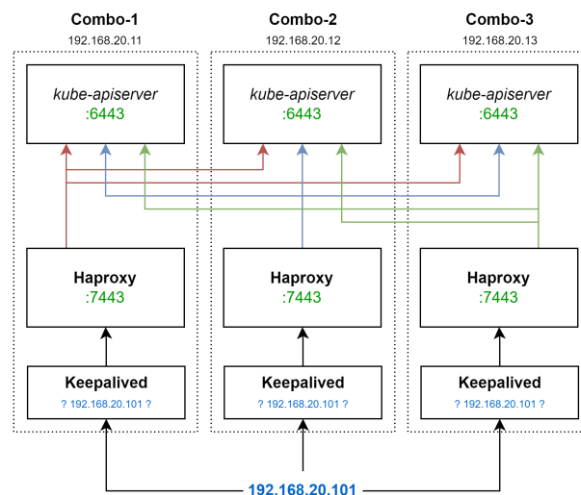
```

ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_haproxy

```

3.4.12 Установка Keepalived

Keepalived здесь будет создавать плавающий IP-адрес **192.168.20.101**, который будет присваиваться одной из control-plane нод, которая будет положительно отвечать на работу скрипта `check_apiserver.sh`. Проверка в скрипте происходит через Haproxy, который проксирует порт 7443 на управляющий порт 6443 управления kube-apiserver.



Создаём роль:

```

cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_keepalived

```

Создаём шаблон конфигурации:

```

vim roles/install_keepalived/templates/check_apiserver.sh.j2

```

```

#!/bin/sh

# Установка переменной APISERVER_VIP - виртуальный IP-адрес сервера
APISERVER_VIP=192.168.20.101

# Установка переменной APISERVER_DEST_PORT - TCP порт для доступа к системе управления
# TCP порт для доступа к системе управления: 7443
APISERVER_DEST_PORT=7443

# Установка переменной PROTO - используемый протокол (http)
PROTO=http

# Определение функции errorExit для вывода сообщения об ошибке и завершения скрипта с кодом 1
errorExit() {
    echo "*** $*" 1>&2 # Вывод сообщения об ошибке в stderr
    exit 1          # Завершение скрипта с кодом ошибки 1
}

# Проверка доступности локального сервера через curl:
# --silent - подавление вывода progress и ошибок
# --max-time 2 - максимальное время ожидания 2 секунды
# --insecure - разрешение небезопасных соединений (без проверки SSL)
# -o /dev/null - перенаправление вывода в никуда
# Если команда завершается с ошибкой (||), вызывается errorExit с сообщением
curl --silent --max-time 2 --insecure ${PROTO}://localhost:${APISERVER_DEST_PORT}/ -o /dev/null || errorExit "Error GET
${PROTO}://localhost:${APISERVER_DEST_PORT}/"

# Проверка, есть ли у текущего хоста виртуальный IP-адрес (APISERVER_VIP)
if ip addr | grep -q ${APISERVER_VIP}; then
    # Если виртуальный IP есть, проверяем доступность сервера через этот IP
    curl --silent --max-time 2 --insecure ${PROTO}://${APISERVER_VIP}:${APISERVER_DEST_PORT}/ -o /dev/null || errorExit "Error GET
    ${PROTO}://${APISERVER_VIP}:${APISERVER_DEST_PORT}/"
fi

```

```

vim roles/install_keepalived/templates/keepalived.conf.j2

```

```

# Глобальные настройки keepalived
global_defs {
    # Включение безопасности скриптов - запрещает выполнение скриптов из ненадежных мест
    enable_script_security

    # Пользователь, от имени которого будут выполняться скрипты (в данном случае nobody)
    script_user nobody
}

```

```

}

# Определение скрипта для проверки состояния (в данном случае API-сервера)
vrrp_script check_apiserver {
    # Путь к скрипту, который будет выполняться для проверки
    script "/etc/keepalived/check_apiserver.sh"

    # Интервал выполнения скрипта в секундах (каждые 3 секунды)
    interval 3
}

# Определение VRRP-инстанса (виртуального маршрутизатора)
vrrp_instance VI_1 {
    # Начальное состояние - BACKUP (будет переходить в MASTER при необходимости)
    {% if inventory_hostname == 'combo-1' %}
    state MASTER
    {% else %}
    state BACKUP
    {% endif %}

    # Сетевой интерфейс, на котором будет работать VRRP
    interface eth0

    # Идентификатор виртуального маршрутизатора (должен быть одинаковым в группе)
    virtual_router_id 5

    # Приоритет этого узла (чем выше, тем больше шансов стать MASTER)
    {% if inventory_hostname == 'combo-1' %}
    priority 200
    {% else %}
    priority 100
    {% endif %}

    # Интервал объявлений (advertisements) в секундах
    advert_int 1

    {% if inventory_hostname == 'combo-1' %}
    # Разрешить перехват мастер-статуса при восстановлении
    nopreempt off

    # Задержка перед перехватом (сек)
    preempt_delay 5
    {% else %}
    # Запрет на переход в MASTER режим, если узел с более высоким приоритетом вернется в строй
    nopreempt
    {% endif %}

    # Настройки аутентификации между узлами VRRP
    authentication {
        # Тип аутентификации - пароль
        auth_type PASS

        # Пароль для аутентификации (должен быть одинаковым на всех узлах группы)
        auth_pass ZqSj#f1G
    }

    # Виртуальный IP-адрес, который будет переключаться между узлами
    virtual_ipaddress {
        192.168.20.101
    }

    # Скрипт, который будет отслеживаться для принятия решений о переходе между состояниями
    track_script {
        check_apiserver
    }
}

```

Редактируем задание:

```
| vim roles/install_keepalived/tasks/main.yml
```

```

---
- name: Установка Keepalived
  become: true
  ansible.builtin.apt:
    name: keepalived
    state: present
  notify: Restart Keepalived

- name: Создание скрипта проверки доступности
  become: true
  ansible.builtin.template:
    src: check_apiserver.sh.j2
    dest: /etc/keepalived/check_apiserver.sh
    owner: root
    group: root
    mode: "0544"
  notify: Restart Keepalived

- name: Создание конфигурации
  become: true
  ansible.builtin.template:
    src: keepalived.conf.j2
    dest: /etc/keepalived/keepalived.conf
    owner: root
    group: root
    mode: "0644"
  notify: Restart Keepalived

- name: Flush handlers
  ansible.builtin.meta: flush_handlers

```

Редактируем реакции:

```
| vim roles/install_keepalived/handlers/main.yml
```

```

---
- name: Restart Keepalived
  become: true
  ansible.builtin.systemd:
    name: keepalived

```

```
state: restarted
enabled: true
daemon_reload: true
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml
```

```
# ...
roles:
# ...
- role: install_keepalived
tags: [install_keepalived]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_keepalived
```

3.4.13 Установка Kubernetes

Устанавливаем Kubernetes на сервера combo-1, combo-2 и combo-3. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_kubernetes
```

Создаём шаблон для описания репозитория:

```
vim roles/install_kubernetes/templates/kubernetes.list.j2
```

```
deb [signed-by=/etc/apt/keyrings/kubernetes.asc] https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /
```

Создаём шаблон службы Cri-Docker:

```
vim roles/install_kubernetes/templates/10-pause.conf.j2
```

```
[Service]
# Два ExecStart нужны специально.
ExecStart=
ExecStart=/usr/bin/cri-dockerd --container-runtime-endpoint fd:// --pod-infra-container-image "registry.k8s.io/pause:3.10"
```

Создание сокета Cri-Docker:

```
vim roles/install_kubernetes/templates/crictl.yaml.j2
```

```
runtime-endpoint: unix:///var/run/cri-dockerd.sock
```

Редактирование значений по умолчанию:

```
vim roles/install_kubernetes/defaults/main.yml
```

```
---
# Имя хоста, на котором будет установлена первая нода, к которой будет подключаться остальные
first_node: combo-1
```

Редактируем задание:

```
vim roles/install_kubernetes/tasks/main.yml
```

```
---
- name: Установка ключей репозитория Kubernetes
  become: true
  ansible.builtin.get_url:
    url: "https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key"
    dest: "/etc/apt/keyrings/kubernetes.asc"
    mode: '0644'

- name: Создать Kubernetes репозитория
  become: true
  ansible.builtin.template:
    src: kubernetes.list.j2
    dest: /etc/apt/sources.list.d/kubernetes.list
    mode: '0644'

- name: Устанавливаем базовые утилиты для работы Kubernetes (kubectl, gnupg, socat)
  become: true
  ansible.builtin apt:
    name:
      - kubectl
      - gnupg
      - socat
    state: present
    update_cache: true

- name: Настройка iptables для Kubernetes
  # Применить настройки только для группы combos
  when: "'combos' in group_names"
  block:

    # Предварительная настройка Kubernetes

    - name: Создаем файл загрузки модулей ядра
      become: true
      ansible.builtin.copy:
        dest: /etc/modules-load.d/k8s.conf
        content: |
          overlay
          br_netfilter
        owner: root
        group: root
        mode: "0644"
      notify: Modprobe load

    - name: Создаем файл параметров ядра
      become: true
      ansible.builtin.copy:
        dest: /etc/sysctl.d/10-k8s.conf
        content: |
          net.bridge.bridge-nf-call-ip6tables = 1
          net.bridge.bridge-nf-call-iptables = 1
          net.ipv4.ip_forward = 1
        owner: root
```

```

    group: root
    mode: "0644"
    notify: Load sysctl

- name: Устанавливаем пакеты Kubernetes
  become: true
  ansible.builtin.apt:
    name:
      - kubeadm
      - kubectl
      - kubelet
    state: present

# Применение правил iptables на Kubernetes хостах

- name: Разрешить доступ к Kubernetes API Server (порт 6443 TCP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 6443
    jump: ACCEPT
    comment: "Kubernetes (API Server)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к etcd (порты 2379-2380 TCP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 2379:2380
    jump: ACCEPT
    comment: "Kubernetes (etcd)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к Kubelet API (порт 10250 TCP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 10250
    jump: ACCEPT
    comment: "Kubernetes (Kubelet API)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к NodePort (порты 30000-32767 TCP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 30000:32767
    jump: ACCEPT
    comment: "Kubernetes (NodePort)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к CNI Flannel (порт 8472 UDP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: udp
    destination_port: 8472
    jump: ACCEPT
    comment: "Kubernetes (CNI Flannel)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к CoreDNS/kube-dns (порт 53 TCP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: tcp
    destination_port: 53
    jump: ACCEPT
    comment: "Kubernetes (CoreDNS/kube-dns tcp)"
    state: present
  notify: Save iptables

- name: Разрешить доступ к CoreDNS/kube-dns (порт 53 UDP)
  become: true
  ansible.builtin.iptables:
    chain: INPUT
    protocol: udp
    destination_port: 53
    jump: ACCEPT
    comment: "Kubernetes (CoreDNS/kube-dns udp)"
    state: present
  notify: Save iptables

- name: Установить политику FORWARD ACCEPT с комментарием
  become: true
  ansible.builtin.command: iptables -P FORWARD ACCEPT -m comment --comment "Kubernetes (Kubernetes forward)"
  changed_when: false
  notify: Save iptables

# Установка cri-docker

- name: Скачиваем архив с cri-docker
  ansible.builtin.get_url:
    url: https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.16/cri-dockerd_0.3.16.3-0.debian-bookworm_amd64.deb
    dest: /tmp/cri-dockerd_0.3.16.3-0.debian-bookworm_amd64.deb
    mode: '0644'

- name: Устанавливаем cri-docker
  become: yes
  # apt модуль автоматически вызовет dpkg с зависимостями и сделает apt --fix-broken install при необходимости
  ansible.builtin.apt:

```

```

deb: /tmp/cri-dockerd_0.3.16.3-0.debian-bookworm_amd64.deb
state: present

- name: Скачиваем архив с crictl
  ansible.builtin.get_url:
    url: https://github.com/kubernetes-sigs/cri-tools/releases/download/v1.34.0/crictl-v1.34.0-linux-amd64.tar.gz
    dest: /tmp/crictl-v1.34.0-linux-amd64.tar.gz
    mode: '0644'

- name: Распаковка архива с crictl
  become: true
  ansible.builtin.unarchive:
    src: /tmp/crictl-v1.34.0-linux-amd64.tar.gz
    dest: /usr/local/bin
    remote_src: true
    mode: '0755'

- name: Создаём директорию для службы cri-docker
  become: true
  ansible.builtin.file:
    path: /etc/systemd/system/cri-docker.service.d
    state: directory
    mode: '0755'

- name: Создаём службу cri-docker из шаблона
  become: true
  ansible.builtin.template:
    src: 10-pause.conf.j2
    dest: /etc/systemd/system/cri-docker.service.d/10-pause.conf
    mode: '0644'

- name: Создание конфигурации сокета cri-docker из шаблона
  become: true
  ansible.builtin.template:
    src: crictl.yaml.j2
    dest: /etc/crictl.yaml
    mode: '0644'

- name: Перезагрузка systemd daemon
  become: true
  ansible.builtin.systemd:
    daemon_reload: true

- name: Включаем службу cri-docker
  become: true
  ansible.builtin.systemd:
    name: cri-docker.service
    enabled: true
    state: restarted
    changed_when: false

# Применяем все хэндлеры

- name: Flush handlers
  ansible.builtin.meta: flush_handlers

# Установка Kubernetes на первых хост

- name: Проверяем, инициализирован ли уже кластер по наличию файла /var/lib/kubelet/config.yaml
  become: true
  ansible.builtin.stat:
    path: /var/lib/kubelet/config.yaml
  register: kubeadm_config_exist

- name: debug
  ansible.builtin.debug:
    msg: "when: not {{ kubeadm_config_exist.stat.exists }} and {{ inventory_hostname }} == {{ first_node }}, playbook_dir: {{ playbook_dir }}"

- name: Установка первого хоста с Kubernetes
  # Применить настройки только для первого хоста из переменной first_node
  when: "not kubeadm_config_exist.stat.exists and 'combos' in group_names and inventory_hostname == first_node"
  block:

    # Создаём первую ноду Kubernetes

    - name: Инициализация Kubernetes control plane через kubeadm
      become: true
      ansible.builtin.command: >
        kubeadm init
        --kubernetes-version=v1.32.5
        --pod-network-cidr=10.244.0.0/16
        --control-plane-endpoint 192.168.20.101:7443
        --upload-certs
        --cri-socket unix:///var/run/cri-dockerd.sock
      args:
        creates: /etc/kubernetes/admin.conf
      register: kubeadm_init_result
      changed_when: "'[init] Using Kubernetes version' in kubeadm_init_result.stdout or kubeadm_init_result.rc == 0"
      failed_when: kubeadm_init_result.rc != 0 and "'already initialized' not in kubeadm_init_result.stderr"

    - name: Забираем файл с init выводом на управляющий хост
      ansible.builtin.copy:
        content: |
          {{ kubeadm_init_result.stdout }}
        dest: "{{ playbook_dir }}/artifacts/kubeadm_init.txt"
        owner: "{{ ansible_facts['user_id'] }}"
        group: "{{ ansible_facts['user_id'] }}"
        mode: "0666"
      delegate_to: localhost

    - name: Забираем файл admin.conf на контроллер
      become: true
      ansible.builtin.fetch:
        src: /etc/kubernetes/admin.conf
        dest: "{{ playbook_dir }}/artifacts/admin.conf"
        flat: true

# Копируем конфигурацию к пользователю administrator

- name: Создать директорию .kube в домашней папке

```

```

ansible.builtin.file:
  path: "/home/{{ ansible_facts['user_id'] }}/.kube"
  state: directory
  owner: "{{ ansible_facts['user_id'] }}"
  group: "{{ ansible_facts['user_id'] }}"
  mode: "0700"

- name: Скопировать admin.conf в /home/administrator/.kube/config
  become: true
  ansible.builtin.copy:
    src: /etc/kubernetes/admin.conf
    dest: "/home/{{ ansible_facts['user_id'] }}/.kube/config"
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"
    mode: "0644"
    remote_src: true

# Устанавливаем сетевой драйвер Flannel

- name: Скачиваем Flannel манифест
  ansible.builtin.get_url:
    url: https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
    dest: /tmp/kube-flannel.yml
    mode: '0644'

- name: Запускаем установку Flannel CNI через kubectl
  ansible.builtin.shell: |
    kubectl apply -f /tmp/kube-flannel.yml
  changed_when: true

- name: Удаляем временный файл манифеста Flannel
  ansible.builtin.file:
    path: /tmp/kube-flannel.yml
    state: absent

- name: Пауза после установки драйвера Flannel 40 секунд
  ansible.builtin.pause:
    seconds: 40

# Установка Kubernetes на остальные хосты

- name: debug
  ansible.builtin.debug:
    msg: "when: not {{ kubeadm_config_exist.stat.exists }} and 'compos' in {{ group_names }} and {{ inventory_hostname }} != {{ first_node }}"
  playbook_dir: {{ playbook_dir }}"

- name: Присоединение хостов к Kubernetes
  # Применить настройки только для хостов compos-* которые не являются первым хостом
  when: "not kubeadm_config_exist.stat.exists and 'compos' in group_names and inventory_hostname != first_node"
  block:

  - name: Читаем файл artifacts/kubeadm_init.txt
    ansible.builtin.slurp:
      src: "{{ playbook_dir }}/artifacts/kubeadm_init.txt"
      register: init_output_base64
      delegate_to: localhost

  - name: Нормализуем файл с командой из base64 в текст
    ansible.builtin.set_fact:
      init_output_text: "{{ init_output_base64.content | b64decode }}"

  - name: Получаем команду kubeadm join для подключения к кластеру
    ansible.builtin.set_fact:
      join_cmd_cp: >-
      {{
        init_output_text |
        regex_search(
          '(kubeadm join .+)\n\t*(.+)\n\t*(.+)',
          '\1',
          '\2',
          '\3',
          multiline=True,
          ignorecase=True
        ) |
        join('')
      }}
      --cri-socket unix:///var/run/cri-dockerd.sock

  - name: debug
    become: true
    ansible.builtin.debug:
      # msg: "Полученная команда подключения к кластеру: {{ join_cmd_cp }}"
      var: join_cmd_cp

  - name: Запускаем join к Kubernetes control plane через kubeadm
    become: true
    ansible.builtin.command: "{{ join_cmd_cp }}"
    args:
      creates: /etc/kubernetes/admin.conf
      register: kubeadm_init_result
      changed_when: "[init] Using Kubernetes version' in kubeadm_init_result.stdout or kubeadm_init_result.rc == 0"
      failed_when: kubeadm_init_result.rc != 0 and "'already initialized' not in kubeadm_init_result.stderr"

# Копируем конфигурацию к пользователю administrator

- name: Создать директорию .kube в домашней папке
  ansible.builtin.file:
    path: "/home/{{ ansible_facts['user_id'] }}/.kube"
    state: directory
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"
    mode: "0700"

- name: Скопировать admin.conf в /home/administrator/.kube/config
  become: true
  ansible.builtin.copy:
    src: /etc/kubernetes/admin.conf
    dest: "/home/{{ ansible_facts['user_id'] }}/.kube/config"
    owner: "{{ ansible_facts['user_id'] }}"
    group: "{{ ansible_facts['user_id'] }}"

```

```

    mode: "0644"
    remote_src: true

- name: Получаем сценарий автодополнения аргументов kubectl
  ansible.builtin.command: kubectl completion bash
  register: kubectl_completion
  changed_when: false

- name: Добавляем сценарий автодополнения аргументов для kubectl в ~/.bashrc
  ansible.builtin.blockinfile:
    path: "{{ ansible_facts['user_dir'] }}/.bashrc"
    marker: "# {mark} ANSIBLE MANAGED BLOCK: kubectl completion"
    block: |
      {{ kubectl_completion.stdout | indent(0) }}
    create: true

# Принудительно запустить всех активированных в этой роли handlers
- name: Flush handlers
  ansible.builtin.meta: flush_handlers

```

Редактируем реакции:

```

vim roles/install_kubernetes/handlers/main.yml

---
- name: Modprobe load
  become: true
  ansible.builtin.command: modprobe -v {{ item }}
  loop:
    - overlay
    - br_netfilter
  failed_when: false
  changed_when: true

- name: Load sysctl
  become: true
  ansible.builtin.command: sysctl -f /etc/sysctl.d/10-k8s.conf
  failed_when: false
  changed_when: true

- name: Save iptables
  become: true
  ansible.builtin.command: netfilter-persistent save
  changed_when: true

```

Добавляем роль в сценарий запуска для **gateways**:

```

vim gateways.yml

# ...
roles:
  # ...
  - role: install_kubernetes
    tags: [install_kubernetes]

```

Запускаем обработку конкретно этой команды для **gateways**:

```

ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_kubernetes

```

Добавляем роль в сценарий запуска для **combos**:

```

vim combos.yml

# ...
roles:
  # ...
  - role: install_kubernetes
    tags: [install_kubernetes]

```

Запускаем обработку конкретно этой команды для **combos**:

```

ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_kubernetes

```

3.4.14 Установка Etcdctl

Устанавливаем утилиту для контроля базы данных ETCD. Создаём роль:

```

cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_etcdctl

```

Редактируем задание:

```

vim roles/install_etcdctl/tasks/main.yml

---
- name: Проверяем наличие etcdctl
  become: yes
  ansible.builtin.stat:
    path: /usr/local/bin/etcdctl
  register: etcdctl_exists

- name: Если etcdctl ещё не установлен, устанавливаем его
  when: etcdctl_exists.stat.exists == false
  block:

  - name: Скачиваем etcdctl .tar.gz
    ansible.builtin.get_url:
      url: https://github.com/etcd-io/etcd/releases/download/v3.5.16/etcd-v3.5.16-linux-amd64.tar.gz
      dest: /tmp/etcd-v3.5.16-linux-amd64.tar.gz # Куда скачивать пакет (установку будем делать из /tmp директории
      mode: '0644'

  - name: Распаковка архива с etcdctl
    become: true
    ansible.builtin.unarchive:
      src: /tmp/etcd-v3.5.16-linux-amd64.tar.gz
      dest: /usr/local/bin
      remote_src: true
      mode: '0755'
      extra_opts:
        - '--strip-components=1'
        - '--exclude=*.md'

```

```
- '--exclude=etcd'
- '--exclude=Documentation'
```

Добавляем роль в сценарий запуска для **gateways**:

```
vim gateways.yml

# ...
roles:
# ...
- role: install_etcdctl
tags: [install_etcdctl]
```

Запускаем обработку конкретно этой команды для **gateways**:

```
ansible-playbook -i inventory/hosts.yaml gateways.yml --tags install_etcdctl
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml

# ...
roles:
# ...
- role: install_etcdctl
tags: [install_etcdctl]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_etcdctl
```

3.4.15 Установка репозитория gitlab-runner

Устанавливаем gitlab-runner для GitLab. Создаём роль:

```
cd ~/ansible/
source .venv/bin/activate
ansible-galaxy init roles/install_gitlab-runner
```

Создаём шаблон для описания репозитория:

```
vim roles/install_gitlab-runner/templates/gitlab-runner.list.j2

# this file was generated by packages.gitlab.com for
# the repository at https://packages.gitlab.com/runner/gitlab-runner

deb [signed-by=/etc/apt/keyrings/runner_gitlab-runner-archive-keyring.asc] https://packages.gitlab.com/runner/gitlab-runner/ubuntu/ noble main
deb-src [signed-by=/etc/apt/keyrings/runner_gitlab-runner-archive-keyring.asc] https://packages.gitlab.com/runner/gitlab-runner/ubuntu/ noble main
```

Редактируем задание:

```
vim roles/install_gitlab-runner/tasks/main.yml

---
- name: Установка ключей репозитория gitlab-runner
  become: true
  ansible.builtin.get_url:
    url: "https://packages.gitlab.com/runner/gitlab-runner/gpgkey"
    dest: "/etc/apt/keyrings/runner_gitlab-runner-archive-keyring.asc"
    mode: '0644'

- name: Создать gitlab-runner репозитория
  become: true
  ansible.builtin.template:
    src: gitlab-runner.list.j2
    dest: /etc/apt/sources.list.d/gitlab-runner.list
    mode: '0644'

- name: Устанавливаем gitlab-runner
  become: true
  ansible.builtin.apt:
    name:
      - gitlab-runner-helper-images=17.11.4-1
      - gitlab-runner=17.11.4-1
    state: present
    update_cache: true
```

Добавляем роль в сценарий запуска для **combos**:

```
vim combos.yml

# ...
roles:
# ...
- role: install_gitlab-runner
tags: [install_gitlab-runner]
```

Запускаем обработку конкретно этой команды для **combos**:

```
ansible-playbook -i inventory/hosts.yaml combos.yml --tags install_gitlab-runner
```

3.4.16 Завершаем работу с Ansible

Выходим из виртуального окружения:

```
deactivate
cd ~
```

3.5 Подключение kubectl на gateway.dev.lan в управление Kubernetes

Создаём на сервере gateway.dev.lan X.509 сертификат, подписываем его в кластере Kubernetes и заходим после этого через него с помощью kubectl. Создаём приватный ключ в папке ~/ . kube:

```
openssl genrsa -out ~/.kube/key.key 2048
```

На основе приватного ключа создаём запрос CSR (Certificate Signing Request):

```
openssl req -new -key ~/.kube/key.key -out ~/.kube/csr.csr -subj "/CN=administrator/O=system:masters"
```

где:

- Common Name (CN) = имя пользователя — здесь administrator;
- Organization (O) = группа — здесь группа администраторов кластера system:masters (имеет полный доступ, только для администраторов). Если нужна своя группа, например O=team, нужно будет настроить RBAC.

Для подписи понадобятся:

- ca.crt — публичный сертификат СА кластера
- ca.key — приватный ключ СА (он находится только у администратора)

Подписываем ключ на первой ноте Kubernetes combo-1.dev.lan, где эти ключи и сертификаты есть и вернём обратно:

```
# Копируем запрос на подпись на сервер combo-1:
scp ~/.kube/csr.csr administrator@combo-1.dev.lan:~

# Подписываем его на нём сертификатом и ключём Kubernetes на 1 год:
ssh administrator@combo-1.dev.lan 'sudo openssl x509 -req -in csr.csr -CA /etc/kubernetes/pki/ca.crt -CAkey /etc/kubernetes/pki/ca.key -CAcreateserial -out crt.crt -days 365'

# Копируем готовый сертификат обратно в папку ~/.kube:
scp administrator@combo-1.dev.lan:~/crt.crt ~/.kube

# Копируем сертификат кластера:
scp administrator@combo-1.dev.lan:/etc/kubernetes/pki/ca.crt ~/.kube

# Удаляем на combo-1 уже не нужные файлы:
ssh administrator@combo-1.dev.lan 'sudo rm {csr,crt}.*'
```

Теперь у нас есть:

- key.key — приватный ключ;
- crt.crt — клиентский сертификат;
- ca.crt — сертификат кластера.

Определяем IP и порт кластера:

```
ssh administrator@combo-1.dev.lan 'kubectl cluster-info'

Kubernetes control plane is running at https://192.168.20.101:7443
CoreDNS is running at https://192.168.20.101:7443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Для теста, делаем прямой запрос к кластеру через API на возврат списка Namespaces:

```
curl -s --cacert ~/.kube/ca.crt --cert ~/.kube/crt.crt --key ~/.kube/key.key \
https://192.168.20.101:7443/api/v1/namespaces | jq -r '.items[].metadata.name'

default
kube-flannel
kube-node-lease
kube-public
kube-system
```

Создаём базовый файл ~/.kube/config с данными кластера, чтобы kubectl на gateway.dev.lan смог к нему подключиться:

```
kubectl config set-cluster kubernetes --server=https://192.168.20.101:7443 --certificate-authority=$HOME/.kube/ca.crt --embed-certs=true

Cluster "kubernetes" set.
```

где:

- kubernetes — имя кластера.

Добавляем в ~/.kube/config данные пользователя:

```
kubectl config set-credentials kubernetes-admin --client-certificate=$HOME/.kube/crt.crt --client-key=$HOME/.kube/key.key \
--embed-certs=true

User "kubernetes-admin" set.
```

где:

- kubernetes-admin — имя пользователя - администратора кластера.

Добавляем в ~/.kube/config контекст:

```
kubectl config set-context kubernetes-admin@kubernetes --cluster=kubernetes --user=kubernetes-admin

Context "kubernetes-admin@kubernetes" created.
```

где:

- kubernetes-admin@kubernetes — имя контекста.

Указываем в ~/.kube/config использовать созданный контекст:

```
kubectl config use-context kubernetes-admin@kubernetes

Switched to context "kubernetes-admin@kubernetes".
```

В результате, на gateway.dev.lan создаётся точно такой же файл ~/.kube/config, как и на серверах combo-1,2,3, где только сертификат пользователя будет другой.

Проверяем, что kubectl подключён к кластеру Kubernetes:

```
kubectl get nodes

NAME      STATUS    ROLES    AGE   VERSION
combo-1   Ready     control-plane   25h   v1.32.11
combo-2   Ready     control-plane   25h   v1.32.11
combo-3   Ready     control-plane   25h   v1.32.11
```

3.5.1 Снимаем taint с кластера для разрешение запуска Pod-с на Control Plane

Т.к. в данной конфигурации Worker-nodes должны работать на Control-plane нодах, необходимо снять с них ограничения taint.

Проверяем, что ограничения установлены на примере ноды combo-1:

```
kubectl describe node combo-1 | grep Taints:

Taints:                node-role.kubernetes.io/control-plane:NoSchedule
```

Такой вывод показывает, что установлено ограничение, запрещающее планировать поды на этот хост Control-plane.

Убираем ограничение с Control-plane хостов, чтобы использовать их как рабочие ноды:

```
kubectl taint node combo-1 node-role.kubernetes.io/control-plane-
kubectl taint node combo-2 node-role.kubernetes.io/control-plane-
kubectl taint node combo-3 node-role.kubernetes.io/control-plane-
```

```
node/combo-1 untainted
node/combo-2 untainted
node/combo-3 untainted
```

Проверяем, что ограничения сняты на примере ноды combo-1:

```
kubectl describe nodes | grep Taints:

Taints:                <none>
Taints:                <none>
Taints:                <none>
```

3.6 Установка Traefik

Docker Hub — https://hub.docker.com/_/traefik

Traefik — обратный проху, чтобы мы могли на одном хосте иметь доступ к нескольким контейнерам. Создаём директорию проекта:

```
mkdir -p ~/compose/traefik
cd ~/compose/traefik
```

Создаём файл Docker-compose. Правила захода на внутренний Dashboard выносим в отдельный файл:

```
vim docker-compose.yaml
```

```
# Версия формата docker-compose файла (3.7 поддерживает расширенные возможности)
version: "3.7"

# Секция определения сервисов
services:
  # Сервис Traefik (обратный прокси и балансировщик нагрузки)
  traefik:
    # Используем официальный образ Traefik с конкретной версией для стабильности
    image: "traefik:v2.11.31"
    # Явное задание имени контейнера для удобства
    container_name: "traefik"
    # Задаем hostname для контейнера
    hostname: "traefik"
    # Политика перезапуска: всегда (при падении или перезагрузке докера)
    restart: always
    # Пропускаем порты:
    ports:
      - "80:80" # порт для HTTP
      - "443:443" # порт для HTTPS
      - "5000:5000" # для Docker Registry

    environment:
      - "TZ=Europe/Moscow"

    # Указываем внешний DNS для доступа в Интернет из контейнера и к внешним именам служб в нём.
    dns: "192.168.20.1"

    command:
      # Уровень логирования (можно изменить на debug для отладки)
      - "--log.level=info"
      - "--providers.docker=true"
      # Включение (true) или отключения (false) Dashboard отладочного WEB-интерфейса Traefik.
      - http://traefik.dev.lan/
      - "--api.dashboard=true"
      - "--api.insecure=true" # Dashboard будет доступен без авторизации
      # Не экспортировать все контейнеры по умолчанию (только с явными метками)
      - "--providers.docker.exposedbydefault=false"

      # Правила будем получать из смонтированной папки ./rules.
      - "--providers.file=true"
      - "--providers.file.directory=/rules" # Директория для dynamic config
      - "--providers.file.watch=true"

      # Определение endpoint для HTTP
      - "--entrypoints.web.address=:80"
      # Определение endpoint для HTTPS
      - "--entrypoints.websecure.address=:443"

      # Перенаправление трафика с HTTP на HTTPS
      - "--entrypoints.web.http.redirects.entrypoint.to=websecure"
      - "--entrypoints.web.http.redirects.entrypoint.scheme=https"
      # Адрес локального сервера сертификации по протоколу ACME - Step CA
      - "--certificatesresolvers.stepca.acme.caserver=https://ca.dev.lan:9443/acme/acme/directory"
      - "--certificatesresolvers.stepca.acme.email=admin@dev.lan"
      # Хранилище полученных от сервера сертификации ключей Traefik-ом
      - "--certificatesresolvers.stepca.acme.storage=/acme.json"
      - "--certificatesresolvers.stepca.acme.tlschallenge=true"

# Монтируем сокет докера для автоматического обнаружения сервисов
volumes:
  - ./rules:/rules:ro # Монтируем локальную папку rules со списком правил
  - /var/run/docker.sock:/var/run/docker.sock:ro # Контроль за активностью Docker
  - /acme.json:/acme.json # Файл для хранения сертификатов.
  # Прокладываем хранилище цифровых ключей с хостовой машины в контейнер,
  # чтобы он начал доверять сертификату от Step CA, корневого ключ которого установлен в хостовую систему
  - /etc/ssl/certs:/etc/ssl/certs:ro

labels:
  # Включаем Traefik в этом контейнере для доступа к его Dashboard и другим правилам
```

```

- "traefik.enable=true"
# Сами правила вынесены в отдельные YAML в папку ./rules

networks:
- traefik-public

# Контейнер для тестов
whoami:
image: traefik/whoami:v1.11.0
container_name: whoami
environment:
- "TZ=Europe/Moscow"
labels:
- "traefik.enable=true"
- "traefik.http.routers.whoami.rule=Host(`whoami.dev.lan`)"
- "traefik.http.routers.whoami.entrypoints=websecure"
# Сертификат TLS заправлять у Step CA
- "traefik.http.routers.whoami.tls.certresolver=stepca"
- "traefik.http.routers.whoami.service=whoami"
- "traefik.http.services.whoami.loadbalancer.server.url=http://whoami:80"
networks:
- traefik-public

# Сеть Traefik.
# Сначала её нужно добавить командой:
# docker network create traefik-public
networks:
traefik-public:
external: true # Сеть внешняя

```

Правила для Dashboard Traefik:

```

mkdir -p rules
vim rules/dashboard.yaml

http:
routers:
traefik-dashboard:
rule: Host(`traefik.dev.lan`)
entrypoints: websecure
service: api@internal
tls:
certresolver: stepca

```

Создаём файл для хранения сертификатов асме.json и устанавливаем права записи/чтения только для владельца:

```

touch acme.json
chmod 600 acme.json

```

Создаём сеть, указанную в Docker-compose файле:

```

docker network create traefik-public

2fc38bd1532f9cd121d2556aa09fd57922de93b95fe06152965c82543b823cda

```

Запускаем контейнер:

```

docker-compose up -d

```

Для проверки работоспособности, смотрим лог контейнера (что нет записей типа **error**):

```

docker logs -f traefik

```

Проверяем, что возвращается правильный сертификат с stepca, а не самоподписанный самим Traefik:

```

openssl s_client -connect localhost:443 -servername traefik.dev.lan | openssl x509 -noout -issuer -subject

depth=2 0 = My Dev CA, CN = My Dev CA Root CA
verify return:1
depth=1 0 = My Dev CA, CN = My Dev CA Intermediate CA
verify return:1
depth=0 CN = traefik.dev.lan
verify return:1
issuer=0 = My Dev CA, CN = My Dev CA Intermediate CA
subject=CN = traefik.dev.lan

```

Зайти во внутрь контейнера:

```

docker exec -it $(docker ps -q -f name=traefik) /bin/sh

#

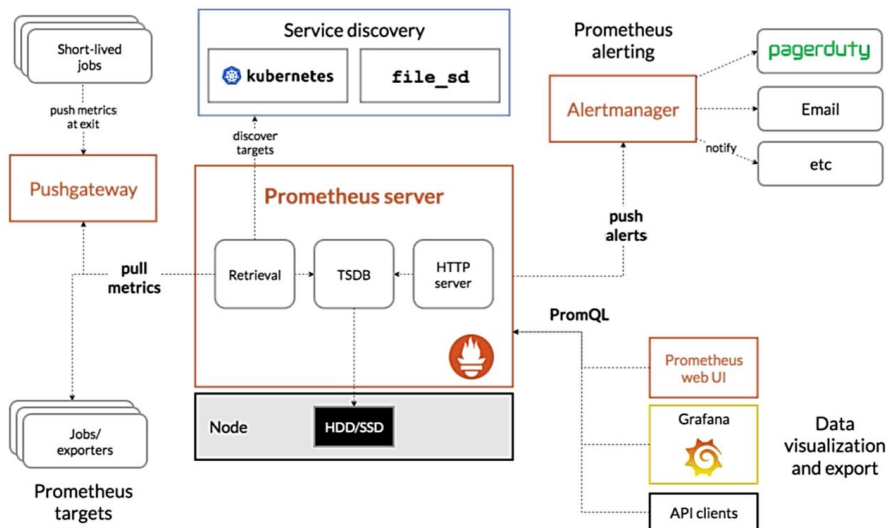
```

Заходим в панель настройки Traefik:

- <https://traefik.dev.lan>
- <https://whoami.dev.lan>

3.7 Настройка мониторинга

3.7.1 Установка Prometheus



Prometheus устанавливаем в Docker через docker-compose. Используем официальный образ из <https://hub.docker.com/u/prom>.

```
mkdir -p ~/compose/prometheus
cd ~/compose/prometheus
vim docker-compose.yaml
```

```
version: "3.7"

services:
  prometheus:
    image: "prom/prometheus:v3.8.1"
    container_name: "prometheus"
    hostname: "prometheus"
    restart: unless-stopped

    environment:
      - "TZ=Europe/Moscow"

    # Указываем внешний DNS для доступа в Интернет из контейнера и к внешним именам служб в нём.
    dns: "192.168.20.1"

    command:
      - "--config.file=/etc/prometheus/prometheus.yml"

    # Монтируем сокет докера для автоматического обнаружения сервисов
    volumes:
      - ".:/etc/prometheus"
      - ".:/data:/prometheus"
      # Прокидываем хранилище цифровых ключей с хостовой машины в контейнер,
      # чтобы он начал доверять сертификату от Step CA, корневой ключ которого установлен в хостовую систему
      - "/etc/ssl/certs/./etc/ssl/certs/:ro"

    labels:
      # Включаем Traefik в этом контейнере для доступа к его Dashboard.
      - "traefik.enable=true"
      - "traefik.http.routers.prometheus.rule=Host(`prometheus.dev.lan`)"
      - "traefik.http.routers.prometheus.entrypoints=websecure"
      - "traefik.http.services.prometheus.loadbalancer.server.port=9090"
      # Сертификат TLS заправлять у Step CA
      - "traefik.http.routers.prometheus.tls.certresolver=stepca"

    networks:
      - traefik-public

networks:
  traefik-public:
    external: true # Сеть внешняя
```

Создаём директорию под данные Prometheus. Prometheus работает от пользователя nobody:nogroup, поэтому выставляет эти права на директорию для данных:

```
mkdir -p data
sudo chown 65534:65534 ./data
```

Создаём файл конфигурации:

```
vim prometheus.yml

global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

    relabel_configs:
      # Указываем имя ноды по имен, т.е. gateway
      - target_label: instance
        replacement: gateway
```

Запускаем docker-compose:

```
docker-compose up -d
```

Для проверки работоспособности, смотрим лог контейнера (что нет записей типа **error**):

```
| docker logs -f prometheus
```

Проверяем, что возвращается правильный сертификат с stepca, а не самоподписанный самим Traefik:

```
| openssl s_client -connect localhost:443 -servername prometheus.dev.lan | openssl x509 -noout -issuer -subject
```

```
depth=2 0 = My Dev CA, CN = My Dev CA Root CA
verify return:1
depth=1 0 = My Dev CA, CN = My Dev CA Intermediate CA
verify return:1
depth=0  CN = traefik.dev.lan
verify return:1
issuer=0 = My Dev CA, CN = My Dev CA Intermediate CA
subject=CN = prometheus.dev.lan
```

Зайти во внутрь контейнера:

```
| docker exec -it $(docker ps -q -f name=prometheus) /bin/sh
```

```
#
```

Заходим в панель Prometheus:

- <https://prometheus.dev.lan>

3.7.2 Установка Grafana

Grafana устанавливаем в Docker через docker-compose. Используем официальные образ из <https://hub.docker.com/u/grafana>.

```
| mkdir -p ~/compose/grafana
| cd ~/compose/grafana
```

Теперь нужно из образа Grafana скопировать существующую заполненную конфигурацию через временный контейнер:

```
# Получаем необходимый образ:
docker pull grafana/grafana:12.4.0-20648027705-ubuntu

# Создаём контейнер без его запуска:
docker create --name temp-grafana grafana/grafana:12.4.0-20648027705-ubuntu
# Посмотреть созданный контейнер можно командой 'docker container ls -a'

# Копируем из контейнера конфигурацию grafana.ini в текущую директорию:
docker cp temp-grafana:/etc/grafana/grafana.ini .

# Удаляем временный контейнер:
docker rm temp-grafana
```

Создаём директории для проброса и устанавливаем правильные права:

```
# Создаём рабочие директории
mkdir -p grafana-data
# Применяем к ним права, установленные в контейнере grafana
sudo chown 472:472 grafana-data
```

```
vim docker-compose.yml
```

```
version: "3.7"

services:

  grafana:

    image: "grafana/grafana:12.4.0-20648027705-ubuntu"
    container_name: "grafana"
    hostname: "grafana"
    restart: unless-stopped

    # Указываем внешний DNS для доступа в Интернет из контейнера и к внешним именам служб в нём.
    dns: "192.168.20.1"

    environment:
      - "TZ=Europe/Moscow"
      - "GF_SERVER_ROOT_URL=https://grafana.dev.lan"
      - "GF_SECURITY_ADMIN_USER=admin"
      - "GF_SECURITY_ADMIN_PASSWORD=admin"
      - "GF_AUTH_ANONYMOUS_ENABLED=false"

    # Монтируем сокет докера для автоматического обнаружения сервисов
    volumes:
      - "./grafana-data:/var/lib/grafana"
      - "./grafana.ini:/etc/grafana/grafana.ini"
      # Прокидываем хранилище цифровых ключей с хостовой машины в контейнер,
      # чтобы он начал доверять сертификату от Step CA, корневой ключ которого установлен в хостовую систему
      - "/etc/ssl/certs:/etc/ssl/certs:ro"

    labels:
      # Включаем Traefik в этом контейнере для доступа к его Dashboard.
      - "traefik.enable=true"
      - "traefik.http.routers.grafana.rule=Host(`grafana.dev.lan`)"
      - "traefik.http.routers.grafana.entrypoints=websecure"
      - "traefik.http.services.grafana.loadbalancer.server.port=3000"
      # Сертификат TLS заправлять у Step CA
      - "traefik.http.routers.grafana.tls.certresolver=stepca"

    networks:
      - traefik-public

networks:
  traefik-public:
    external: true # Сеть внешняя
```

Запускаем docker-compose:

```
| docker-compose up -d
```

Для проверки работоспособности, смотрим лог контейнера (что нет записей типа **error**):

```
| docker logs -f grafana
```

Проверяем, что возвращается правильный сертификат с stepca, а не самоподписанный самим Traefik:

```
| openssl s_client -connect localhost:443 -servername grafana.dev.lan | openssl x509 -noout -issuer -subject
```

```
depth=2 0 = My Dev CA, CN = My Dev CA Root CA
verify return:1
depth=1 0 = My Dev CA, CN = My Dev CA Intermediate CA
verify return:1
depth=0 CN = traefik.dev.lan
verify return:1
issuer=0 = My Dev CA, CN = My Dev CA Intermediate CA
subject=CN = grafana.dev.lan
```

Зайти во внутрь контейнера:

```
docker exec -it $(docker ps -q -f name=grafana) /bin/bash
```

```
grafana@grafana:/usr/share/grafana$
```

Заходим в панель Grafana:

- <https://grafana.dev.lan>

При первом входе, меняем пароль на **Pa\$\$w0rd**.

Подключаем Prometheus в качестве базы данных. Для этого заходим:

- **Menu -> Connections -> Data sources -> + Add new data sources -> Prometheus:**
- **Prometheus server URL** — `http://prometheus:9090`
- Кнопка **Save & test**

3.7.3 Установка node_exporter для нод Kubernetes

Node_exporter разворачиваем через DaemonSet Kubernetes:

```
mkdir -p ~/kubernetes/monitoring
cd ~/kubernetes/monitoring
```

Создаём namespace для использования мониторинга:

```
kubectl create namespace monitoring
```

```
namespace/monitoring created
```

Создаём манифест:

```
vim node_exporter.yaml
```

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: node-exporter
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-exporter
rules:
- apiGroups: [""]
  resources: ["nodes", "nodes/proxy", "services", "endpoints", "pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-exporter
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: node-exporter
subjects:
- kind: ServiceAccount
  name: node-exporter
  namespace: monitoring
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      serviceAccountName: node-exporter
      # Метрики доступны по IP ноды
      hostNetwork: true
      hostPID: true
      containers:
      - name: node-exporter
        image: prom/node-exporter:v1.10.2
        args:
        - --path.procfs=/host/proc
        - --path.sysfs=/host/sys
        - --path.rootfs=/host/root
        - --collector.filesystem.ignored-mount-points="^(dev|proc|sys|var/lib/docker/.+)($/)"
        - --collector.filesystem.ignored-fs-
types="^(autofs|binfmt_misc|cgroup|configfs|debugfs|devpts|devtmpfs|fusectl|hugetlbfs|mqqueue|overlay|proc|procfs|pstore|rpc_pipefs|securityfs|sysfs|tracefs)$"
        ports:
        - name: metrics
          containerPort: 9100
          hostPort: 9100
```

```
resources:
  limits:
    cpu: 100m
    memory: 128Mi
  requests:
    cpu: 50m
    memory: 64Mi
volumeMounts:
- name: proc
  mountPath: /host/proc
  readOnly: true
- name: sys
  mountPath: /host/sys
  readOnly: true
- name: root
  mountPath: /host/root
  readOnly: true
volumes:
- name: proc
  hostPath:
    path: /proc
- name: sys
  hostPath:
    path: /sys
- name: root
  hostPath:
    path: /
---
apiVersion: v1
kind: Service
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app: node-exporter
spec:
  clusterIP: None # headless
  ports:
  - name: metrics
    port: 9100
    targetPort: 9100
  selector:
    app: node-exporter
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f node_exporter.yaml

serviceaccount/node-exporter created (dry run)
clusterrole.rbac.authorization.k8s.io/node-exporter created (dry run)
clusterrolebinding.rbac.authorization.k8s.io/node-exporter created (dry run)
daemonset.apps/node-exporter created (dry run)
service/node-exporter created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f node_exporter.yaml

serviceaccount/node-exporter created
clusterrole.rbac.authorization.k8s.io/node-exporter created
clusterrolebinding.rbac.authorization.k8s.io/node-exporter created
daemonset.apps/node-exporter created
service/node-exporter created
```

Проверяем результат в объекте DaemonSet:

```
kubectl get daemonsets.apps -n monitoring -o wide
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE	CONTAINERS	IMAGES	SELECTOR
node-exporter	3	3	3	3	3	<none>	30m	node-exporter	prom/node-exporter:v1.10.2	app=node-exporter

Смотрим распределение Pod-ов:

```
kubectl get pods -n monitoring -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
node-exporter-c465t	1/1	Running	0	2m23s	192.168.20.12	combo-2	<none>	<none>
node-exporter-d4p9m	1/1	Running	0	2m29s	192.168.20.11	combo-1	<none>	<none>
node-exporter-pmz75	1/1	Running	0	2m23s	192.168.20.13	combo-3	<none>	<none>

Проверяем, что на хосте combo-1.dev.lan порт :9100 выведен наружу:

```
ssh combo-1.dev.lan 'sudo ss -tulnp' | grep 9100
```

tcp	LISTEN	0	4096		*:9100	*:*	users:(("node_exporter",pid=216028,fd=3))
-----	--------	---	------	--	--------	-----	---

Проверяем доступность метрик на хосте combo-1.dev.lan:

```
curl -v http://combo-1.dev.lan:9100/metrics
```

```
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.101e-05
go_gc_duration_seconds{quantile="0.25"} 2.101e-05
go_gc_duration_seconds{quantile="0.5"} 2.3323e-05
go_gc_duration_seconds{quantile="0.75"} 2.3323e-05
go_gc_duration_seconds{quantile="1"} 2.3323e-05
go_gc_duration_seconds_sum 4.4333e-05
go_gc_duration_seconds_count 2
...
```

Настраиваем Prometheus на автоматический поиск хостов с node_exporter. Для этого для Prometheus создаём ServiceAccount и Secret с сохранённым туда токеном доступа. Переходим в папку с настройками для Kubernetes:

```
cd ~/kubernetes/monitoring/
vim prometheus.yaml
```

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
```

```

  name: prometheus
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
- apiGroups: [""]
  resources: ["nodes", "services", "endpoints", "pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: prometheus
  namespace: monitoring
---
# Создаём токен и помещаем его в Secret по ключу prometheus-token
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  namespace: monitoring
  name: prometheus-token
  annotations:
    kubernetes.io/service-account.name: prometheus

```

Применяем спецификацию:

```
| kubectl apply -f prometheus.yaml
```

```

serviceaccount/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
secret/prometheus-token created

```

Получаем токен и сохраняем его в директорию с настройками Prometheus:

```
| kubectl -n monitoring get secret prometheus-token -o jsonpath='{.data.token}' | base64 -d > ~/compose/prometheus/token
```

Переходим в папку с настройками docker-compose для Prometheus:

```
| cd ~/compose/prometheus
```

Копируем сертификат СА кластера Kubernetes:

```
| # Копируем сертификат СА:
cp ~/.kube/ca.crt .
```

Дополняем конфигурацию:

```

| vim prometheus.yml

# ...

- job_name: 'node-exporter'

# Настройка автоматического обнаружения нод Kubernetes
kubernetes_sd_configs:
- role: node # обнаруживает все ноды кластера
  api_server: https://192.168.20.101:7443 # Адрес и порт кластера
  tls_config:
    ca_file: /etc/prometheus/ca.crt # Сертификат кластера Kubernetes
    bearer_token_file: /etc/prometheus/token # Токен доступа к кластеру

# Фильтруем только ноды, где поднят node-exporter (по наличию эндпоинта 9100)
relabel_configs:

# Берём IP ноды (не пода!)
- source_labels: [__address__]
  regex: '(.*)\:.*'
  target_label: __address__
  replacement: '${1}:9100'

# Присваиваем имя job и instance
- target_label: job
  replacement: node-exporter
- target_label: instance
  source_labels: [__meta_kubernetes_node_name]

```

Перезагружаем Prometheus:

```
| docker-compose down
| docker-compose up -d
```

Создаём рабочие столы в Grafana через импорт специализированный Dashboards:

- **Menu -> Dashboards -> New -> Import**
- Указываем **ID** — 11074
- Нажимаем **Load**
- Выбрать источником данных нужную БД **Prometheus**
- Нажимаем **Import**

После этого в разделе **Dashboards** появится соответствующая панель.

3.7.4 Установка node_exporter на сервер gateway

На сервере **gateway** скачиваем архив с node_exporter:

```
wget -P /tmp https://github.com/prometheus/node_exporter/releases/download/v1.10.2/node_exporter-1.10.2.linux-amd64.tar.gz
```

Распаковываем архив:

```
sudo tar -xzf /tmp/node_exporter-1.10.2.linux-amd64.tar.gz --strip-components=1 -C /usr/local/bin/ --exclude={LICENSE,NOTICE}
```

Создаём сервис для запуска:

```
sudo tee /etc/systemd/system/node-exporter.service << 'EOF' > /dev/null
[Unit]
Description=Node exporter
After=network-online.target

[Service]
User=root
Restart=on-failure
ExecStart=/usr/local/bin/node_exporter

[Install]
WantedBy=multi-user.target
EOF
```

Запускаем службу, поставив её в автозагрузку:

```
sudo systemctl daemon-reload
sudo systemctl enable node-exporter
sudo systemctl start node-exporter
sudo systemctl status node-exporter
```

```
• node-exporter.service - Node exporter
  Loaded: loaded (/etc/systemd/system/node-exporter.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-01-06 12:06:55 MSK; 8ms ago
  Main PID: 24277 (node_exporter)
  Tasks: 6 (limit: 10630)
  Memory: 2.8M (peak: 2.9M)
  CPU: 4ms
  CGroup: /system.slice/node-exporter.service
          └─24277 /usr/local/bin/node_exporter
```

Проверяем, что порт :9100 открыт:

```
sudo ss -tulnp | grep 9100
```

```
tcp    LISTEN 0      4096      *:9100      *:.*      users:((("node_exporter",pid=24277,fd=3))
```

Добавляем в настройки Prometheus в секцию - `job_name: 'node-exporter'`:

```
cd ~/compose/prometheus/
vim prometheus.yml
```

```
scrape_configs:
# ...
- job_name: 'node-exporter'
  static_configs:
  - targets: ['gateway.dev.lan:9100']
    labels:
      instance: gateway
# ...
```

Перезагружаем Prometheus:

```
docker-compose down
docker-compose up -d
```

Проверяем по логам, что ошибок нет:

```
docker-compose logs -f
```

или:

```
docker-compose logs | grep -iE '(err|warn)'
```

Зайти в контейнер можно командой:

```
docker exec -it prometheus /bin/sh
```

Теперь в Grafana появится новый инстанс (правда, в отличие от кластерных серверов, он будет отображаться как `gateway.dev.lan:9100`).

3.7.5 Установка Loki

Loki агрегирует потоки логов с агентов Promtail. Устанавливаем в Docker на `gateway.dev.lan`.

```
mkdir -p ~/compose/loki
cd ~/compose/loki
```

Вынимаем файл конфигурации `local-config.yaml` из образа. Для этого делаем временный контейнер:

```
# Получаем необходимый образ:
docker pull grafana/loki:3.5.9

# Создаём контейнер без его запуска:
docker create --name temp-loki grafana/loki:3.5.9
# Посмотреть созданный контейнер можно командой `docker container ls -a`

# Копируем из контейнера конфигурацию grafana.ini в текущую директорию:
docker cp temp-loki:/etc/loki/local-config.yaml .

# Удаляем временный контейнер:
docker rm temp-loki
```

Создаём директорию с данными для Loki и устанавливаем правильные права на него:

```
mkdir -p loki-data
sudo chown 10001:10001 loki-data
```

Создаём Docker-compose файл:

```
vim docker-compose.yaml
```

```
version: '3.7'
```

```

services:
  loki:
    image: grafana/loki:3.5.9
    container_name: loki
    restart: unless-stopped
    command: -config.file=/etc/loki/local-config.yaml
    ports:
      - "3100:3100"
    volumes:
      - ./local-config.yaml:/etc/loki/local-config.yaml:ro
      - ./loki-data:/loki
    networks:
      - loki-bridge

# Создаём сеть типа bridge, чтобы порт Loki был наружу, для возможности взаимодействия
# с ним с других хостов.
networks:
  loki-bridge:
    name: loki-bridge
    driver: bridge

```

Создаём сеть loki-bridge:

```
| docker network create loki-bridge
```

Запускаем Docker-compose:

```
| docker-compose up -d
```

Проверяем по логам, что ошибок нет:

```
| docker-compose logs -f
```

или:

```
| docker-compose logs | grep -iE '(err|warn)'
```

Зайти в контейнер можно командой:

```
| docker exec -it loki /bin/sh
```

Проверяем, что порт вынесен наружу контейнера:

```
| sudo ss -tulnp | grep 3100
```

```

tcp    LISTEN 0      4096             0.0.0.0:3100     0.0.0.0:*       users:(("docker-proxy",pid=29113,fd=7))
tcp    LISTEN 0      4096             [::]:3100       [::]:*          users:(("docker-proxy",pid=29119,fd=7))

```

Проверяем запросом:

```
| curl http://localhost:3100
```

```
404 page not found
```

3.7.6 Установка Promtail для нод Kubernetes

Promtail разворачиваем через DaemonSet Kubernetes:

```
| mkdir -p ~/kubernetes/monitoring
| cd ~/kubernetes/monitoring
```

Создаём namespace для использования мониторинга:

```
| kubectl create namespace monitoring
```

```
namespace/monitoring created
```

Создаём манифест:

```
| vim promtail.yaml
```

```

---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: promtail
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: promtail
rules:
  - apiGroups: [""]
    resources: ["nodes", "nodes/proxy", "services", "endpoints", "pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: promtail
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: promtail
subjects:
  - kind: ServiceAccount
    name: promtail
    namespace: monitoring
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: promtail
  namespace: monitoring
  labels:
    app: promtail
spec:
  selector:
    matchLabels:
      app: promtail
  template:

```

```

metadata:
  labels:
    app: promtail
spec:
  serviceAccountName: promtail
  tolerations:
    - key: node-role.kubernetes.io/control-plane
      operator: Exists
      effect: NoSchedule
  containers:
    - name: promtail
      image: grafana/promtail:3.6.3
      args:
        # Конфигурацию берём из смонтируемого ConfigMap
        - -config.file=/etc/promtail/promtail.yaml
        # Создаём лейбл instance для передачи а Loki
        - -client.external-labels=instance=$(NODE_NAME)
        - -config.expand-env=true
      env:
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
      volumeMounts:
        # Монтируем конфигурацию
        - name: config
          mountPath: /etc/promtail
          readOnly: true
        # Монтируем корневую файловую систему
        - name: root
          mountPath: /host/root
  volumes:
    # Конфигурация
    - name: config
      configMap:
        name: promtail-config
    # Доступ к корневой файловой системе
    - name: root
      hostPath:
        path: /
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: promtail-config
  namespace: monitoring
data:
  promtail.yaml: |
    server:
      http_listen_port: 9080
      grpc_listen_port: 0

    positions:
      filename: /host/root/var/log/positions.yaml

    clients:
      - url: http://gateway.dev.lan:3100/loki/api/v1/push

    scrape_configs:
      - job_name: syslog
        static_configs:
          - targets:
              - localhost
            labels:
              job: syslog
              __path__: /host/root/var/log/syslog

        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_node_name]
            target_label: instance

        pipeline_stages:
          # 1. Парсим формат rsyslog (RFC3164 / RFC5424-like)
          - regex:
              expression:
                '^(?P<timestamp>\w+\s+\d+\s+\d+:\d+:\d+)\s+(?P<hostname>\S+)\s+(?P<process>\S+)(?:(?P<pid>\d+)\s+)?\s+(?P<message>.*)$'

          # 2. Преобразуем timestamp в стандартный формат
          - timestamp:
              source: timestamp
              format: "Jan _2 15:04:05" # для локального syslog без года

          # 3. Убираем шум
          - match:
              selector: '{process=~"systemd|kernel|CRON|sudo"}'
              stages:
                - regex:
                    expression: '^(?P<level>\w+):\s+(?P<message>.*)$'
                    source: message
                    # Извлекаем level из "systemd[1]: Started ...", "sudo: ...", и т.п.

          # Убираем сообщения "unable to parse \"max 0\" as a uint from Cgroup file ..."
          - drop:
              expression: "unable to parse.*max 0"
              source: message

          # 4. Фильтрация неинтересных записей
          - drop:
              # Пропускаем периодические записи, не несущие ценности
              expression: '^(Temporary failure in name resolution|pam_unix.*session opened for user.*by \(\uid=0\)|audit: .*)$'
              source: message
              drop_counter_reason: noisy_syslog

          # 5. Добавляем полезные label'ы (будут в Loki)
          - labels:
              process:
              hostname:
              # level будет только у подходящих записей (например, sudo: session opened → level="session")

```

```
# 6. Оставляем только message как payload
- output:
  source: message
```

Проверяем спецификацию на корректность:

```
| kubectl apply --dry-run=client -f promtail.yaml
```

```
serviceaccount/promtail created (dry run)
clusterrole.rbac.authorization.k8s.io/promtail created (dry run)
clusterrolebinding.rbac.authorization.k8s.io/promtail created (dry run)
daemonset.apps/promtail created (dry run)
configmap/promtail-config created (dry run)
```

Запускаем спецификацию:

```
| kubectl apply -f promtail.yaml
```

```
serviceaccount/promtail created
clusterrole.rbac.authorization.k8s.io/promtail created
clusterrolebinding.rbac.authorization.k8s.io/promtail created
daemonset.apps/promtail created
configmap/promtail-config created
```

Проверяем результат в объекте DaemonSet:

```
| kubectl get daemonsets.apps -n monitoring -o wide promtail
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE	CONTAINERS	IMAGES	SELECTOR
promtail	3	3	0	3	0	<none>	22s	promtail	grafana/promtail:3.6.3	app=promtail

Смотрим распределение Pod-ов:

```
| kubectl -n monitoring get pods -l app=promtail -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
promtail-2zhtw	1/1	Running	0	46s	10.244.1.3	combo-3	<none>		<none>	
promtail-hgcxp	1/1	Running	0	46s	10.244.0.7	combo-1	<none>		<none>	
promtail-ssttm	1/1	Running	0	46s	10.244.2.3	combo-2	<none>		<none>	

Просмотр логов:

```
| kubectl -n monitoring logs -l app=promtail --tail=10
```

Зайти на Pod для анализа неисправностей:

```
| kubectl exec -n monitoring -it promtail-625cw -- /bin/bash
```

Полезные команды внутри Pod-a:

```
# Просмотр конфигурации promtail
cat /etc/promtail/promtail.yaml

# Просмотр саантированной папки логов
ll /host/root/var/log
```

Пробросить порт :9080 для просмотра WEB-интерфейса Promtail:

```
| kubectl port-forward -n monitoring pods/promtail-px4gc --address 0.0.0.0 9999:9080
```

Проверить можно:

- <http://gateway.dev.lan:9999>

Настраиваем Prometheus на автоматический поиск хостов с node_exporter. Для этого для Prometheus создаём ServiceAccount и Secret с сохранённым туда токеном доступа. Переходим в папку с настройками для Kubernetes:

```
| cd ~/kubernetes/monitoring/
vim prometheus.yaml
```

```
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: prometheus
  namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: prometheus
rules:
- apiGroups: [""]
  resources: ["nodes", "services", "endpoints", "pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: prometheus
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: prometheus
subjects:
- kind: ServiceAccount
  name: prometheus
  namespace: monitoring
---
# Создаём токен и помещаем его в Secret по ключу prometheus-token
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  namespace: monitoring
  name: prometheus-token
  annotations:
    kubernetes.io/service-account.name: prometheus
```

Применяем спецификацию:

```
| kubectl apply -f prometheus.yaml
```

```
serviceaccount/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
secret/prometheus-token created
```

Получаем токен и сохраняем его в директорию с настройками Prometheus:

```
| kubectl -n monitoring get secret prometheus-token -o jsonpath='{.data.token}' | base64 -d > ~/compose/prometheus/token
```

Переходим в папку с настройками docker-compose для Prometheus:

```
| cd ~/compose/prometheus
```

Копируем сертификат CA кластера Kubernetes:

```
| # Копируем сертификат CA:
cp ~/.kube/ca.crt .
```

Дополняем конфигурацию:

```
| vim prometheus.yml
```

```
# ...

- job_name: 'node-exporter'

# Настройка автоматического обнаружения нод Kubernetes
kubernetes_sd_configs:
- role: node # обнаруживает все ноды кластера
  api_server: https://192.168.20.101:7443 # Адрес и порт кластера
  tls_config:
    ca_file: /etc/prometheus/ca.crt # Сертификат кластера Kubernetes
    bearer_token_file: /etc/prometheus/token # Токен доступа к кластеру

# Фильтруем только ноды, где поднят node-exporter (по наличию эндпоинта 9100)
relabel_configs:

# Берём IP ноды (не пода!)
- source_labels: [__address__]
  regex: '(.*)\:.*'
  target_label: __address__
  replacement: '${1}:9100'

# Присваиваем имя job и instance
- target_label: job
  replacement: node-exporter
- target_label: instance
  source_labels: [__meta_kubernetes_node_name]
```

Перезагружаем Prometheus:

```
| docker-compose down
docker-compose up -d
```

Создаём рабочие столы в Grafana через импорт специализированный Dashboards:

- **Menu -> Dashboards -> New -> Import**
- Указываем **ID** — 11074
- Нажимаем **Load**
- Выбрать источником данных нужную БД **Prometheus**
- Нажимаем **Import**

После этого в разделе **Dashboards** появится соответствующая панель.

3.7.7 Установка Promtail на сервер gateway

На сервере **gateway** скачиваем архив с promtail:

```
| wget -P /tmp https://github.com/grafana/loki/releases/download/v3.6.3/promtail-linux-amd64.zip
```

Распаковываем архив:

```
| sudo unzip /tmp/promtail-linux-amd64.zip promtail-linux-amd64 -d /usr/local/bin/ && \
sudo mv /usr/local/bin/promtail-linux-amd64 /usr/local/bin/promtail
```

Создаём файл конфигурации из значений по умолчанию, вшитых в сам бинарник promtail в файл `/etc/promtail/config.yaml`:

```
| sudo mkdir -p /etc/promtail
promtail -print-config-stderr 2>&1 | grep -v "level=" | sudo tee /etc/promtail/config.yaml > /dev/null
```

где:

- `rep -v "level="` — нужен для удаления строк лога, которые promtail добавит к выводу, т.к. конфигурация по умолчанию, на его взгляд, некорректна.

Изменяем в существующей конфигурации:

```
| sudo vim /etc/promtail/config.yaml

client:
  url: "http://127.0.0.1:3100/loki/api/v1/push"

server:
  http_listen_port: 9080
  grpc_listen_port: 0
```

Добавляем в конец конфигурации сбор данных из syslog:

```
| sudo vim /etc/promtail/config.yaml

scrape_configs:
  - job_name: syslog
    static_configs:
      - targets:
          - localhost
        labels:
          job: syslog
          instance: gateway
          __path__: /var/log/syslog

    pipeline_stages:
      # 1. Парсим формат rsyslog (RFC3164 / RFC5424-like)
      - regex:
          expression: '^(?P<timestamp>\w+\s+\d+\s+\d+:\d+:\d+)\s+(?P<hostname>\S+)\s+(?P<process>\S+)(?:\[(?P<pid>\d+)\])?:\s+(?P<message>.*)*$'

      # 2. Преобразуем timestamp в стандартный формат
      - timestamp:
          source: timestamp
          format: "Jan _2 15:04:05" # для локального syslog без года

      # 3. Убираем шум
      - match:
          selector: '{process=~"systemd|kernel|CRON|sudo"}'
          stages:
            - regex:
                expression: '^(?P<level>\w+):\s+(?P<message>.*)*$'
                source: message
                # Извлекаем level из "systemd[1]: Started ...", "sudo: ...", и т.п.

      # 4. Фильтрация неинтересных записей
      - drop:
          # Пропускаем периодические записи, не несущие ценности
          expression: '^((Temporary failure in name resolution|pam_unix.*session opened for user.*by \\\(uid=0\\)|audit: .*))*$'
          source: message
          drop_counter_reason: noisy_syslog

      # 5. Добавляем полезные label'ы (будут в Loki)
      - labels:
          process:
          hostname:
          # level будет только у подходящих записей (например, sudo: session opened → level="session")

      # 6. Оставляем только message как payload
      - output:
          source: message
```

Создаём сервис для запуска:

```
| sudo tee /etc/systemd/system/promtail.service << 'EOF' > /dev/null
[Unit]
Description=Promtail
After=network-online.target

[Service]
User=root
Restart=on-failure
# --config.expand-env=true - для резолва ${HOSTNAME}
ExecStart=/usr/local/bin/promtail --config.file=/etc/promtail/config.yaml --client.external-labels=instance=${NODE_NAME} --config.expand-env=true

[Install]
WantedBy=multi-user.target
EOF
```

Запускаем службу, поставив её в автозагрузку:

```
| sudo systemctl daemon-reload
sudo systemctl enable promtail
sudo systemctl restart promtail
sudo systemctl status promtail

• promtail.service - Promtail
  Loaded: loaded (/etc/systemd/system/promtail.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-01-06 18:14:12 MSK; 55min ago
  Main PID: 30136 (promtail)
    Tasks: 9 (limit: 10630)
  Memory: 18.9M (peak: 19.8M)
     CPU: 9.078s
  CGroup: /system.slice/promtail.service
          └─30136 /usr/local/bin/promtail --config.file=/etc/promtail/config.yaml
```

Проверяем, что порт :9080 открыт:

```
| sudo ss -tulnp | grep 9080

tcp    LISTEN 0      4096             *:9080           *:*
```

Проверяем работу сервиса через просмотр лога:

```
| sudo journalctl -u promtail -f
```

Зайти на WEB-интерфейс:

- <http://gateway.dev.lan:9080>

3.8 Установка Hashicorp Vault (опционально)

Vault — хранилище секретов (паролей, ключей и т.п.).

Скачиваем файл программы (с зеркала Yandex, т.к. оригинальный сервер заблокирован для РФ):

```
wget https://hashicorp-releases.yandexcloud.net/vault/1.19.3/vault_1.19.3_linux_amd64.zip
```

Устанавливаем:

```
sudo unzip -j vault_1.19.3_linux_amd64.zip vault -d /usr/local/bin/
```

Проверяем работоспособность:

```
vault --version
```

```
Vault v1.19.3 (a2de3bb7bcf4a073cbb8724863a5a88d3c2f83da), built 2025-04-29T10:34:52Z
```

Создаём рабочие каталоги программы:

```
sudo mkdir /etc/vault.d
sudo mkdir -p /opt/vault/{data,ssl}
```

Создаём конфигурационный файл:

```
sudo tee /etc/vault.d/config.hcl << 'EOF' > /dev/null
# Copyright (c) HashiCorp, Inc.
# SPDX-License-Identifier: BUSL-1.1

# Full configuration options can be found at https://developer.hashicorp.com/vault/docs/configuration

ui = true

storage "file" {
  path = "/opt/vault/data"
}

# HTTP listener
listener "tcp" {
  address = "0.0.0.0:8201"
  tls_disable = 1
}

# HTTPS listener
# listener "tcp" {
#   address = "0.0.0.0:8200"
#   tls_cert_file = "/opt/vault/ssl/tls.crt"
#   tls_key_file = "/opt/vault/ssl/tls.key"
# }
EOF
```

Добавляем в конец файла переменную окружения, которую vault будет использовать для доступа к своему серверу:

```
sudo tee -a /etc/environment << 'EOF' > /dev/null
VAULT_ADDR=http://127.0.0.1:8201
EOF
```

Устанавливаем переменную для этой сессии, чтобы не перезагружать терминал:

```
export VAULT_ADDR=http://127.0.0.1:8201
```

Создаём пользователя vault:

```
sudo useradd --system --home /etc/vault --shell /bin/false vault
```

Устанавливаем созданного пользователя владельцем каталогов:

```
sudo chown -R vault:vault /etc/vault.d /opt/vault
```

Создаём файл сервиса для запуска сервиса vault:

```
sudo tee /etc/systemd/system/vault.service << 'EOF' > /dev/null
[Unit]
# Описание сервиса
Description="HashiCorp Vault - A tool for managing secrets"
# Ссылка на документацию
Documentation=https://www.vaultproject.io/docs/
# Требуется, чтобы сеть была доступна перед запуском
Requires=network-online.target
# Запускать только после того, как сеть станет доступной
After=network-online.target
# Условие: файл конфигурации должен существовать и не быть пустым
ConditionFileNotEmpty=/etc/vault.d/config.hcl

[Service]
# Запускать сервис от имени пользователя vault
User=vault
# Запускать сервис от имени группы vault
Group=vault
# Защита системных файлов и директорий (полный доступ только для чтения)
ProtectSystem=full
# Защита домашних директорий (только чтение)
ProtectHome=read-only
# Использовать изолированное временное пространство для /tmp
PrivateTmp=yes
# Ограничить доступ к устройствам (/dev)
PrivateDevices=yes
# Сохранить capabilities после смены пользователя
SecureBits=keep-caps
# Разрешить процессу использовать CAP_IPC_LOCK (необходимо для Vault)
AmbientCapabilities=CAP_IPC_LOCK
# Запретить процессу получать новые привилегии
NoNewPrivileges=yes
# Команда для запуска Vault с указанием конфигурационного файла
ExecStart=/usr/local/bin/vault server -config=/etc/vault.d/config.hcl
# Команда для перезагрузки конфигурации (отправка сигнала HUP)
ExecReload=/bin/kill --signal HUP $MAINPID
# Режим завершения процесса (убивать только основной процесс)
KillMode=process
# Сигнал для завершения процесса (SIGINT - мягкое завершение)
KillSignal=SIGINT
EOF
```

```
# Перезапускать сервис при ошибках
Restart=on-failure
# Задержка перед перезапуском (5 секунд)
RestartSec=5
# Таймаут для остановки сервиса (30 секунд)
TimeoutStopSec=30
# Максимальное количество быстрых перезапусков перед отказом
StartLimitBurst=3
# Лимит на количество открытых файловых дескрипторов
LimitNOFILE=65536

[Install]
# Указывает, что сервис должен быть запущен в multi-user режиме
WantedBy=multi-user.target
EOF
```

Перезагружает конфигурацию systemd:

```
sudo systemctl daemon-reload
sudo systemctl enable vault
sudo systemctl start vault
sudo systemctl status vault
```

```
• vault.service - "HashiCorp Vault - A tool for managing secrets"
  Loaded: loaded (/etc/systemd/system/vault.service; enabled; preset: enabled)
  Active: active (running) since Sun 2025-05-11 16:34:50 MSK; 15s ago
  Invocation: 6f221f6237f043688e0bb312e824c32f
  Docs: https://www.vaultproject.io/docs/
  Main PID: 141973 (vault)
  Tasks: 7 (limit: 6390)
  Memory: 312.4M (peak: 312.7M)
  CPU: 163ms
  CGroup: /system.slice/vault.service
          └─141973 /usr/local/bin/vault server -config=/etc/vault.d/config.hcl
```

Проверяем статус vault:

```
| vault status

Key
---
Seal Type      shamir
Initialized    false
Sealed         true
Total Shares   0
Threshold      0
Unseal Progress 0/0
Unseal Nonce   n/a
Version        1.19.3
Build Date     2025-04-29T10:34:52Z
Storage Type   file
HA Enabled     false
```

Выполняем инициализацию. В результате будут выведены ключи, которые необходимо сохранить (т.к. их пересоздать нельзя). Так же будет создана база ключей /opt/vault/data/:

```
| vault operator init

Unseal Key 1: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Unseal Key 2: BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
Unseal Key 3: CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
Unseal Key 4: DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD
Unseal Key 5: EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE

Initial Root Token: FFF.FFFFFFFFFFFFFFFFFFFFFFFF

Vault initialized with 5 key shares and a key threshold of 3. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 3 of these keys to unseal it
before it can start servicing requests.

Vault does not store the generated root key. Without at least 3 keys to
reconstruct the root key, Vault will remain permanently sealed!

It is possible to generate new unseal keys, provided you have a quorum of
existing unseal keys shares. See "vault operator rekey" for more information.
```

Эти ключи и root-токен лучше сохранить в описании виртуальной машины Нурег-V. Для этого заходим в **Настройки → Имя** и сохраняем все ключи **Unseal Key 1...5** и **Initial Root Token**.

Если ключ потерян, базу восстановить невозможно. Для создания новых ключей, нужно удалить текущую базу данных и сделать переинициализацию базы:

```
sudo systemctl stop vault
sudo rm -R /opt/vault/data/*
sudo systemctl start vault
vault operator init
```

Проверяем статус после инициализации (**Initialized = true** — инициализация произведена, **Sealed = true** — база запечатана, т.е. недоступна):

```
| vault status

Key
---
Seal Type      shamir
Initialized    true
Sealed         true
Total Shares   5
Threshold      3
Unseal Progress 0/3
Unseal Nonce   n/a
Version        1.19.3
Build Date     2025-04-29T10:34:52Z
Storage Type   file
HA Enabled     false
```

Создаём автоматическое распечатывание базы данных при загрузке системы (**ключи вставляем свои, а не из примера**):

```
sudo tee /etc/vault-unseal.sh << 'EOF' > /dev/null
#!/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin

# Vault может не успеть загрузиться, поэтому делаем паузу
sleep 3
vault operator unseal AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA > /dev/null
vault operator unsealBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB > /dev/null
vault operator unsealCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC > /dev/null
EOF
```

Устанавливаем права только на чтение и выполнение пользователем root:

```
sudo chmod 500 /etc/vault-unseal.sh
```

Проверяем, что база открывается через этот скрипт (**Sealed** должен перейти в статус **false**):

```

sudo /etc/vault-unseal.sh
vault status

```

Key	Value
---	----
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	1.19.3
Build Date	2025-04-29T10:34:52Z
Storage Type	file
Cluster Name	vault-cluster-18750bb4
Cluster ID	750e8f0e-2bce-25ee-ad20-128295b56b40
HA Enabled	false

Создаём сервис, который запустит скрипт распечатывания:

```
sudo tee /etc/systemd/system/vault-unseal.service < 'EOF' > /dev/null
[Unit]
# Описание юнита (единицы systemd)
Description=Vault Auto Unseal Service
# Краткое описание сервиса: "Сервис автоматического распечатывания Vault"
After=network.target
# Запускать этот сервис после того, как будет доступна сеть (network.target)
After=vault.service
# Запускать этот сервис после запуска сервиса Vault

[Service]
# Настройки самого сервиса
Environment="VAULT_ADDR=http://127.0.0.1:8201"
# Установка переменной окружения VAULT_ADDR с адресом Vault API
ExecStart=/etc/vault-unseal.sh
# Команда для запуска сервиса - выполнение скрипта /etc/vault-unseal.sh
Type=oneshot
# Тип сервиса - однократное выполнение (запускается и завершается)
RemainAfterExit=no
# Не считать сервис активным после завершения выполнения команды

[Install]
# Настройки установки и автозапуска сервиса
WantedBy=multi-user.target
# Сервис будет автоматически запускаться при достижении уровня запуска multi-user.target (обычный многопользовательский режим)
EOF
```

Ставим сервис в автозагрузку:

```
sudo systemctl daemon-reload
sudo systemctl enable vault-unseal
```

Перезагружаем компьютер для проверки автораспечатывания базы Vault:

```
sudo reboot
```

После перезагрузки проверяем распечатывание базы данных (**ждём несколько секунд**, распечатывание будет не сразу):

vault status	
Key	Value
Seal Type	shamir
Initialized	true
Sealed	false <- ЕСЛИ СКРИПТ СРАБОТАЛ, БУДЕТ FALSE
Total Shares	5
Threshold	3
Version	1.19.3
Build Date	2025-04-29T10:34:52Z
Storage Type	file
Cluster Name	vault-cluster-eb28aea5
Cluster ID	8541c17f-eeec2-a16e-1780-db2ec07fcec8
HA Enabled	false

3.9 Установка Helm

Helm устанавливаем на компьютере **gateway**.

Каталог чартов:

- <https://artifacthub.io/packages/search?kind=0>

Скачиваем последний релиз со страницы релизов <https://github.com/helm/helm/releases>:

```
cd ~
wget -P /tmp https://get.helm.sh/helm-v3.17.3-linux-amd64.tar.gz
sudo tar -xzf /tmp/helm-v3.17.3-linux-amd64.tar.gz --strip-components=1 -C /usr/local/bin linux-amd64/helm
```

Проверка результату установки:

```
helm version
```

```
version.BuildInfo{Version:"v3.17.3", GitCommit:"e4da49785aa6e6ee2b86efd5dd9e43400318262b", GitTreeState:"clean", GoVersion:"go1.23.7"}
```

Подключаем репозитории чартов.

Проверяем, что сейчас репозитория не подключено ни одного:

```
helm repo list
```

```
Error: no repositories to show
```

Добавление репозитория Helm (Bitnami):

Оригинальный репозиторий `helm repo add bitnami https://charts.bitnami.com/bitnami` заблокирован, нужно использовать зеркало.

```
helm repo add bitnami https://mirror.yandex.ru/helm/charts.bitnami.com/
```

```
"bitnami" has been added to your repositories
```

Просматриваем список подключённых репозиторий:

```
helm repo list
```

NAME	URL
bitnami	https://mirror.yandex.ru/helm/charts.bitnami.com/

Обновляем список файлов из подключённых репозиторий:

```
helm repo update
```

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. 🎉Happy Helming!🎉
```

Поиск пакетов по репозиториям:

```
helm search repo nginx
```

NAME	CHART VERSION	APP VERSION	DESCRIPTION
bitnami/nginx	21.1.23	1.29.1	NGINX Open Source is a web server that can be a...
bitnami/nginx-ingress-controller	12.0.7	1.13.1	NGINX Ingress Controller is an Ingress controll...
bitnami/nginx-intel	2.1.15	0.4.9	DEPRECATED NGINX Open Source for Intel is a lig...

3.9.1 Установка утилиты kube-ops-view

Все действия производим на хосте Gateway.

Устанавливаем программу kube-ops-view, которая будет показывать в WEB-интерфейсе текущее состояние узлов кластеров (минималистский мониторинг).

Ищем установочный Helm-чарт в подключённом репозитории:

```
helm search hub kube-ops-view --output yaml
```

```
- app_version: 20.4.0
  description: A read-only system dashboard for multiple K8s clusters
  repository:
    name: geek-cookbook
    url: https://geek-cookbook.github.io/charts/
  url: https://artifacthub.io/packages/helm/geek-cookbook/kube-ops-view
  version: 1.2.2
- app_version: 23.5.0
  description: A Helm chart for bootstrapping kube-ops-view.
  repository:
    name: christianhuth
    url: https://charts.christianhuth.de
  url: https://artifacthub.io/packages/helm/christianhuth/kube-ops-view
  version: 5.0.5
- app_version: "20.4"
  description: A Helm chart kubeops, a read only kubernetes dashboard
  repository:
    name: fluent-operator
    url: https://tanmay-bhat.github.io/helm-charts/
  url: https://artifacthub.io/packages/helm/fluent-operator/kube-ops-view
  version: 0.1.2
```

Оригинальным здесь будет чарт от создателя christianhuth. Автор на странице <https://artifacthub.io/packages/helm/christianhuth/kube-ops-view> предлагает подключить его личный репозиторий чартов и установить его оттуда:

```
helm repo add christianhuth https://charts.christianhuth.de
```

```
"christianhuth" has been added to your repositories
```

Обновляем список пакетов:

```
helm repo update
```

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "christianhuth" chart repository
...Successfully got an update from the "bitnami" chart repository
Update Complete. 🎉Happy Helming!🎉
```

Устанавливаем чарт под своим именем - my-kube-ops-view. Но для начала сохраняем файл с переменными в локальную папку для этого проекта:

```
mkdir -p ~/helm/kube-ops-view
cd ~/helm/kube-ops-view
helm show values christianhuth/kube-ops-view > values.yaml
```

Изменяем переменные в values.yaml, а конкретно:

```
vim values.yaml
```

```
image:
  registry: docker.io
```

```
repository: hjacobs/kube-ops-view
pullPolicy: IfNotPresent # <-- Меняем политику получения образов Always на IfNotPresent
tag: "23.5.0"
```

Устанавливаем чарт под именем **my-kube-ops-view** в пространстве имён kube-system, чтоб он мог видеть системные данные kubernetes:

```
helm install my-kube-ops-view christianhuth/kube-ops-view -n kube-system -f values.yaml
```

```
NAME: my-kube-ops-view
LAST DEPLOYED: Thu Aug 21 17:14:24 2025
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

Проверяем статистику по загрузке пода. Т.е. поду сначала придётся скачать Docker образ и его развернуть, то события изменения состояния запускаемого пода легче смотреть в режиме `tail -f`:

```
kubectl get pods -n kube-system -w
```

Если под не грузится, проверяем описание событий и доступность пути к репозиторию и образу:
`kubectl describe pod my-kube-ops-view-XXXXXXXXXX-YYYYY`

После того, когда под будет запущен, смотрим, какие сущности были запущены:

```
kubectl get all -n kube-system -l app.kubernetes.io/name=kube-ops-view
```

NAME	pod/my-kube-ops-view-6868df6684-bn5wc	READY	1/1	STATUS	Running	RESTARTS	0	AGE	3m44s
NAME	service/my-kube-ops-view	TYPE	ClusterIP	CLUSTER-IP	10.111.156.234	EXTERNAL-IP	<none>	PORT(S)	80/TCP
NAME	deployment.apps/my-kube-ops-view	READY	1/1	UP-TO-DATE	1	AVAILABLE	1	AGE	3m44s
NAME	replicaset.apps/my-kube-ops-view-6868df6684	DESIRED	1	CURRENT	1	READY	1	AGE	3m44s

Скачать чарт в виде локального архива:

```
helm pull christianhuth/kube-ops-view
```

Скачать чарт и сразу его разархивировать в набор файлов:

```
helm pull christianhuth/kube-ops-view --untar --untardir ./my-kube-ops-view
```

Удалить чарт можно командой:

```
helm uninstall ops-view
```

На gateway делаем Port-Forwarding к сервису, созданному программой порт наружу, чтобы можно было подключиться к программе через внешний браузер. Проброс порта **9999** на **80** порт запускаем в фоне:

```
kubectl port-forward service/my-kube-ops-view -n kube-system --address 0.0.0.0 9999:80 &
```

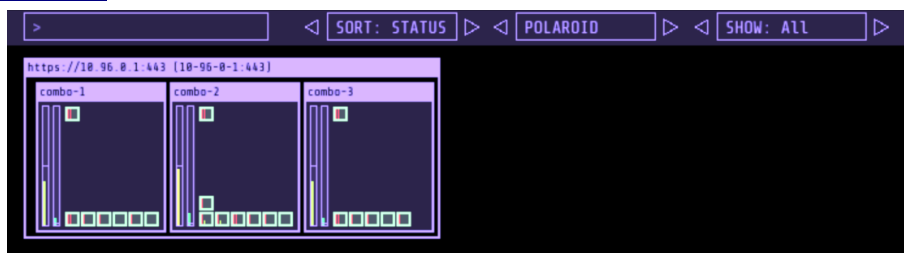
Проверяем, что программа доступна с самого gateway:

```
curl http://gateway.dev.lan:9999
```

Должен появиться HTML контент.

Заходим на страницу из браузера:

- <http://gateway.dev.lan:9999>



После проверки, можно остановить Port-Forwarding:

```
kill %1
```

3.10 Установка GitLab

GitLab Community Edition docker image — <https://hub.docker.com/r/gitlab/gitlab-ce>

Установку осуществляем с поддержкой внутреннего Registry GitLab.

3.10.1 Установка

Скачиваем образа:

```
docker pull gitlab/gitlab-ce:17.11.7-ce.0
docker pull gitlab/gitlab-runner:v17.11.4
docker pull registry.gitlab.com/gitlab-org/gitlab-runner/gitlab-runner-helper:x86_64-v17.11.1
docker pull docker:28.1.1
docker pull docker:28.1.1-dind
docker pull alpine:3.21.3
```

Подготавливаем рабочую директорию:

```
mkdir -p ~/compose/gitlab
cd ~/compose/gitlab
```

Создаём Docker-compose файл:

```
tee docker-compose.yml << 'EOF' > /dev/null
# Версия схемы docker-compose (3.7 поддерживает расширенные возможности)
version: "3.7"

# Секция определения сервисов
services:
  # Сервис GitLab CE (Community Edition)
  gitlab:
    # Официальный образ GitLab с конкретной версией
    image: gitlab/gitlab-ce:17.11.7-ce.0
    # Явное имя контейнера для удобства управления
    container_name: gitlab
    # Имя хоста внутри контейнера
    hostname: gitlab
    # Политика перезапуска: всегда (при падении или перезагрузке докера)
    restart: always

    # Переменные окружения для настройки GitLab
    environment:
      UMASK: 002
      # Конфигурация Omnibus GitLab
      GITLAB_OMNIBUS_CONFIG: |
        external_url 'https://gitlab.dev.lan'
        gitlab_rails['gitlab_shell_ssh_port'] = 2222
        gitlab_rails['password_authentication_enabled_for_web'] = true
        nginx['listen_port'] = 80
        nginx['listen_https'] = false
        puma['worker_processes'] = 0 # Отключает режим кластера для освобождения памяти
        gitlab_rails['registry_enabled'] = true # Включение Registry
        registry['enable'] = true
        registry_external_url 'https://registry.dev.lan' # Указываем доступ через отдельный домен
        registry_nginx['enable'] = false # Отключаем конфликтующий с Traefik nginx https://gitlab.com/gitlab-org/omnibus-gitlab/-/issues/5560
        registry['registry_http_addr'] = "0.0.0.0:5000"
      TZ: "Europe/Moscow"

    # Указываем внешний DNS для доступа в Интернет из контейнера.
    dns: 192.168.20.1

    # Тома для сохранения данных между перезапусками
    volumes:
      - ./gitlab/config:/etc/gitlab # Конфигурационные файлы
      - ./gitlab/logs:/var/log/gitlab # Логи
      - ./gitlab/data:/var/opt/gitlab # Основные данные (репозитории, БД и т.д.)
      - ./gitlab/ssh:/etc/ssh # Для SSH:
      - ./gitlab-registry:/var/opt/gitlab/gitlab-rails/shared/registry # Для Registry

    # Метки для Traefik (добавлено для интеграции)
    labels:
      - "traefik.enable=true" # Явно включаем обработку этого контейнера
      - "traefik.http.routers.gitlab-http.rule=Host(`gitlab.dev.lan`)" # Правило для HTTP
      - "traefik.http.routers.gitlab-http.entrypoints=websecure" # Используем HTTPS entrypoint
      - "traefik.http.routers.gitlab-http.tls.certresolver=stepca" # Центр сертификации stepca
      - "traefik.http.routers.gitlab-http.service=gitlab-http"
      - "traefik.http.services.gitlab-http.loadbalancer.server.port=80" # Порт контейнера GitLab

    # Для встроенного Docker Registry
      - "traefik.http.routers.registry.rule=Host(`registry.dev.lan`)" #registry
      - "traefik.http.routers.registry.entrypoints=websecure" # Используем HTTP entrypoint
      - "traefik.http.routers.registry.tls.certresolver=stepca" # Центр сертификации stepca
      - "traefik.http.routers.registry.service=registry"
      - "traefik.http.services.registry.loadbalancer.server.port=5000" # Порт Registry в контейнере
    # Docker Registry требует, чтобы прокси (Traefik) передавал специфический заголовок Docker-Distribution-API-Version,
    # иначе клиент Docker будет получать ошибки или Traefik не сможет корректно проксировать бинарный трафик Registry.
      - "traefik.http.middlewares.registry-headers.headers.customResponseHeaders.Docker-Distribution-API-Version=registry/2.0"
      - "traefik.http.routers.registry.middlewares=registry-headers"

    cap_add:
      - NET_ADMIN

    networks:
      - traefik-public

gitlab-runner:
  image: gitlab/gitlab-runner:v17.11.4

  # Явное имя контейнера для удобства управления
  container_name: gitlab-runner
  # Имя хоста внутри контейнера
  hostname: gitlab-runner
  # Политика перезапуска: всегда (при падении или перезагрузке докера)
  restart: always
  environment:
    TZ: "Europe/Moscow"

  volumes:
    - /var/run/docker.sock:/var/run/docker.sock # Для работы с Docker внутри раннера
    - ./gitlab-runner-config:/etc/gitlab-runner # Постоянное хранилище для config.toml
    # Прокидываем хранилище цифровых ключей с хостовой машины в контейнер,
    # чтобы он начал доверять сертификату от Step CA, корневой ключ которого установлен в хостовую систему
    - "/etc/ssl/certs:/etc/ssl/certs/:ro"

  # Подключаем к сети gitlab-runner-public, чтобы контейнер был внешним к комтейнеру gitlab
  # и мог так же резолвить DNS.
  networks:
    - gitlab-runner-public

  # Указываем, какой DNS сервер должен использовать Docker внутри контейнера
  dns: 192.168.20.1

# Сети
networks:
  # Сеть должна быть создана командой:
  # docker network create traefik-public
  traefik-public:
    external: true

  # Сеть должна быть создана командой:
  # docker network create gitlab-runner-public
  gitlab-runner-public:
```

EOF

Создаём сеть для Runner:

```
docker network create gitlab-runner-public
```

5012b18e5729fd6b93a6beac86ea91b93e47a9b84b4e078dbdf00932810f2196

Запускаем контейнер:

```
docker-compose up -d --build
```

Использование ресурсов контейнером можно наблюдать в (когда пропадёт жёлтый кружок слева от контейнера):

```
ctop
```

Запуск GitLab долгий, поэтому этот процесс можно отслеживать через контроль логов:

```
docker logs -f gitlab
```

• • •

```
gitlab | ==> /var/log/gitlab/gitlab-exporter/current <==
gitlab | 2025-05-10 13:46:23.06423 :1 - - [10/May/2025:13:46:23 UTC] "GET /ruby HTTP/1.1" 200 1089
gitlab | 2025-05-10 13:46:23.06425 - -> /ruby
gitlab | 2025-05-10 13:46:27.06755 :1 - - [10/May/2025:13:46:27 UTC] "GET /sidekiq HTTP/1.1" 200 507
gitlab | 2025-05-10 13:46:27.06757 - -> /sidekiq
gitlab | 2025-05-10 13:46:30.39609 :1 - - [10/May/2025:13:46:30 UTC] "GET /database HTTP/1.1" 200 1778
gitlab | 2025-05-10 13:46:30.39611 - -> /database
```

Проверяем, что успешно создан TLS сертификат:

```
openssl s_client -connect localhost:443 -servername gitlab.dev.lan | openssl x509 -noout -issuer -subject
```

```
Connecting to 127.0.0.1
depth=2 0=Local CA, CN=Local CA Root CA
verify return:1
depth=1 0=Local CA, CN=Local CA Intermediate CA
verify return:1
depth=0 CN=gitleab.dev.lan
verify return:1
issuer=0=Local CA, CN=Local CA Intermediate CA
subject=CN=gitleab.dev.lan
```

Получаем пароль по умолчанию, запустив команду:

```
docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
```

Password: kk=

Обязательно ждем, пока GitLab загрузится полностью. После этого можно заходить на WEB-интерфейс с хостовой **Windows** машины и ввести данные:

- <https://gitlab.dev.lan>
- login — root
- password — kk=

На запрос:

Check your sign-up restrictions

Your GitLab instance allows anyone to register for an account, which is a security risk on public-facing GitLab instances. You should deactivate new sign ups if public users aren't expected to register for an account.

Проверьте ограничения по регистрации

Ваш экземпляр GitLab позволяет любому человеку зарегистрировать учетную запись, что представляет угрозу безопасности для общедоступных экземпляров GitLab. Вам следует деактивировать новые регистрации, если не предполагается, что обычные пользователи будут регистрировать учетные записи.

... отвечаем **Deactivate** и когда нас перебросит на страницу настроек снимаем галки:

- [] — Sign-up enabled

... и нажимаем **Save Changes**.

Меняем пароль для пользователя **root**, под которым сейчас находимся. Для этого нажимаем на значке профиля (вверху окна) и выбираем пункт **Edit Profile** → **Password** и задаём:

- Current password — kk=
- New password — Pa\$\$w0rd
- Password confirmation — Pa\$\$w0rd

После перекидывание на страницу авторизации, заходим под пользователем **root** с паролем **Pa\$\$w0rd**.

3.10.2 Добавление пользователя

Добавляем рабочего пользователя с именем **administrator**.

Т.к. через WEB-интерфейс добавить пользователя затруднительно (т.к. пароль будет выслан пользователю на e-mail, что при визуализации затруднительно), добавляем пользователя **через консоль**. Заходим в запущенный контейнер GitLab командой

```
docker exec -it gitlab /bin/bash
```

Запускаем саму консоль (консоль запускается довольно **долго**, нужно дождаться следующего текста):

```
gitlab-rails console
```

```
Ruby:      ruby 3.2.5 (2024-07-26 revision 31d0f1a2e7) [x86_64-linux]
GitLab:    17.11.2 (059055d0bcc) FOSS
GitLab Shell: 14.41.0
PostgreSQL: 16.8
-----[ booted in 23.49s ]
Loading production environment (Rails 7.0.8.7)
```

```
irb(main):001:0>
```

Данные добавляются только после запуска команды `user.save!`

Проверяем, что пользователя ещё нет:

```
User.find_by(username: 'administrator')
```

```
=> nil
```

В консоли вводим данные нового пользователя:

```
user = User.new(
  name: 'Administrator',
  username: 'administrator', # username и path должны быть в нижнем регистре
  password: 'Pa$$w0rd', # Пароль пользователя
  password_confirmation: 'Pa$$w0rd', # Повтор пароля
  email: 'administrator@mail.dev.lan', # GitLab требует email, даже фейковый
  skip_confirmation: true
)
```

```
#<User id: @administrator>
```

Создаём "организацию":

```
org = Organizations::Organization.find_or_create_by!(name: "Default Organization", path: 'default-organization')
```

```
#<Organizations::Organization:0x000070f66540a548
```

Создаём Namespace:

```
user.build_namespace(
  path: user.username.downcase, # обязательно lowercase, если есть ограничение
  name: user.name,
  visibility_level: 20, # 20 = private, 10 = public (если нужно)
  description: '',
  organization: org, # Ссылка на организацию
)
```

```
#<Namespaces::UserNamespace id: @administrator>
```

Непосредственно создаём пользователя, запустив этот код:

```
if user.save!
  puts "✓ Пользователь создан: #{user.username}"
else
  puts "✗ Ошибка: #{user.errors.full_messages}"
end
```

```
✓ Пользователь создан: administrator
```

```
=> nil
```

Для выхода из консоли запускаем:

```
exit
exit
```

Теперь можно выйти из пользователя **root** и зайти под пользователем **administrator** с паролем **Pa\$\$w0rd**.

Для удаления пользователя, определяем его существование:

```
User.find_by(username: 'administrator')
#<User id:6 @administrator>
```

Получаем обреч пользователя:

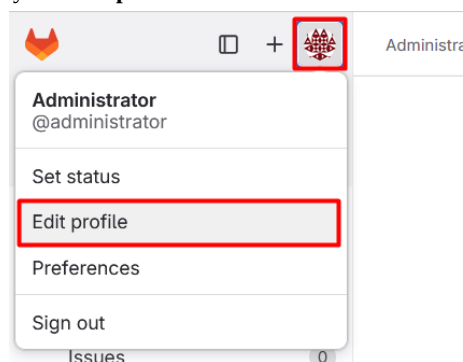
```
user = User.find_by(username: 'administrator')
#<User id:6 @administrator>
```

Удаляем все зависимости и самого пользователя:

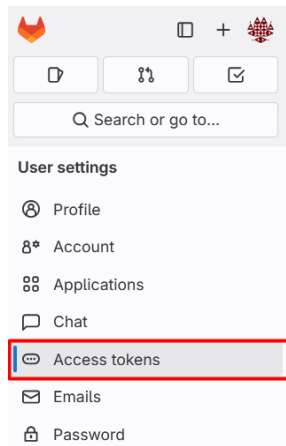
```
user.projects.each { |p| p.destroy! }
user.destroy!
#<User id:6 @administrator>
```

3.10.3 Создание токена доступа для Git

Для доступа Git к GitHub, создаём токен для пользователя **administrator**. Для этого авторизируемся под пользователем **administrator** в GitLab и заходим в редактирование профиля и пункт **Edit profile**:

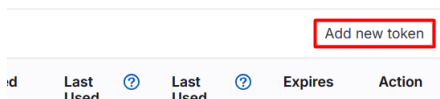


Заходим в раздел **Access tokens**:



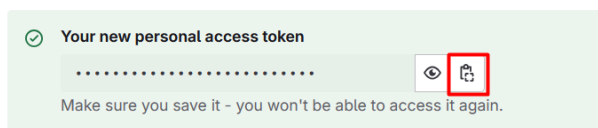
Нажимаем кнопку добавления нового токена — **Add new token:**

Use personal access tokens to authenticate against Git over



Указываем имя токена **full_access** в разделе Token name, удаляем **ограничение по времени** в разделе Expiration date, **выделяем все** пункты и нажимаем на кнопку добавления токена Create personal access token:

В результате, будет создан токен, который нужно скопировать и сохранить (он будет вида **glpat -***).



Эти токен лучше сохранить в описании виртуальной машины **Hyper-V**. Для этого заходим в **Настройки** → **Имя** и сохраняем токен в виде строки авторизации: **Git: https://administrator:glpat-xxxxxxxxx-xxxxxxxxx@gitlab.dev.lan**

Теперь, когда нужно, можно с помощью этого токена клонировать репозитории с одновременной авторизацией для его изменения командой:

```
git clone https://administrator:glpat-xxxxxxxxx-xxxxxxxxx@gitlab.dev.lan/administrator/hello-world.git
... или глобально сохранить токен для доступа ко всем репозиториям пользователя:
git config --global credential.helper store
echo "https://administrator:glpat-xxxxxxxxx-xxxxxxxxx@gitlab.dev.lan" >> ~/.git-credentials
```

Проверяем доступ к Gitlab Registry от пользователя root (пользователь administrator сможет получить доступ, только когда будет создан проект с его участием).

В консоли получаем JWT токен доступа от <https://gitlab.dev.lan> для пользователя **root** и посылает запрос <https://registry.dev.lan> на показ списка репозиторий (он должен быть пуст, но ответа будет достаточно чтобы понять, что доступ к реестру есть):

```
TOKEN=$(curl -s -u 'root:Pa$$w0rd' "https://gitlab.dev.lan/jwt/auth?service=container_registry&scope=registry:catalog:*" | jq -r
'.token')
echo $TOKEN

curl -H "Authorization: Bearer $TOKEN" https://registry.dev.lan/v2/_catalog

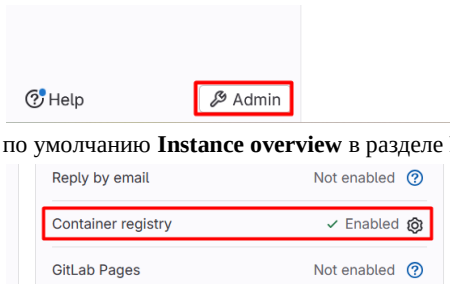
{"repositories":["administrator/hello-world"]}
```

Если результат не верен, можно посмотреть содержимое токена командой и проверить, что права на просмотр указаны верно:

```
echo $TOKEN | cut -d'.' -f2 | base64 -d | jq .
```

```
base64: invalid input
{
  "auth_type": "gitlab_or_ldap",
  "access": [
    {
      "type": "registry",
      "name": "catalog",
      "actions": [
        "*"
      ]
    }
  ],
  "user": "XXXX ... XXXX",
  "jti": "4c6bd758e-8478-40d1-8585-980a344e70fa",
  "aud": "container_registry",
  "sub": "root",
  "iss": "omnibus-gitlab-issuer",
  "iat": 1755695891,
  "nbf": 1755695886,
  "exp": 1755696191
}
```

Так же проверяем работу реестра через WEB-Gui. Заходим в него под пользователем **root**, далее переходим в раздел **Admin** (внизу слева):

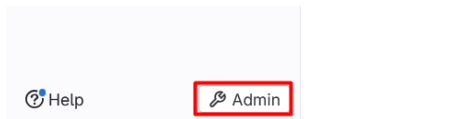


... и в появившемся в правой части разделе по умолчанию **Instance overview** в разделе **Features** должны увидеть:

3.10.4 Добавление Runner

Видео по созданию Runner — https://www.youtube.com/watch?v=Rvh7OZbDJ_o

Добавляем Runner для сборки проекта. Для этого заходим в GitLab под пользователем **root**. Далее нажимаем на кнопку **Admin** слева внизу:



... и идем:

- **CI/CD → Runners → New instance runner**
 - Tags — **shared**
- Нажимаем кнопку **Create runner**
 - Operating systems — **Linux**

Видим строку, которую нужно выполнить в контейнере `gitlab-runner`, чтобы зарегистрировать его как исполнитель заданий для контейнера `gitlab`, т.к. Runner работает в отдельном контейнере и держит сам связь с GitLab. Запоминаем **только токен**, его подставим в свою строку, чтобы небыло интерактивных запросов:

```
gitlab-runner register
--url http://gitlab.dev.lan
--token glrt-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.XXXXXXXXXX
```

Переходим в консоль и входим в контейнер **gitlab-runner** :

```
docker exec -it gitlab-runner /bin/bash
```

Выполняем в контейнере сохранённую команду (заменяв только токен на свой):

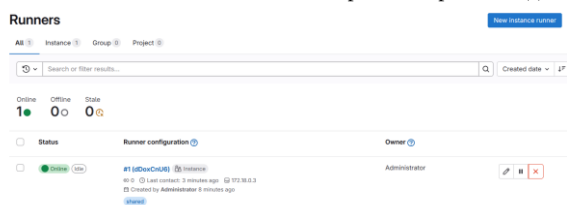
```
gitlab-runner register \
--non-interactive \
--url https://gitlab.dev.lan \
--executor docker \
--docker-image alpine:3.21.3 \
--description "docker-runner" \
--token glrt-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!
Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
```

После этого будет создан файл конфигурации Runner-a `gitlab-runner-config/config.toml`. Выходим из контейнера `gitlab-runner`:

```
exit
```

Далее в WEB-интерфейсе GitLab заходим снова в **CI/CD** → **Runners** . Теперь в настройка видим один рабочий раннер:



Нужно добавить или изменить только выделенные **жирным** параметры в файле. Если файл менять полностью, не забыть заменить **token** на тот, что использовался при добавлении.

Скачиваемые gitlab-runnetr образа сохраняются в Docker на хосте **gateway**.

Устанавливаем 3 Runner на всех рабочих хостах `combo-1.dev.lan`, `combo-2.dev.lan` и `combo-3.dev.lan`.

```
sudo vim /etc/gitlab-runner/config.toml

volumes = ["/var/run/docker.sock:/var/run/docker.sock", "/cache", "/etc/ssl/certs/ca-certificates.crt:/etc/ssl/cert.pem:ro"]

sudo systemctl restart gitlab-runner.service
```



- **CI/CD → Runners → New instance runner**
 - Tags — **shared**
- Нажимаем кнопку **Create runner**
 - Operating systems — **Linux**

Видим строку, которую нужно выполнить в контейнере `gitlab-runner`, чтобы зарегистрировать его как исполнитель заданий для контейнера `gitlab`, т.к. Runner работает в отдельном контейнере и держит сам связь с GitLab. Запоминаем **только токен**, его подставим в свою строку, чтобы не было интерактивных запросов:

```
gitlab-runner register
--url http://gitlab.dev.lan
--token glrt-xxxxxxxxxxxxxxxxxxxx
```

Заходим на сервер **combo-1.dev.lan** и регистрируем сервер, заменив токен на полученный из GitLab:

```
sudo gitlab-runner register \
--url "https://gitlab.dev.lan" \
--non-interactive \
--token "glrt-xxxxxxxxxxxxxxxxxxxx" \
--name ${HOSTNAME} \
--executor "docker" \
--docker-image "alpine:3.21.3" \
--docker-volumes "/var/run/docker.sock:/var/run/docker.sock"
```

```
Runtime platform arch=amd64 4 os=linux pid=642121 revision=cc77edaf version=17.11.4
Running in system-mode.

Verifying runner... is valid runner=6XxP3TzTv
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
```

Повторяем процесс добавления нового инстанса через **New instance runner** для остальных серверов **combo-2.dev.lan** и **combo-3.dev.lan**. В результате, в разделе **Admin -> CI/CD -> Runners** должны быть видны все 3 Runner-a:

Online	Offline	State			
3	0	0			
			Status	Runner configuration	Owner
☐	Online	Idle	#3 (HzTp3H6)	Instance eo 0 Last contact: 4 minutes ago 172.18.0.4 Created by Administrator 1 hour ago shared	Administrator
☐	Online	Idle	#2 (V73KB5EC)	Instance eo 0 Last contact: 5 minutes ago 172.18.0.4 Created by Administrator 1 hour ago shared	Administrator
☐	Online	Idle	#1 (6XxP3TzT)	Instance eo 0 Last contact: 57 minutes ago 172.18.0.4 Created by Administrator 2 hours ago shared	Administrator

3.10.6 Наполняем локальный Docker контейнерами

Gitlab runner будет использовать контейнеры только из локально установленного Docker через сокет `/var/run/docker.sock`. Сам скачивать контейнеры он не сможет. Поэтому накатать локальный Docker контейнерами необходимо в ручную:

```
docker pull alpine:3.21.3
docker pull gitlab/gitlab-runner:v17.11.4
docker pull golang:1.23.10-alpine3.22
docker pull registry.gitlab.com/gitlab-org/gitlab-runner/gitlab-runner-helper:x86_64-v17.11.1
docker pull docker:28.1.1
docker pull docker:28.1.1-dind
```

3.10.7 Создаём тестовый проект в GitLab

Для тестирования работы системы CI создаём тестовый проект. Авторизуемся в WEB-интерфейсе под пользователем **administrator**. Заходим:

- Projects → Create a project → Create blank project:
 - Project name — **hello-world**
 - Visibility Level — **Public**
 - Project Configuration:
 - [] — Initialize repository with a README

Нажимаем кнопку **Create project**.

```
Появится плашка:
You can't push or pull repositories using SSH until you add an SSH key to your profile.
Вы не сможете продвигать или извлекать репозитории с помощью SSH, пока не добавите SSH-ключ в свой профиль.
Нажать Don't show again
```

3.10.8 Устанавливаем Golang

С сайта <https://go.dev/dl/> скачиваем нужную версию Go, распаковываем её в директорию `/opt` и делаем все остальные сопутствующие операции:

```
wget -P /tmp https://go.dev/dl/go1.25.5.linux-amd64.tar.gz

# Распаковываем в /opt:
sudo tar -C /opt -xzf /tmp/go1.25.5.linux-amd64.tar.gz

# Создаём ссылки на запускные файлы:
sudo ln -s /opt/go/bin/go /usr/local/bin/go
sudo ln -s /opt/go/bin/gofmt /usr/local/bin/gofmt

mkdir -p $HOME/go/{bin, pkg, src}

# Добавляем пути в переменные окружения для всех пользователей
sudo vim /etc/profile.d/golang.sh

# Set PATH so it includes Go bin's if it exists
```

```
PATH="$PATH:$HOME/go/bin"
export GOROOT=/opt/go
export GOPATH=$HOME/go
```

Перегружаем текущую Bash сессию, и проверяем, что изменения вступили в силу:

```
env | sort | grep -i go
```

```
GOPATH=/home/administrator/go
GOROOT=/opt/go
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/administrator/go/bin
```

```
go version
```

```
go version go1.25.5 linux/amd64
```

Так же закидываем в локальный Docker сборочный образ Go такой же версии, как и установленная:

```
docker pull golang:1.25.5
```

3.10.9 Создаём тестовый проект на Go

В виртуальной машине создаём папку для проектов:

```
mkdir -p ~/projects
cd ~/projects
```

Клонируем проект (сразу указав токен, чтобы не нужно было авторизоваться при внесении изменений):

```
git clone https://administrator:glpat-xxxxxxxxx-xxxxxxxxx@gitlab.dev.lan/administrator/hello-world.git
```

```
Cloning into 'hello-world'...
warning: You appear to have cloned an empty repository.
```

Заходим в клонированный проект:

```
cd hello-world
```

Создаём Go проект:

```
go mod init gitlab.dev.lan/administrator/hello-world
```

```
go: creating new go.mod: module gitlab.dev.lan/administrator/hello-world
```

Создаём файл main.go:

```
tee main.go << 'EOF' > /dev/null
package main

import (
    "context"
    "flag"
    "fmt"
    "log/slog"
    "net"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/prometheus/client_golang/prometheus/promhttp"
)

var version = "v0.0.0"
var buildDate = "0000-00-00T00:00:00Z"
var commitHash = "00000000"

func main() {
    // Flags
    serverPort := flag.Int("port", 4300, "WEB-server port")
    logLevel := flag.String("level", "debug", "Log level debug|info|warn|error")
    flag.Parse()

    // Logging
    slogLevel := &slog.LevelVar{} // Уровень логирования
    slogLevel.Set(map[string]slog.Level{"debug": slog.LevelDebug, "info": slog.LevelInfo, "warn": slog.LevelWarn, "error":
slog.LevelError}[*logLevel])
    slog.SetDefault(slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{Level: slogLevel})))

    slog.Info(fmt.Sprintf("%s %s %s logLevel: %s", version, buildDate, commitHash, slogLevel.Level()))

    mux := http.NewServeMux()

    mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        if r.URL.Path == "/" {
            slog.Debug(fmt.Sprintf("handling request for %s", r.URL.Path))
            host, _ := os.Hostname()
            addrs, _ := net.LookupIP(host)
            fmt.Fprintf(w, "Version: %s %s %s\nHostname: %s\nIP: %s", version, buildDate, commitHash, host,
addrs[0])
        } else {
            customNotFoundHandler(w, r)
        }
    })

    // liveness
    mux.HandleFunc("/healthz", func(w http.ResponseWriter, r *http.Request) {
        slog.Debug(fmt.Sprintf("handling request for %s", r.URL.Path))
        w.WriteHeader(http.StatusOK)
        fmt.Fprint(w, "OK")
    })

    // readiness
    mux.HandleFunc("/ready", func(w http.ResponseWriter, r *http.Request) {
        slog.Debug(fmt.Sprintf("handling request for %s", r.URL.Path))
        w.WriteHeader(http.StatusOK)
        fmt.Fprint(w, "OK")
    })

    // metrics
```

```

mux.HandleFunc("/metrics", func(w http.ResponseWriter, r *http.Request) {
    slog.Debug(fmt.Sprintf("handling request for %s", r.URL.Path))
    promhttp.Handler().ServeHTTP(w, r)
})

// Запуск WEB-сервера с graceful остановкой.

server := &http.Server{
    Addr:    fmt.Sprintf(":%d", *serverPort),
    Handler: mux,
}

go func() {
    slog.Info(fmt.Sprintf("server running on port %s", server.Addr))
    if err := server.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        slog.Error(fmt.Sprintf("server terminated with an error: %v", err))
        os.Exit(1)
    }
}()

stop := make(chan os.Signal, 1)
signal.Notify(stop, syscall.SIGINT, syscall.SIGTERM)

<-stop // Ожидаем сигнала остановки
slog.Info("server stop signal received")

ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()

if err := server.Shutdown(ctx); err != nil {
    slog.Error(fmt.Sprintf("error while shutting down the server: %v", err))
    os.Exit(1)
} else {
    slog.Info("server has terminated successfully")
}
}

func customNotFoundHandler(w http.ResponseWriter, r *http.Request) {
    slog.Error(fmt.Sprintf("error 404; handling request for %s", r.URL.Path))
    w.WriteHeader(http.StatusNotFound)
    _, _ = w.Write([]byte("404 Not Found\n"))
}
EOF

```

Подгружаем используемые модули:

```

go get github.com/prometheus/client_golang/prometheus/promhttp
go mod tidy

```

Создаём файл тестов:

```

tee main_test.go << 'EOF' > /dev/null
package main

import "testing"

func Test_main(t *testing.T) {
    tests := []struct {
        name string
    }{}
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            main()
        })
    }
}
EOF

```

Запускаем проект:

```

go run main.go --port=4300 --level=info &

```

```

time=2026-01-09T15:32:51.464+03:00 level=INFO msg="v0.0.0 0000-00-00T00:00:00Z 00000000\n"
time=2026-01-09T15:32:51.465+03:00 level=INFO msg="server running on port :4300"

```

Проверяем доступность запросами:

```

curl http://localhost:4300/
curl http://localhost:4300/healthz
curl http://localhost:4300/ready

```

Завершаем работу программы:

```

kill %1

```

Проверяем, что проект запускается из Go расположенного в Docker:

```

docker run --rm -v "$PWD":/app -e GOCACHE=/tmp/go-build -e GOMODCACHE=/tmp/go-mod -w /app -u "$(id -u):$(id -g)" \
    golang:1.23.10-alpine3.22 go run main.go

v0.0.0 0000-00-00T00:00:00Z 00000000
Server started on port :4300

```

Нажимаем **Ctrl + C** для завершения работы программы (будет небольшая задержка с выходом).

Создаём Dockerfile:

```

vim Dockerfile

FROM alpine:3.21.3

RUN apk --no-cache add ca-certificates

WORKDIR /root/

# Копируем бинарник, собранный в GitLab CI (он будет в ./build/hello-world)
COPY build/hello-world .

CMD ["/hello-world", "--port", "4300", "--level", "info"]

```

Создаём манифест для деплоя результирующего образа в Kubernetes:

```

mkdir -p kubernetes
vim kubernetes/hello-world-deployment.yaml

```

```

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          # Переменные будут подставлены через envsubst из файла .gitlab-ci.yml
          image: $CI_REGISTRY_IMAGE:$IMAGE_TAG
          ports:
            - containerPort: 4300
              protocol: TCP
              name: http
          imagePullSecrets:
            - name: gitlab-registry-credentials
---
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  # NodePort открывает порт непосредственно на хостах
  type: NodePort
  selector:
    app: hello-world
  ports:
    - protocol: TCP
      port: 4301
      targetPort: 4300
      nodePort: 31001 # опционально, иначе назначится автоматически в диапазоне 30000-32767

```

Создаём файл CI `.gitlab-ci.yml` (спецификация YAML-файла <https://docs.gitlab.com/ci/yaml/>):

```
vim .gitlab-ci.yml
```

```

---
# Глобальный фильтр, который определяет, будет ли пайплайн запущен вообще до выполнения каких-либо jobs.
# Это "правила для правил", которые действуют на уровне всего пайплайна.
workflow:
  rules:
    - if: $CI_COMMIT_TAG # Если передан тэг - тест, сборка и деплой в prod.
    - if: $CI_COMMIT_BRANCH == "main" # Если добавлен коммит в main - тест, сборка и деплой в staging-окружение
    - if: $CI_PIPELINE_SOURCE == "web" # Ручной запуск через UI
    - when: never # Блокировка всего остального

# Pipeline
stages:
  - test # Стадия тестирования
  - build # Стадия сборки
  - push-to-registry # Пересылка сборки в локальный реестр
  - deploy # CD в Kubernetes

# Job тестирования
test:
  stage: test # Привязываем job к стадии "test"
  image: golang:1.25.5 # Используем официальный образ Go
  tags: ["shared"] # Используемые Runners

  # Действия до запуска скрипта
  before_script:
    - go mod download # Скачиваем зависимости перед тестами

  # Рабочий скрипт
  script:
    - printenv | sort > job-test.env # Сохраняем для дальнейшего анализа переменные окружения
    - go test -v ./... # Запускаем все тесты в проекте (рекурсивно) с выводом логов (-v)

  # Сохраняем файлы в артефактах
  artifacts:
    paths:
      - job-test.env # Сохраняем файл с переменными в артефактах для дальнейшего анализа
    expire_in: 1 day

  # Срабатывает только при соответствии правилам
  rules:
    - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_TAG # Запускать при коммите в main или установке тэгов

# Job сборки без версии (запускается только после успешного тестирования)
build_no_tag:
  stage: build
  image: golang:1.25.5
  needs: ["test"] # Явная зависимость от тестов
  tags: ["shared"] # Используемые Runners

  # Действия до запуска скрипта
  before_script:
    - go mod download # Скачиваем зависимости перед тестами

  script:
    - export BUILD_DATE=$(date -u +"%Y-%m-%dT%H:%M:%SZ") # Создаём переменную с датой сборки
    - >
      CGO_ENABLED=0 GOOS=linux go build -ldflags
      "
      -X main.buildDate=$BUILD_DATE
      -X main.commitHash=$CI_COMMIT_SHORT_SHA
      "

```

```

    -o build/hello-world main.go
  - echo "Build - BUILD_DATE=$BUILD_DATE, CI_COMMIT_SHORT_SHA=$CI_COMMIT_SHORT_SHA"

# Помещаем директорию с результатом сборки в "артефакты" GitLab для получения из на следующих шагах
artifacts:
  paths:
    - build/ # Директория, для помещения в "артефакты"
  expire_in: 1 day # Храним артефакты 1 день

rules:
  - if: $CI_COMMIT_BRANCH == "main" # Запустить ТОЛЬКО при коммитах в main, без версий

# Job сборки с тэгом сборки (запускается только после успешного тестирования)
build_with_tag:
  stage: build
  image: golang:1.25.5
  needs: ["test"] # Явная зависимость от тестов
  tags: ["shared"] # Используемые Runners

# Действия до запуска скрипта
before_script:
  - go mod download # Скачиваем зависимости перед тестами

script:
  - export BUILD_DATE=$(date -u +"%Y-%m-%dT%H:%M:%SZ") # Создаём переменную с датой сборки
  - >
    CGO_ENABLED=0 GOOS=linux go build -ldflags
    "
    -X main.version=$CI_COMMIT_TAG
    -X main.buildDate=$BUILD_DATE
    -X main.commitHash=$CI_COMMIT_SHA
    "
  -o build/hello-world main.go
  - echo "Build - CI_COMMIT_TAG=$CI_COMMIT_TAG, BUILD_DATE=$BUILD_DATE, CI_COMMIT_SHORT_SHA=$CI_COMMIT_SHORT_SHA"

# Помещаем директорию с результатом сборки в "артефакты" GitLab для получения из на следующих шагах
artifacts:
  paths:
    - build/ # Директория, для помещения в "артефакты"
  expire_in: 1 day # Храним артефакты 1 день

rules:
  - if: $CI_COMMIT_TAG # Запустить ТОЛЬКО при коммитах с версией

# Job пересылки сборки в локальный registry (запускается только после присвоения тэга).
push_to_registry:
  stage: push-to-registry
  image: docker:28.1.1
  needs: ["build_with_tag"]
  tags: ["shared"] # Используемые Runners

services:
  - name: docker:28.1.1-dind
    alias: docker # Явный алиас для сервиса

script:
  # Для теста
  - ls -la build/
  # Авторизируемся в Registry с помощью временных логина и пароля
  - echo "$CI_REGISTRY_PASSWORD" | docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin
  # Собираем Docker-образ
  - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG .
  - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG
  # Дополнительно пушим latest (опционально)
  - docker tag $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG $CI_REGISTRY_IMAGE:latest
  - docker push $CI_REGISTRY_IMAGE:latest
  - rm -f /root/.docker/config.json

rules:
  - if: $CI_COMMIT_TAG # Запустить ТОЛЬКО при коммитах с версией

# CD в Kubernetes
deploy:
  stage: deploy
  image:
    name: bitnami/kubectl:latest
    entrypoint: [""] # Сбрасываем entrypoint, чтобы запускались команды типа sh -c "kubectl version --client"
  tags: ["shared"]
  variables:
    # Переменная окружения для kubectl, где ему искать конфигурацию
    KUBECONFIG: /tmp/kubeconfig
  before_script:
    - echo "$KUBECONFIG_B64" | base64 -d > $KUBECONFIG
    - chmod 600 $KUBECONFIG
  script:
    # Для тестирования (`kubectl` пропускаем, оставляем только аргументы к нему)
    - kubectl cluster-info
    - kubectl get nodes
    # Подставляем переменные окружения в файл манифеста и передаём его в kubectl для деплоя
    - export IMAGE_TAG=$CI_COMMIT_TAG
    - envsubst < kubernetes/hello-world-deployment.yaml | kubectl apply -n staging -f -
  rules:
    - if: $CI_COMMIT_TAG # Запустить ТОЛЬКО при коммитах с версией

```

Валидировать YAML синтаксис можно командой:

```
| yamllint .gitlab-ci.yml
```

Валидировать корректность .gitlab-ci.yml можно в online валидаторе:

- <https://gitlab.dev.lan/administrator/hello-world/-/ci/lint>

Создаём Namespace staging в Kubernetes для deployment результата:

```
| kubectl create namespace staging
```

Для шага deploy нужен токен для доступа к кластеру Kubernetes. Помещаем текущий ~/.kube/config, через который работает kubectl на gateway в переменную GitLab (в Base64) и помещаем её в переменную.

Кодируем текущий ~/.kube/config в Base64:

```
cat ~/.kube/config | base64 -w 0
```

... и вывод помещаем в переменную GitLab (переменные GitLab — <https://docs.gitlab.com/ci/variables/>):

- **Страница проекта hello-world -> Project settings** (справа вверху три точки) -> **CI/CD -> Variables -> Add variable:**
 - **Type** — Variables
 - **Environments** — All
 - **Visibility** — Visible
 - **Protected variable** — []
 - **Key** — KUBECONFIG_B64
 - **Value** — <вставить скопированный base64-текст>
 - Нажимаем кнопку **Add variable**.

Добавляем изменения в Git и отправляем на сервер:

```
git config --global user.email "administrator@mail.dev.lan"
git config --global user.name "Administrator"

# Создаём и переключаемся на ветку main
git switch --create main
git status
git add .
git commit -m "Первый коммит"
git push origin main
```

Заходим в GUI GitLab в папку проекта **hello-world** → **Build** (слева) → **Pipelines** и видим, как прошёл процесс тестирования и сборки проекта с помещением результата компилирования в "артефакты":

Status	Pipeline	Created by	Stages	Actions
Passed 00:00:12 19 seconds ago	Первый коммит #1 P main ac435399 latest branch		✓ ✓	↓

Теперь присваиваем тэг для текущего коммита и отправляем на сервер. Обработывает другой сценарий: тестирование, сборка и помещение результата в GitLab Registry:

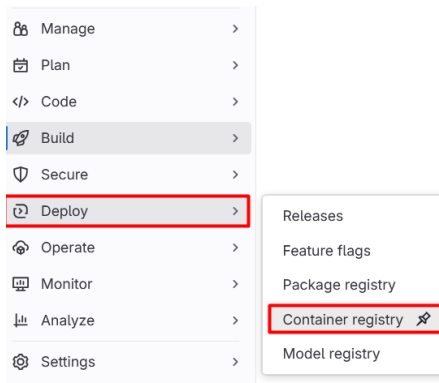
```
git tag v1.0.0
git push origin v1.0.0
```

```
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://gitlab.dev.lan/administrator/hello-world.git
* [new tag]          v1.0.0 -> v1.0.0
```

Заходим в GUI GitLab в папку проекта **hello-world** → **Build** (слева) → **Pipelines** и видим, что к процессу сборки добавился процесс сохранения результата в виде образа в локальном репозитории Registry:

Status	Pipeline	Created by	Stages	Actions
Passed 00:00:15 19 seconds ago	Первый коммит #2 v1.0.0 ac435399 latest tag		✓ ✓ ✓	↓

Проверяем, что образы сохранены в GitLab Registry. Для этого заходим в локальный Registry через меню **Deploy** → **Container registry**:



Видим, что образ сохранён с тэгами **latest** (здесь всегда будет самый свежий из добавляемых образов; они одинаковы, что можно увидеть по их контрольной сумме *Digest*) и **v1.0.0**:

hello-world		
2 tags Cleanup disabled Created Aug 20, 2025 12:56		
Filter results		
2 tags		
latest	756 MB	Published 1 minute ago Digest: 8b44302
v1.0.0	756 MB	Published 1 minute ago Digest: 8b44302

В консоли проверяем, что пользователь **administrator** через свой токен может видеть эти образа. Для этого получаем токен (указав зону его действия `registry:repository:administrator/hello-world:pull`):

```
TOKEN=$(curl -s -u "administrator:glpat-XXXXXXXXXXXXXXXXXXXX"
"https://gitlab.dev.lan/jwt/auth?service=container_registry&scope=repository:administrator/hello-world:pull" | jq -r '.token')
```

Получаем список тэгов репозитория:

```
curl -H "Authorization: Bearer $TOKEN" https://registry.dev.lan/v2/administrator/hello-world/tags/list
```

```
{"name": "administrator/hello-world", "tags": ["latest", "v1.0.0"]}
```

Если возникли ошибки, распаковываем токен и проверяем, что права на запрос указаны правильно:

```
echo $TOKEN | cut -d'.' -f2 | base64 -d | jq .
```

```
{
  "auth_type": "personal_access_token",
  "access": [ // <-- это поле отвечает за разрешения доступа
    {
      "type": "repository",
      "name": "administrator/hello-world",
      "actions": ["pull"], // <-- это разрешение на чтения репозитория administrator/hello-world
      "meta": {
        "project_path": "administrator/hello-world",
        "project_id": 1,
        "root_namespace_id": 2
      }
    }
  ],
  "user": "XXX...",
  "jti": "903dfde7-36bc-447b-ab51-3f5363c6deb0",
  "aud": "container_registry",
  "sub": "administrator",
  "iss": "omnibus-gitlab-issuer",
  "iat": 1755696715,
  "nbf": 1755696710,
  "exp": 1755697015
}
```

Конфигурируем Traefik на балансировку трафика между хостами combo-1.dev.lan, combo-2.dev.lan и combo-3.dev.lan. Для этого добавляем файл правил в уже существующую папку /rules:

```
cd ~/compose/traefik/
vim rules/hello.yaml
```

```
http:
  services:
    hello:
      loadBalancer:
        servers:
          - url: "http://combo-1.dev.lan:31001"
          - url: "http://combo-2.dev.lan:31001"
          - url: "http://combo-3.dev.lan:31001"

  routers:
    hello:
      rule: "Host(`hello.dev.lan`)"
      entryPoints:
        - websecure
      service: hello
      tls:
        certResolver: stepca
```

Перезагружать Traefik не нужно, правила подхватятся автоматически. Проверяем результат:

```
curl https://hello.dev.lan
```

... или заходим в браузере:

- <https://hello.dev.lan/>

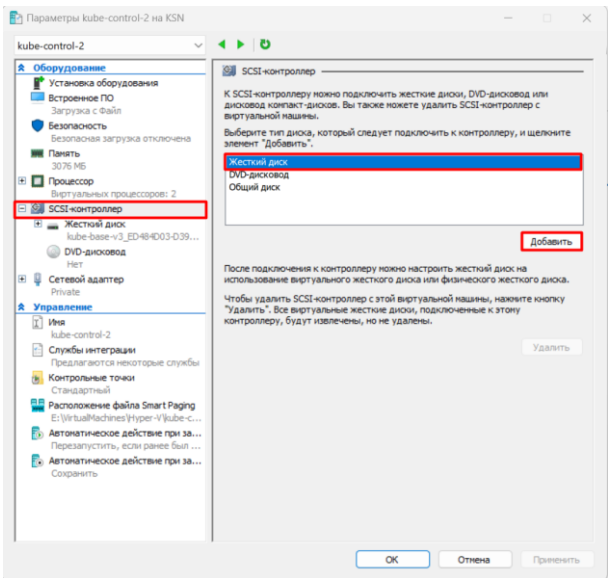
3.11 Установка MinIO

MinIO — это высокопроизводительный объектный сервер хранения данных (Object Storage) с открытым исходным кодом, совместимый с API Amazon S3. Он предназначен для хранения и управления большими объемами неструктурированных данных (фотографии, видео, логи, резервные копии и т.д.). Устанавливаем на все 3 управляющих сервера в виде кластера.

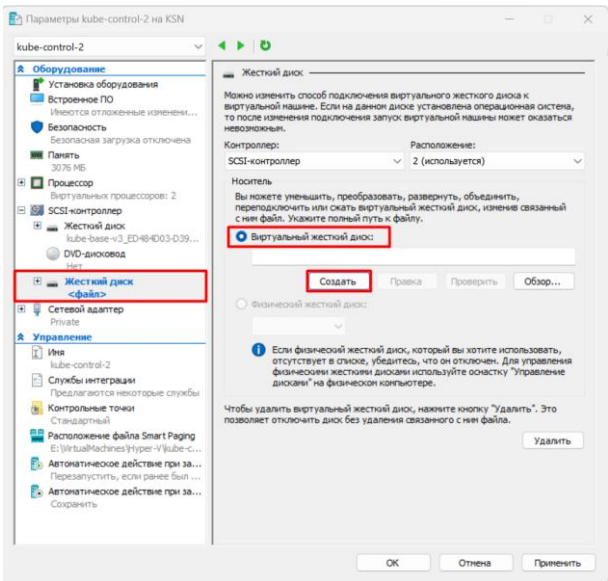
3.11.1 Добавляем отдельный диск для объектного хранилища

Для работы MinIO рекомендуется сделать отдельный диск, только для монопольного использования (чтобы зафиксировать размер раздела). Для этого в Hyper-V каждого сервера добавляем по новому диску, которые примонтируем к виртуальной машине.

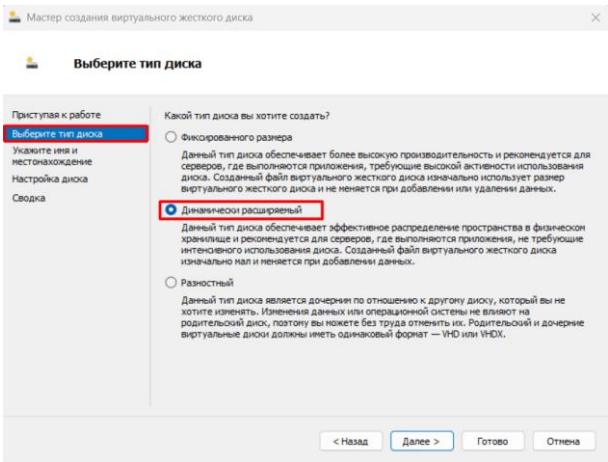
Заходим в параметры виртуальной машины и переходим в раздел SCSI-контроллер → Жёсткий диск → Добавить:



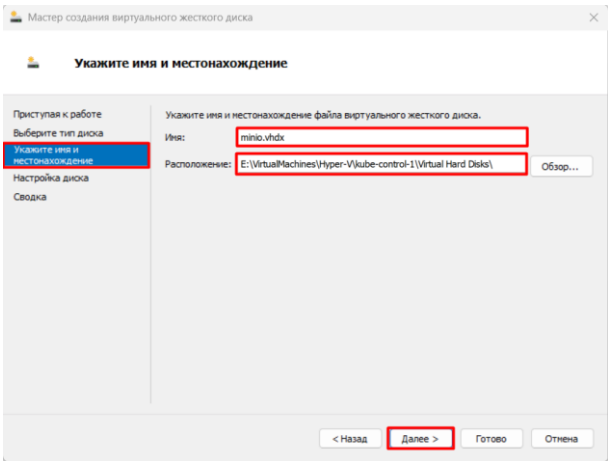
Переходим к созданному жёсткому диску и выбираем пункт Виртуальный жёсткий диск → Создать:



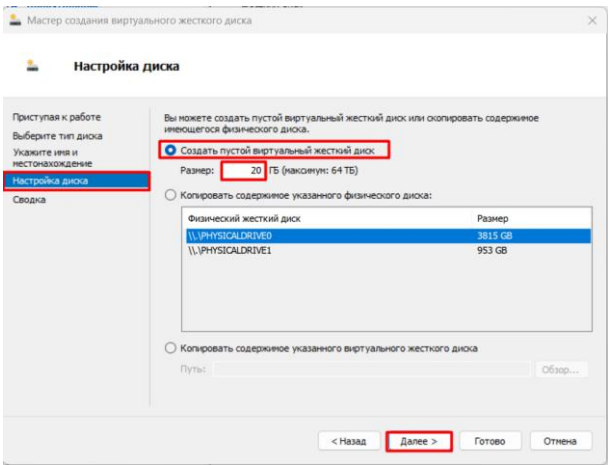
Выбираем тип диска Динамически расширяемый:



Указываем имя диска **minio.vhdx** и расположение виртуальной машины и виртуальных жёстких дисков в ней:



Указываем размер виртуального жёсткого диска в **20 Гб**:



Ищем созданный раздел на загруженной виртуальной машине:

```
| lsblk -f
```

NAME	FSTYPE	FSVER	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINTS
sda							
└─sda1	vfat	FAT32		8AA6-202B	945M	1%	/boot/efi
└─sda2	ext4	1.0		ead4a3fe-28c3-4952-ac39-61730ace06c6	13.7G	21%	/
sdb							
sr0							

Форматируем его:

```
| sudo mkfs.ext4 /dev/sdb
```

```
mke2fs 1.47.1 (20-May-2024)
Discarding device blocks: done
Creating filesystem with 5242880 4k blocks and 1310720 inodes
Filesystem UUID: 0508f52c-962b-4cf7-8252-7cf98691e48b
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000
Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done
```

Проверяем, что раздел успешно создан:

```
| lsblk -f
```

NAME	FSTYPE	FSVER	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINTS
sda							
└─sda1	vfat	FAT32		8AA6-202B	945M	1%	/boot/efi
└─sda2	ext4	1.0		ead4a3fe-28c3-4952-ac39-61730ace06c6	13.7G	21%	/
sdb	ext4	1.0		0508f52c-962b-4cf7-8252-7cf98691e48b			
sr0							

Для автозагрузки раздела, определяем его UUID:

```
| sudo blkid /dev/sdb
```

```
/dev/sdb: UUID="0508f52c-962b-4cf7-8252-7cf98691e48b" BLOCK_SIZE="4096" TYPE="ext4"
```

Создаём пользователя **minio-user**, от которого будет работать MinIO:

```
| sudo groupadd -r minio-user
sudo useradd -M -r -g minio-user minio-user
sudo chown minio-user:minio-user /usr/local/bin/minio
```

Создаём директорию, куда будет смонтирован диск:

```
| sudo mkdir /mnt/minio-data
sudo chown -R minio-user:minio-user /mnt/minio-data
```

Добавляем автомонтирование при загрузке системы, добавив в файл **/etc/fstab** запись:

```
sudo nano /etc/fstab

...

# MinIO disk /dev/sdb
/dev/disk/by-uuid/0508f52c-962b-4cf7-8252-7cf98691e48b /mnt/minio-data ext4 defaults 0 2
```

где:

- UUID=... — идентификатор диска.
- /mnt/minio-data — точка монтирования.
- ext4 — файловая система.
- defaults — стандартные параметры монтирования (можно уточнить, например, defaults,nofail если диск не критичен для загрузки).
- 0 — отключение dump (резервного копирования).
- 2 — порядок проверки файловой системы (2 для не-root разделов).

Проверяем, что раздел монтируется выполнив все монтирования из файла /etc/fstab:

```
sudo mount -a
```

Проверяем, что раздел смонтирован успешно:

```
mount | grep sdb
```

```
/dev/sdb on /mnt/minio-data type ext4 (rw,relatime)
```

3.11.2 Устанавливаем MinIO

Скачиваем и устанавливаем бинарный файл minio:

Необходимо устанавливать версии только **RELEASE.2025-04-22T22-12-26Z** и ниже. В версиях выше, в WEB-интерфейсе отключена все административные настройки.

```
# Скачает последнюю версию, с отключёнными основными настройками через WEB-интерфейс. Нужно искать версию RELEASE.2025-04-22T22-12-26Z
wget https://dl.min.io/server/minio/release/linux-amd64/archive/minio.RELEASE.2025-04-22T22-12-26Z -O ~/minio
```

Устанавливаем MinIO и задаём ему соответствующие права:

```
sudo mv minio /usr/local/bin/
sudo chown minio-user:minio-user /usr/local/bin/minio
sudo chmod +x /usr/local/bin/minio
```

Создаём сервис для запуска MinIO:

```
sudo nano /etc/systemd/system/minio.service
```

```
[Unit]
Description=MinIO
After=network.target

[Service]
Type=simple
User=minio-user
Group=minio-user
Environment="MINIO_ROOT_USER=admin"
Environment="MINIO_ROOT_PASSWORD=PasswordMinio123"
ExecStart=/usr/local/bin/minio server \
  http://192.168.20.11/mnt/minio-data \
  http://192.168.20.12/mnt/minio-data \
  http://192.168.20.13/mnt/minio-data \
  --address ":9000" \
  --console-address ":9001"

Restart=always
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

где:

- MINIO_ROOT_USER=admin — логин администратора
- MINIO_ROOT_PASSWORD=PasswordMinio123 — пароль администратора
- --address ":9000" — порт по умолчанию для API
- --console-address ":9001" — порт для доступа к WEB-интерфейсу

Ставим сервис в автозагрузку и перезагружаем его:

```
sudo systemctl daemon-reload
sudo systemctl enable minio
sudo systemctl start minio
sudo systemctl status minio # Проверяем статус
```

```
• minio.service - MinIO
  Loaded: loaded (/etc/systemd/system/minio.service; enabled; preset: enabled)
  Active: active (running) since Fri 2025-05-02 20:56:44 MSK; 3s ago
  Invocation: 378a6b6f70af4ac3ac58743f1c113f65
  Main PID: 51121 (minio)
  Tasks: 8 (limit: 3001)
  Memory: 151.7M (peak: 152.2M)
  CPU: 270ms
  CGroup: /system.slice/minio.service
          └─51121 /usr/local/bin/minio server http://192.168.20.11/mnt/minio-data http://192.168.20.12/mnt/minio-data
            http://192.168.20.13/mnt/minio-data --console-address :9001
```

MinIO заработает только тогда, когда будет видеть всех 3-х своих элемента кластера и будет видеть все диски /mnt/minio-data. Поэтому перед заходом на WEB-интерфейс нужно включить его на всех нодах (иначе WEB-интерфейс будет недоступен).

Проверяем, что порт :9001 поднят:

```
| sudo ss -tulnp | grep minio

tcp  LISTEN  0      4096      127.0.0.1:9000      0.0.0.0:*      users:(("minio",pid=5326,fd=6))
tcp  LISTEN  0      4096      *:9000             *:~*      users:(("minio",pid=5326,fd=8))
tcp  LISTEN  0      4096      *:9001             *:~*      users:(("minio",pid=5326,fd=15))
```

Проверка через заход на WEB-интерфейс:

- <http://192.168.20.11:9001/browser> — login: **admin** pass: **PasswordMinio123**

Просмотр журнала, где можно понять, почему WEB-интерфейс не запущен:

```
| sudo journalctl -u minio -f
```

3.11.3 Настройка Keepalived

Делаем кластерный IP-адрес для кластеризации MinIO.

Создаём скрипт, предназначенный для проверки доступности серверов:

```
| sudo nano /etc/keepalived/check_minio.sh

#!/bin/sh

curl -s http://localhost:9000/minio/health/live >/dev/null || exit 1
```

Устанавливаем разрешение на запуск скриптов:

```
| sudo chmod +x /etc/keepalived/check_minio.sh
```

Дополняем файл конфигурации /etc/keepalived/keepalived.conf новым экземпляром:

```
| sudo nano /etc/keepalived/keepalived.conf

#...

# Определение скрипта для проверки состояния (в данном случае API-сервера)
vrrp_script check_minio {
    # Путь к скрипту, который будет выполняться для проверки
    script "/etc/keepalived/check_minio.sh"

    # Интервал выполнения скрипта в секундах (каждые 3 секунды)
    interval 3
}

# Определение VRRP-инстанса (виртуального маршрутизатора)
vrrp_instance VI_2 {
    # Начальное состояние - BACKUP (будет переходить в MASTER при необходимости)
    state BACKUP

    # Сетевой интерфейс, на котором будет работать VRRP
    interface eth0

    # Идентификатор виртуального маршрутизатора (должен быть одинаковым в группе)
    # Должен отличаться от первого инстанса (5)
    virtual_router_id 6

    # Приоритет этого узла (чем выше, тем больше шансов стать MASTER)
    priority 100

    # Интервал объявлений (advertisements) в секундах
    advert_int 1

    # Запрет на переход в MASTER режим, если узел с более высоким приоритетом вернется в строй
    nopreempt

    # Настройки аутентификации между узлами VRRP
    authentication {
        # Тип аутентификации - пароль
        auth_type PASS

        # Пароль для аутентификации (должен быть одинаковым на всех узлах группы)
        # Пароль может совпадать с первым инстансом
        auth_pass ZqSj#f16
    }

    # Виртуальный IP-адрес, который будет переключаться между узлами MinIO
    virtual_ipaddress {
        192.168.20.102
    }

    # Скрипт, который будет отслеживаться для принятия решений о переходе между состояниями
    track_script {
        check_minio
    }
}
```

Запускаем Keepalived:

```
| sudo systemctl restart keepalived
| sudo systemctl status keepalived

● keepalived.service - Keepalive Daemon (LVS and VRRP)
   Loaded: loaded (/usr/lib/systemd/system/keepalived.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-05-02 21:55:13 MSK; 9ms ago
 Invocation: a7bf202f35c04223aab361ec2c3694c7
    Docs: man:keepalived(8)
          man:keepalived.conf(5)
          man:genhash(1)
          https://keepalived.org
   Main PID: 78489 (keepalived)
     Tasks: 2 (limit: 3001)
  Memory: 1.8M (peak: 3.8M)
     CPU: 10ms
   CGroup: /system.slice/keepalived.service
           └─78489 /usr/sbin/keepalived --dont-fork
             └─78490 /usr/sbin/keepalived --dont-fork
```

Проверяем журнал keepalived:

```
| journalctl -u keepalived -f
```

Повторяем для остальных управляющих серверов **combo-2** и **combo-3**.

3.11.4 Проверка работоспособности

Проверяем, что виртуальный IP поднят:

```
curl http://192.168.20.102:9000
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Error><Code>AccessDenied</Code><Message>Access
Denied.</Message><Resource></Resource><RequestId>183BCAED9A7FD37F</RequestId><HostId>b8deefb17e4fc93acbb4efdddf4fdafe365df01c09c2b3b47feffb1e3
8963031</HostId></Error>
```

Или заходим через WEB-интерфейс:

- login: **admin** pass: **PasswordMinio123** — <http://192.168.20.11:9001/browser>

Чтобы понять, какому серверу сейчас принадлежит виртуальный IP, нужно пройтись по всем серверам **combo-1,2,3** и проверить присвоение IP. В примере смотрим так же и первый виртуальный IP:

```
ip -br addr | grep "192.168.20.10[12]"
```

```
eth0          UP          192.168.20.11/24 192.168.20.102/32 192.168.20.101/32 fe80::215:5dff:fe01:c639/64
```

4 Kubernetes — отработка

4.1 Namespace

Создаём свой рабочий namespace:

```
kubectl create namespace my-namespace

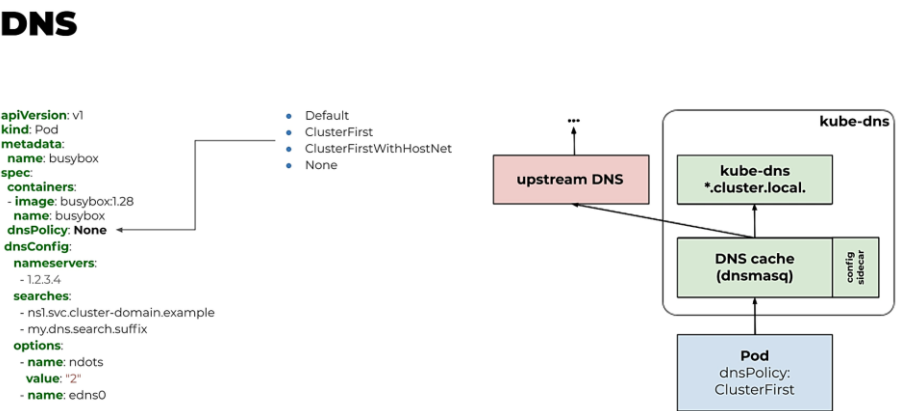
namespace/my-namespace created
```

Проверяем, что namespace создан:

```
kubectl get namespaces

NAME              STATUS   AGE
default           Active   39d
kube-flannel      Active   39d
kube-node-lease   Active   39d
kube-public       Active   39d
kube-system       Active   39d
my-namespace      Active   58s
```

4.2 DNS



Посмотрим, какой DNS-резолвер будет использоваться в поде по умолчанию. Запускаем кастомный временный под и заходим в него:

```
kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
```

Уже внутри пода, смотрим его IP-адрес:

```
ip addr

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if40: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue state UP qlen 1000
   link/ether 9a:8a:62:09:27:3f brd ff:ff:ff:ff:ff:ff
   inet 10.244.0.61/24 brd 10.244.0.255 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::988a:62ff:fe09:273f/64 scope link
       valid_lft forever preferred_lft forever
```

Адрес шлюза:

```
ip route

default via 10.244.0.1 dev eth0
10.244.0.0/24 dev eth0 scope link src 10.244.0.61
10.244.0.0/16 via 10.244.0.1 dev eth0
```

DNS-адрес:

```
cat /etc/resolv.conf

nameserver 10.96.0.10
search my-namespace.svc.cluster.local svc.cluster.local cluster.local dev.lan
options ndots:5
```

... где:

- 10.96.0.10 — IP-адрес DNS-сервера
- search ... — доменные суффиксы

Определяем FDQN имя DNS-сервера:

```
nslookup 10.96.0.10

Server:      10.96.0.10
Address:     10.96.0.10:53

10.0.96.10.in-addr.arpa name = kube-dns.kube-system.svc.cluster.local
```

Проверяем, что на DNS-сервере открыт :53 порт (сам хост не пингуется):

```
nc -zv 10.96.0.10 53

10.96.0.10 (10.96.0.10:53) open
```

Смотрим имя текущего хоста:

```
hostname
```

```
my-temp
```

Смотрим место, где имя хоста резолвится в его IP:

```
cat /etc/hosts
```

```
# Kubernetes-managed hosts file.
127.0.0.1      localhost
::1           localhost ip6-localhost ip6-loopback
fe00::0       ip6-localnet
fe00::0       ip6-mcastprefix
fe00::1       ip6-allnodes
fe00::2       ip6-allrouters
10.244.0.61    my-temp
```

Трассируем путь до хоста 8.8.8.8:

```
traceroute -n 8.8.8.8
```

```
 1  10.244.0.1  0.004 ms  0.002 ms  0.002 ms
 2  192.168.20.1  0.131 ms  0.113 ms  0.078 ms
 3  172.23.96.1  0.212 ms  0.277 ms  0.308 ms
 4  192.168.1.1  2.065 ms  1.928 ms  1.800 ms

... # Далее уже роутеры провайдера
```

4.3 Node

Получаем текущую версию API для спецификации Node:

```
kubectl explain node | sed '/^$/{Q;}'
```

```
KIND:      Node
VERSION:   v1
```

Получить весь список лейблов, присвоенных всем нодам:

```
kubectl get nodes --show-labels
```

```
NAME          STATUS    ROLES    AGE   VERSION   LABELS
combo-1       Ready     control-plane  21d   v1.32.9   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=combo-1,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node,kubernetes.io/exclude-from-external-load-balancers=combo-2,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node,kubernetes.io/exclude-from-external-load-balancers=combo-3
combo-2       Ready     control-plane  21d   v1.32.9   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=combo-2,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node,kubernetes.io/exclude-from-external-load-balancers=combo-3
combo-3       Ready     control-plane  21d   v1.32.9   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=combo-3,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node,kubernetes.io/exclude-from-external-load-balancers=
```

Добавить собственную метку на ноду:

```
kubectl label nodes combo-1 my-label=my-value --overwrite
```

```
node/combo-1 labeled
```

... где:

- `--overwrite` — перезаписать метку, если она уже существует.

```
kubectl get nodes combo-1 --show-labels
```

```
NAME          STATUS    ROLES    AGE   VERSION   LABELS
combo-1       Ready     control-plane  21d   v1.32.9   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=combo-1,kubernetes.io/os=linux,my-label=my-value,node-role.kubernetes.io/control-plane=node,kubernetes.io/exclude-from-external-load-balancers=
```

... где:

- `--show-labels` — показать лейблы объекта.

4.4 События в Kubernetes

Можно посмотреть события, которые происходят в kubernenes в режиме живого времени:

```
kubectl get events -A --watch
```

```
my-namespace 14m      Normal    Pulled          pod/my-initcontainer  Container image "busybox:1.37.0" already present on machine
my-namespace 14m      Normal    Created         pod/my-initcontainer  Created container: my-init-app
my-namespace 14m      Normal    Started         pod/my-initcontainer  Started container my-init-app
```

Можно отфильтровать события только для Pod с нужным именем, например my-initcontainer:

```
kubectl get events -n my-namespace --field-selector involvedObject.name=my-initcontainer
```

```
LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
28m         Normal    Scheduled    pod/my-initcontainer  Successfully assigned my-namespace/my-initcontainer to combo-1
28m         Normal    Pulled       pod/my-initcontainer  Container image "busybox:1.37.0" already present on machine
28m         Normal    Created      pod/my-initcontainer  Created container: my-init-app
28m         Normal    Started      pod/my-initcontainer  Started container my-init-app
9m7s        Normal    Pulled       pod/my-initcontainer  Container image "busybox:1.37.0" already present on machine
9m7s        Normal    Created      pod/my-initcontainer  Created container: my-app
9m7s        Normal    Started      pod/my-initcontainer  Started container my-app
```

4.5 Pod

Получаем текущую версию API для спецификации Pod:

```
kubectl explain pod | sed '/^$/{Q;}'
```

```
KIND:      Pod
VERSION:   v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-pods
```

4.5.1 Простой под

Создаём файл с описанием спецификации:

```
cd ~/kubernetes/example-pods
```

```
| vim pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
spec:
  containers:
    - name: my-app
      image: nginx:1.29.1
```

Т.к. это новый под, создаём его через create, а не обновляем через apply:

```
| kubectl apply -f pod.yaml

pod/my-pod created
```

Проверяем, что под создался успешно:

```
| kubectl get pods -n my-namespace -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-pod	1/1	Running	0	15s	10.244.0.3	combo-1	<none>	<none>	<none>	<none>

Пробрасываем порт my-pod с порта :9375 на порт :9376 для всех интерфейсов (в фоновом режиме):

```
| kubectl port-forward -n my-namespace pods/my-pod --address 0.0.0.0 9376:9376 &
```

Проверяем работоспособность пода:

```
| curl http://127.0.0.1:9376

my-pod
```

... или снаружи <http://gateway.dev.lan:9376>

Завершаем переедресацию:

```
| kill %1
```

Убиваем под:

```
| kubectl delete pods -n my-namespace my-pod

pod "my-pod" deleted
```

4.5.2 Два контейнера в поде

Создаём два контейнера в поде, один из которых будет сервер, второй - клиент.

Сервер создаём в виде программы на Python упакованного в контейнер. Клиент будет в виде официального контейнера curl, запущенного с заданными параметрами.

Создаём контейнер с сервером:

```
| cd ~/kubernetes/example-pods
vim pods-2-in-1.py

from flask import Flask, jsonify
import datetime

app = Flask(__name__)

@app.route('/date')
def index():
    dateNow = datetime.date.today().strftime("%m/%d/%Y")
    return jsonify(date=dateNow)

if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port='8080')
```

Создаём файл зависимостей для Python:

```
| vim requirements.txt

Flask
```

Создаём Docker файл для запуска сервера:

```
| vim Dockerfile

FROM python:3.12.11-alpine3.22
WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip3 install -r requirements.txt
COPY . .
CMD [ "python3", "pods-2-in-1.py"]
```

Собираем контейнер:

```
| docker buildx build -t server:1.0 .
```

Проверяем, что контейнер создан:

```
| docker images | grep server
```

server	1.0	3176c16be01a	39 seconds ago	62.3MB
server	1.0	3176c16be01a	39 seconds ago	62.3MB

Переносим созданный образ с gateway в локальные Docker нод combo-1,2,3:

```
| # Экспортируем контейнер в сжатый файл:
docker save server:1.0 | gzip > server-1.0.tar.gz

for host in combo-1.dev.lan combo-2.dev.lan combo-3.dev.lan; do
# Удаляем старый образ, если он есть
ssh $host "docker rmi server:1.0 2>/dev/null || true"
# Копируем сжатый контейнер на ноды в папку /tmp
scp server-1.0.tar.gz $host:/tmp/
```

```
# Запускаем процесс импорта образа в локальный Docker
ssh $host "gunzip -c /tmp/server-1.0.tar.gz | docker load"
# Удаляем старый файл
ssh $host "rm /tmp/server-1.0.tar.gz"
done
```

Создаём спецификацию пода:

```
vim pods-2-in-1.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
spec:
  containers:
    - name: my-server
      image: server:1.0

    - name: my-client
      image: curlimages/curl:8.16.0
      command: ["/bin/sh"]
      args: [ "-c", "while true; do curl -s http://localhost:8080/date;sleep 2;done" ]
```

Проверяем манифест на корректность:

```
kubectl apply --dry-run=client -f pods-2-in-1.yaml
```

```
pod/my-pod created (dry run)
```

Создаём под:

```
kubectl apply -f pods-2-in-1.yaml
```

```
pod/my-pod created
```

Проверяем, что под создался успешно:

```
kubectl get pods -n my-namespace-o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-app	2/2	Running	0	15s	10.244.0.4	combo-1	<none>		<none>	

Смотрим логи контейнера сервера (через -с указав имя контейнера server):

```
# Под сервера
kubectl logs -n my-namespace pods/my-pod -c my-server
```

```
...
127.0.0.1 - - [22/Sep/2025 12:14:44] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:46] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:48] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:50] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:52] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:54] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:56] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:14:58] "GET /date HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2025 12:15:00] "GET /date HTTP/1.1" 200 -
```

Смотрим логи контейнера клиента (через -с указав имя контейнера client):

```
kubectl logs -n my-namespace pods/my-pod -c my-client
```

```
...
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
{"date": "09/22/2025"}
```

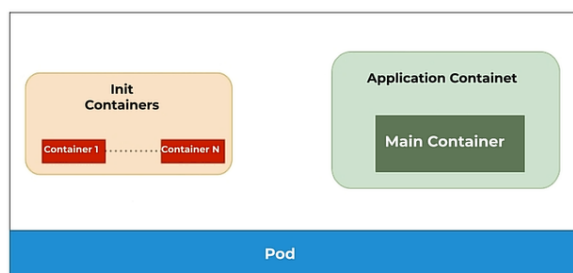
Убиваем под:

```
kubectl delete -n my-namespace pods my-pod
```

```
pod "my-pod" deleted
```

4.5.3 Init Container в поде

Init Conetiner — можно использовать для инициализации Pod, что запускается перед основным приложением (клонирование Git-репозитория, провести миграцию БД, задержка запуска основного контейнера Pod для выполнения определённого условия... например дожидаться пока будет доступна БД). Init Container отличается от обычного, как у Job у них есть определённый объём работы. Их может быть несколько, но они выполняются последовательно и только после того, как предыдущий успешно завершил свою работу. При ошибке он перезапускается до успешного завершения. Так же они могут содержать утилиты и приложения, которых нет в основном приложении. Объявляется о спецификации Pod-а.



Создаём Pod с Init Container:

```
cd ~/kubernetes/example-pods
vim initcontainer.yaml

apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-initcontainer
spec:
  initContainers: # Описывает список Init Container
  - name: my-init-app
    image: busybox:1.37.0
    # Команда ожидания доступности DNS имени сервиса my-db
    command: ['sh', '-c', 'until nslookup my-db.my-namespace.svc.cluster.local; do echo waiting for mydb; sleep 2; done']
  containers: # Основные контейнеры Pod
  - name: my-app
    image: busybox:1.37.0
    command: ['sh', '-c', 'echo The app is running! && sleep 3600']
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f initcontainer.yaml

pod/my-initcontainer created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f initcontainer.yaml

pod/my-initcontainer created
```

Проверяем состояние Pod:

```
kubectl get -n my-namespace pods my-initcontainer -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-initcontainer	0/1	Init:0/1	0	45s	10.244.0.84	combo-1	<none>		<none>	

... и видим, что Init Container не отработал. Проверяем его логи:

```
kubectl logs -n my-namespace pods/my-initcontainer my-init-app

waiting for mydb
Server:      10.96.0.10
Address:     10.96.0.10:53

** server can't find my-db.my-namespace.svc.cluster.local: NXDOMAIN

** server can't find my-db.my-namespace.svc.cluster.local: NXDOMAIN

waiting for mydb
Server:      10.96.0.10
Address:     10.96.0.10:53

** server can't find my-db.my-namespace.svc.cluster.local: NXDOMAIN

** server can't find my-db.my-namespace.svc.cluster.local: NXDOMAIN
```

DNS имя my-db.my-namespace.svc.cluster.local не обнаружено, поэтому Init Container находится в постоянной перезагрузке.

Для прохождения Init Containers создадим сервис my-db (который будет внутренний DNS резолвить и как полный адрес my-db.my-namespace.svc.cluster.local) с указанным именем:

```
kubectl create -n my-namespace service clusterip my-db --tcp=8082:80

service/my-db created
```

Проверяем список событий, которые произошли в Pod, отфильтровав события кластера и видим, что сначала создался Init Container my-init-app, а затем, после создания сервиса запустился контейнер my-app:

```
kubectl get events -n my-namespace --field-selector involvedObject.name=my-initcontainer
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
28m	Normal	Scheduled	pod/my-initcontainer	Successfully assigned my-namespace/my-initcontainer to combo-1
28m	Normal	Pulled	pod/my-initcontainer	Container image "busybox:1.37.0" already present on machine
28m	Normal	Created	pod/my-initcontainer	Created container: my-init-app
28m	Normal	Started	pod/my-initcontainer	Started container my-init-app
9m7s	Normal	Pulled	pod/my-initcontainer	Container image "busybox:1.37.0" already present on machine
9m7s	Normal	Created	pod/my-initcontainer	Created container: my-app
9m7s	Normal	Started	pod/my-initcontainer	Started container my-app

Проверяем, что основной контейнер стартовал, как только завершил работу Init Container:

```
kubectl get -n my-namespace pods my-initcontainer -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-initcontainer	1/1	Running	0	19m	10.244.0.84	combo-1	<none>		<none>	

Завершаем работу запущены объектов:

```
kubectl delete -n my-namespace service/my-db pod/my-initcontainer

service "my-db" deleted
pod "my-initcontainer" deleted
```

4.6 ReplicaSet

Получаем текущую версию API для спецификации ReplicaSet:

```
kubectl explain replicaset | sed '/^$/{Q;}'

GROUP:      apps
KIND:       ReplicaSet
VERSION:    v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-replicaset
```

4.6.1 Простой пример ReplicaSet

Создаём проект для спецификации:

```
cd ~/kubernetes/example-replicaset
vim replicaset.yaml

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  # Имя самого ReplicaSet
  name: my-replicaset
  namespace: my-namespace
  # Лейблы самого ReplicaSet, они не относятся к подам внутри
  labels:
    app: my-app      # Метка для ReplicaSet (не используется селектором)
    rs_cluster: prod # Метка для ReplicaSet (не используется селектором)
spec:
  # Количество подов, которые должен поддерживать ReplicaSet
  replicas: 3
  selector:
    # Только поды с этими лейблами будет искать и контролировать ReplicaSet
    matchLabels:
      app: my-app
  # Шаблон кода, который ReplicaSet будет использовать для создания новых подов
  template:
    metadata:
      # Метки здесь должны совпадать с метками в selector в matchLabels
      labels:
        app: my-app # Метка пода (должна совпадать с селектором)
        role: my-web # Дополнительная метка (не используется селектором)
    spec:
      containers:
        - name: my-server
          image: nginx:1.29.1
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f replicaset.yaml

replicaset.apps/nginx created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f replicaset.yaml

replicaset.apps/my-replicaset created
```

Проверяем результат запуска:

```
kubectl get replicaset.apps -n my-namespace -o wide --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR	LABELS
my-replicaset	3	3	0	44s	my-server	nginx:1.29.1	app=my-app	app=my-app,rs_cluster=prod

Проверяем, что под ReplicaSet запущены Pod-ы:

```
kubectl get pods -n my-namespace -o wide --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES	LABELS
my-replicaset-bpn72	1/1	Running	0	3m16s	10.244.2.7	combo-3	<none>		<none>		app=my-app,role=my-web
my-replicaset-gqbg6	1/1	Running	0	3m16s	10.244.1.14	combo-2	<none>		<none>		app=my-app,role=my-web
my-replicaset-m4q6t	1/1	Running	0	3m16s	10.244.0.4	combo-1	<none>		<none>		app=my-app,role=my-web

Изменяем количество реплик до 5 штук:

```
kubectl scale replicaset -n my-namespace my-replicaset --replicas=5

replicaset.apps/nginx scaled
```

Проверяем результат:

```
kubectl get pods -n my-namespace -o wide --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES	LABELS
my-replicaset-bpn72	1/1	Running	0	3m16s	10.244.2.7	combo-3	<none>		<none>		app=my-app,role=my-web
my-replicaset-gqbg6	1/1	Running	0	3m16s	10.244.1.14	combo-2	<none>		<none>		app=my-app,role=my-web
my-replicaset-knc7q	1/1	Running	0	21s	10.244.1.15	combo-2	<none>		<none>		app=my-app,role=my-web
my-replicaset-m4q6t	1/1	Running	0	3m16s	10.244.0.4	combo-1	<none>		<none>		app=my-app,role=my-web
my-replicaset-msp8p	1/1	Running	0	21s	10.244.2.8	combo-3	<none>		<none>		app=my-app,role=my-web

Проверяем, что у подов есть создатель:

```
kubectl get pods -n my-namespace -o json | jq -r '.items[].metadata.ownerReferences[0]'

{
  "apiVersion": "apps/v1",
  "blockOwnerDeletion": true,
  "controller": true,
  "kind": "ReplicaSet",
  "name": "my-replicaset",
  "uid": "91be8911-ddba-4dfc-8bef-b0446969ed1a"
}
{
  "apiVersion": "apps/v1",
  "blockOwnerDeletion": true,
  "controller": true,
  "kind": "ReplicaSet",
  "name": "my-replicaset",
  "uid": "91be8911-ddba-4dfc-8bef-b0446969ed1a"
}
// ...
```

Удаляем ReplicSet, но оставляем порождённые им поды работать (по умолчанию ReplicaSet удаляет за собой все порождённые им поды; -cascade=orphan означает оставить поды сиротами):

```
kubectl delete replicaset.apps -n my-namespace my-replicaset --cascade=orphan

replicaset.apps "my-replicaset" deleted
```

Проверяем, что поды работают, но у них теперь нет контролирующего их создателя:

```
kubectl get pods -n my-namespace -o json | jq -r '.items[].metadata.ownerReferences[0]'
```

```
null
null
null
null
null
```

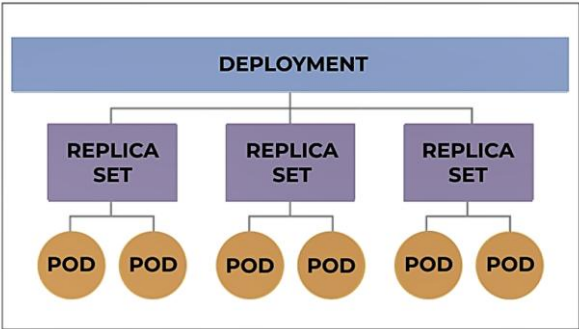
Удаляем поды по именам их лейблов:

```
kubectl delete pods -n my-namespace -l app=my-app
```

```
pod "my-replicaset-bpn72" deleted
pod "my-replicaset-gqbg6" deleted
pod "my-replicaset-knc7q" deleted
pod "my-replicaset-m4q6t" deleted
pod "my-replicaset-msp8p" deleted
```

4.7 Deployment

Deployment — это объект, предназначенный для деплоя и обновления приложения декларативным способом.



Получаем текущую версию API для спецификации Deployment:

```
kubectl explain deployment | sed '/^$/Q;'
```

```
GROUP:      apps
KIND:        Deployment
VERSION:     v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-deployment
```

4.7.1 Простой deploy

Создаём проект для спецификации:

```
cd ~/kubernetes/example-deployment
vim deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  namespace: my-namespace
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-server
          image: nginx:1.29.1
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f deployment.yaml
```

```
deployment.apps/my-deployment created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f deployment.yaml
```

```
deployment.apps/my-deployment created
```

Проверяем результат запуска спецификации Deployment:

```
kubectl get deployments.apps -n my-namespace -o wide --show-labels
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR	LABELS
my-deployment	3/3	3	3	44s	my-server	nginx:1.29.1	app=my-app	<none>

Проверяем результат запуска подчинённой спецификации ReplicaSet:

```
kubectl get replicaset.apps -n my-namespace -o wide --show-labels
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR	LABELS
my-deployment-585bbff568	3	3	3	73s	my-server	nginx:1.29.1	app=my-app, pod-template-hash=585bbff568	app=my-app, pod-template-hash=585bbff568

Проверяем результат запуска подчинённых спецификаций Pod:

```
kubectl get pods -n my-namespace -o wide --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES	LABELS
------	-------	--------	----------	-----	----	------	----------------	-----------------	--------

my-deployment-585bbff568-lk2td	1/1	Running	0	119s	10.244.1.16	combo-2	<none>	<none>	app=my-app,pod-template-hash=585bbff568
my-deployment-585bbff568-vsgbz	1/1	Running	0	119s	10.244.2.9	combo-3	<none>	<none>	app=my-app,pod-template-hash=585bbff568
my-deployment-585bbff568-w5pbj	1/1	Running	0	119s	10.244.0.5	combo-1	<none>	<none>	app=my-app,pod-template-hash=585bbff568

Проверяем статус Deployment через команду rollout:

```
kubectl rollout status deployment -n my-namespace my-deployment
```

deployment "my-deployment" successfully rolled out

Удаляем Deployment:

```
kubectl delete deployments.apps -n my-namespace my-deployment
```

deployment.apps "my-deployment" deleted

4.7.2 Deploy со стратегией

- **RollingUpdate** — удаляет старые модули и одновременно добавляет новые. Если новая версия корректно работает со старой.
- **Recreate** — удаляет все старые модули перед созданием новых. Если новая версия не совместима со старой и нужно полностью остановить старую перед поднятием новой.

На всех рабочих нодах

Создаём спецификацию развёртывания со старой версией пода **k8s.gcr.io/echoserver:1.9**:

```
cd ~/kubernetes/example-deployment
vim deployment-rollingupdate.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
  namespace: my-namespace
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
        # Для теста смены версий
        version: v1
    spec:
      containers:
        - name: my-app
          image: hashicorp/http-echo:1.0.0
          # Поля можно смотреть через `kubectl explain pod` и `kubectl describe pods my-deployment-b7c5ff48d-s7hjr`
          args: ["-text='Pod: ${POD_NAME}, IP: ${POD_IP}'"]
          env:
            - {name: POD_NAME, valueFrom: {fieldRef: {fieldPath: metadata.name}}}
            - {name: POD_IP, valueFrom: {fieldRef: {fieldPath: status.podIP}}}
          ports:
            - containerPort: 5678
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f deployment-rollingupdate.yaml
```

deployment.apps/my-deployment created (dry run)

Запускаем deployment:

```
kubectl apply -f deployment-rollingupdate.yaml
```

deployment.apps/my-deployment created

Проверяем, что запущена версия **v1**:

```
kubectl get pods -n my-namespace --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS	
my-deployment-5f48f48b68-hkfh2	1/1	Running	0	1s	app=my-app,pod-template-hash=5f48f48b68,version=v1	
my-deployment-5f48f48b68-tnb12	1/1	Running	0	1s	app=my-app,pod-template-hash=5f48f48b68,version=v1	
my-deployment-5f48f48b68-l2jc7	1/1	Running	0	1s	app=my-app,pod-template-hash=5f48f48b68,version=v1	

Меняем в файле номер версии на **v2**:

```
vim deployment-rollingupdate.yaml
```

```
# ...
spec:
# ...
  template:
    metadata:
      labels:
        app: my-app
        # Для теста смены версий
        version: v2
# ...
```

Запускаем отслеживание изменений состояний подов в фоне и обновляем спецификацию:

```
kubectl get pods -n my-namespace --show-labels --watch &
kubectl apply -f deployment-rollingupdate.yaml
```

deployment.apps/my-deployment configured

Далее в онлайн будет видно, как создаются и запускаются новые версии и уничтожаются старые:

my-deployment-7b4f7d6455-vxs2v	0/1	Pending	0	0s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-7b4f7d6455-vxs2v	0/1	ContainerCreating	0	0s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-7b4f7d6455-vxs2v	1/1	Running	0	1s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-5f48f48b68-tnb12	1/1	Terminating	0	41s	app=my-app,pod-template-hash=5f48f48b68,version=v1
my-deployment-7b4f7d6455-6vcx9	0/1	Pending	0	0s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-7b4f7d6455-6vcx9	0/1	Pending	0	0s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-7b4f7d6455-6vcx9	0/1	ContainerCreating	0	0s	app=my-app,pod-template-hash=7b4f7d6455,version=v2
my-deployment-5f48f48b68-tnb12	0/1	Completed	0	41s	app=my-app,pod-template-hash=5f48f48b68,version=v1

my-deployment-5f48f48b68-tnb12	0/1	Completed	0	42s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-tnb12	0/1	Completed	0	42s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-7b4f7d6455-6vcx9	1/1	Running	0	1s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-5f48f48b68-l2jc7	1/1	Terminating	0	42s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-7b4f7d6455-nxc6z	0/1	Pending	0	0s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-7b4f7d6455-nxc6z	0/1	Pending	0	0s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-7b4f7d6455-nxc6z	0/1	ContainerCreating	0	0s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-5f48f48b68-l2jc7	0/1	Completed	0	42s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-7b4f7d6455-nxc6z	1/1	Running	0	1s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-7b4f7d6455-hkfh2	1/1	Terminating	0	43s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-hkfh2	0/1	Completed	0	43s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-l2jc7	0/1	Completed	0	43s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-l2jc7	0/1	Completed	0	43s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-hkfh2	0/1	Completed	0	44s	app=my-app, pod-template-hash=5f48f48b68, version=v1
my-deployment-5f48f48b68-hkfh2	0/1	Completed	0	44s	app=my-app, pod-template-hash=5f48f48b68, version=v1

И в результате везде будет версия **v2**:

kubectl get pods -n my-namespace --show-labels					
NAME	READY	STATUS	RESTARTS	AGE	LABELS
my-deployment-7b4f7d6455-n7gvq	1/1	Running	0	6s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-7b4f7d6455-qhsw	1/1	Running	0	9s	app=my-app, pod-template-hash=7b4f7d6455, version=v2
my-deployment-7b4f7d6455-sb8kh	1/1	Running	0	7s	app=my-app, pod-template-hash=7b4f7d6455, version=v2

Смотрим, что с новым ReplicaSet остался предыдущий, который перешёл в выключенное состояние:

kubectl get replicaset.apps -n my-namespace -o wide							
NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
my-deployment-5f48f48b68	0	0	0	3m8s	my-app	registry.k8s.io/e2e-test-images/agnhost:2.47	app=my-app, pod-template-hash=5f48f48b68
my-deployment-7b4f7d6455	3	3	3	2m46s	my-app	registry.k8s.io/e2e-test-images/agnhost:2.47	app=my-app, pod-template-hash=7b4f7d6455

Откатываем обновление на предыдущую версию:

kubectl rollout undo deployment -n my-namespace my-deployment	
deployment.apps/my-deployment rolled back	

Снова проверяем состояние ReplicaSet и видим, что Deployment откатился на предыдущую ReplicaSet:

kubectl get replicaset.apps -n my-namespace -o wide							
NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
my-deployment-5f48f48b68	3	3	3	4m52s	my-app	registry.k8s.io/e2e-test-images/agnhost:2.47	app=my-app, pod-template-hash=5f48f48b68
my-deployment-7b4f7d6455	0	0	0	4m30s	my-app	registry.k8s.io/e2e-test-images/agnhost:2.47	app=my-app, pod-template-hash=7b4f7d6455
# Версия повов:							
kubectl get pods -n my-namespace --show-labels							
NAME	READY	STATUS	RESTARTS	AGE	LABELS		
my-deployment-5f48f48b68-hgpn8	1/1	Running	0	85s	app=my-app, pod-template-hash=5f48f48b68, version=v1		
my-deployment-5f48f48b68-m58qx	1/1	Running	0	87s	app=my-app, pod-template-hash=5f48f48b68, version=v1		
my-deployment-5f48f48b68-pkm5f	1/1	Running	0	86s	app=my-app, pod-template-hash=5f48f48b68, version=v1		
# Состояние развёртывания:							
kubectl rollout status deployment -n my-namespace my-deployment							
deployment "my-deployment" successfully rolled out							

Удаляем задачу отслеживания изменений подов:

kill %1

Удаляем Deployment:

kubectl delete deployments.apps -n my-namespace my-deployment	
deployment.apps "my-deployment" deleted	

4.7.3 Создание через kubectl

Создадим deployment с именем my-deployment через командную строку (имена подов в этом случае указывать нельзя, они будут называться так же, как и deployment).

kubectl create deployment my-deployment --namespace my-namespace --image=nginx:1.29.1 --replicas=3	
deployment.apps/my-deployment created	

Проверяем, что он создан:

kubectl get deployments.apps -n my-namespace -o wide										
NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR			
my-deployment	3/3	3	3	62s	nginx	nginx:1.29.1	app=my-deployment			
kubectl get pods -n my-namespace -o wide										
NAME		READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS GATES
my-deployment-f88f89fdf-4tkdl		1/1	Running	0	92s	10.244.0.66	combo-1	<none>		<none>
my-deployment-f88f89fdf-7ltzw		1/1	Running	0	92s	10.244.1.44	combo-2	<none>		<none>
my-deployment-f88f89fdf-x8lmq		1/1	Running	0	92s	10.244.2.37	combo-3	<none>		<none>

Удаляем созданный deployment:

kubectl delete deployments.apps -n my-namespace my-deployment	
deployment.apps "my-deployment" deleted	

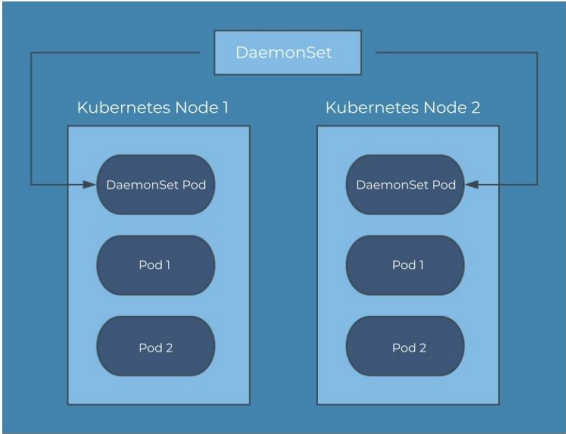
4.7.4 Изменение числа реплик через kubectl

Число реплик можно менять интерактивно, через команды kubectl. Например, для deployment с именем my-deployment, меняем количество реплик до 1-го:

kubectl scale deployment -n my-namespace my-deployment --replicas=1	
deployment.apps/my-deployment scaled	

4.8 DaemonSet

DaemonSet — нужен для запуска приложений, которые должны работать на всех (или выбранных по фильтру) нодах. Типа демона Linux.



Получаем текущую версию API для спецификации DaemonSet:

```
kubectl explain daemonset | sed '/^$/{Q;}'

GROUP:      apps
KIND:       DaemonSet
VERSION:    v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-daemonset
cd ~/kubernetes/example-daemonset
```

Создаём манифест:

```
vim daemonset-node.yaml

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: my-daemonset
  namespace: my-namespace
spec:
  selector:
    matchLabels:
      app: my-daemonset
  template:
    metadata:
      labels:
        app: my-daemonset
    spec:
      hostNetwork: true
      hostPID: true
      hostIPC: true
      containers:
        - name: my-daemonset
          image: prom/node-exporter:v1.9.1
          args:
            - '--path.procfs=/host/proc'
            - '--path.sysfs=/host/sys'
            - '--path.rootfs=/host/root'
            - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($|/)'
          ports:
            - containerPort: 9100
              hostPort: 9100
              protocol: TCP
          volumeMounts:
            - name: proc
              mountPath: /host/proc
              readOnly: true
            - name: sys
              mountPath: /host/sys
              readOnly: true
            - name: root
              mountPath: /host/root
              readOnly: true
      volumes:
        - name: proc
          hostPath:
            path: /proc
        - name: sys
          hostPath:
            path: /sys
        - name: root
          hostPath:
            path: /
      tolerations:
        - operator: Exists
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f daemonset-node.yaml

daemonset.apps/my-daemonset created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f daemonset-node.yaml

daemonset.apps/my-daemonset created
```

Проверяем результат запуска спецификации DaemonSet:

```
kubectl get daemonsets.apps -n my-namespace -o wide
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE	CONTAINERS	IMAGES	SELECTOR
my-daemonset	3	3	0	3	0	<none>	2m6s	my-daemonset	prom/node-exporter:v1.9.1	app=my-daemonset

Проверяем результат запуска подчинённых спецификаций Pod:

<code>kubectl get pods -n my-namespace -o wide</code>										
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES		
my-daemonset-6nhbg	1/1	Running	0	33s	192.168.20.11	combo-1	<none>	<none>		
my-daemonset-8hcmq	1/1	Running	0	33s	192.168.20.13	combo-3	<none>	<none>		
my-daemonset-fwqcr	1/1	Running	0	33s	192.168.20.12	combo-2	<none>	<none>		

Пробрасываем порт с порта :9100 одного из подов на порт :9100 локального хоста для всех интерфейсов (в фоновом режиме):

<code>kubectl port-forward -n my-namespace pods/my-daemonset-6nhbg --address 0.0.0.0 9100:9100 &</code>										
[1] 7917 Forwarding from 0.0.0.0:9100 -> 9100										

Проверяем работоспособность пода:

<code>curl http://127.0.0.1:9100/metrics</code>										
Handling connection for 9100 # HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles. # TYPE go_gc_duration_seconds summary go_gc_duration_seconds{quantile="0"} 1.1043e-05 go_gc_duration_seconds{quantile="0.25"} 1.1043e-05 go_gc_duration_seconds{quantile="0.5"} 1.1043e-05 go_gc_duration_seconds{quantile="0.75"} 1.1043e-05 go_gc_duration_seconds{quantile="1"} 1.1043e-05 go_gc_duration_seconds_sum 1.1043e-05 ...										

... или снаружи <http://gateway.dev.lan:9100/metrics>

Завершаем переадресацию:

<code>kill %1</code>										
----------------------	--	--	--	--	--	--	--	--	--	--

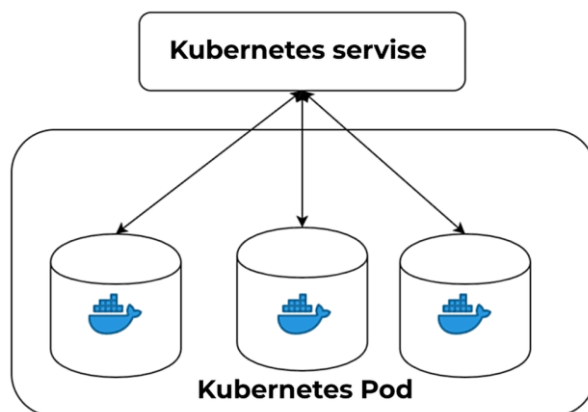
Удаляем DaemonSet:

<code>kubectl delete daemonsets.apps -n my-namespace my-daemonset</code>										
daemonsets.apps "my-daemonset" deleted										

4.9 Service

Service — единая постоянная точка входа для группы подов. IP-адрес и порт не поменяются, пока сервис работает. IP-адрес и порт маршрутизируются к одному из подов. По умолчанию сервис равномерно адресует запросы по всем подам. Подчинённые поды определяются через селекторы. По умолчанию, сервис имеет тип **ClusterIP**.

Service, по факту, является **набором правил iptables**, поэтому его **нельзя пропинговать**.



Получаем текущую версию API для спецификации DaemonSet:

<code>kubectl explain service sed '/^\$/Q;}'</code>										
KIND: Service VERSION: v1										

Подготавливаем директорию для проекта:

<code>mkdir -p ~/kubernetes/example-service</code>										
--	--	--	--	--	--	--	--	--	--	--

Объект конечных точек Endpoints:

<code>kubectl explain endpoints sed '/^\$/Q;}'</code>										
KIND: Endpoints VERSION: v1										

4.9.1 Простой service

Создаём проект для спецификации:

<code>cd ~/kubernetes/example-service</code> <code>vim service.yaml</code>										
<pre> --- apiVersion: apps/v1 kind: Deployment metadata: name: my-deployment </pre>										

```
  namespace: my-namespace
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-server
        image: hashicorp/http-echo:1.0.0
        args: ["-text=Pod: $(POD_NAME), IP: $(POD_IP)"]
        env:
        - {name: POD_NAME, valueFrom: {fieldRef: {fieldPath: metadata.name}}}
        - {name: POD_IP, valueFrom: {fieldRef: {fieldPath: status.podIP}}}
      ports:
      - containerPort: 5678

---
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: my-namespace
spec:
  selector:
    app: my-app
  ports:
  - protocol: TCP
    port: 8082
    targetPort: 5678
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f service.yaml
```

```
deployment.apps/my-deployment created (dry run)
service/my-service created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f service.yaml
```

```
deployment.apps/my-deployment created
service/my-service created
```

Проверяем ресурс Service:

```
kubectl get services -n my-namespace my-service -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
my-service	ClusterIP	10.111.36.162	<none>	8082/TCP	41s	app=my-app

Смотрим подробное описание сервиса:

```
kubectl describe services -n my-namespace my-service
```

```
Name: my-service
Namespace: my-namespace
Labels: <none>
Annotations: <none>
Selector: app=my-app
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.100.142.9
IPs: 10.100.142.9
Port: <unset> 8082/TCP
TargetPort: 5678/TCP
Endpoints: 10.244.0.30:5678,10.244.1.36:5678,10.244.2.30:5678
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
```

Смотрим ресурс Endpoints, созданный сервисом:

```
kubectl get endpoints -n my-namespace my-service
```

NAME	ENDPOINTS	AGE
my-service	10.244.0.30:5678,10.244.1.36:5678,10.244.2.30:5678	45s

Смотрим созданный ресурс Deploy:

```
kubectl get deployments.apps -n my-namespace my-deployment -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
my-deployment	3/3	3	3	5m38s	my-server	hashicorp/http-echo:1.0.0	app=my-app

Смотрим созданные поды:

```
kubectl get pods -n my-namespace -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
my-deployment-848fd47d64-cxxzq	1/1	Running	0	6m7s	10.244.0.30	combo-1	<none>	<none>
my-deployment-848fd47d64-fnknr	1/1	Running	0	6m7s	10.244.2.30	combo-3	<none>	<none>
my-deployment-848fd47d64-qg5l7	1/1	Running	0	6m7s	10.244.1.36	combo-2	<none>	<none>

Создаём временный контейнер с сетевыми утилитами внутри кластера (чтобы не заморачиваться с переедресациями и находиться в пространстве имён сервиса):

```
kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
```

Уже внутри контейнера. Резолвим и определяем полное FQDN имя сервиса и его IP:

```
nslookup ns my-service
```

```
Server: 10.96.0.10
Address: 10.96.0.10:53
```

```
Name:    my-service.my-namespace.svc.cluster.local
Address: 10.105.66.70

** server can't find my-service.svc.cluster.local: NXDOMAIN

** server can't find my-service.svc.cluster.local: NXDOMAIN

** server can't find my-service.cluster.local: NXDOMAIN

** server can't find my-service.cluster.local: NXDOMAIN

** server can't find my-service.dev.lan: NXDOMAIN

** server can't find my-service.dev.lan: NXDOMAIN
```

Получаем данные через сервис у подов:

```
curl http://my-service:8082

# Или по полному имени:
curl http://my-service.my-namespace.svc.cluster.local:8082

Pod: my-deployment-848fd47d64-cxxzq, IP: 10.244.0.30
```

Выходим из контейнера:

```
exit

pod "my-temp" deleted
```

Удаляем все рабочие ресурсы:

```
kubectl delete -f service.yaml

deployment.apps "my-deployment" deleted
service "my-service" deleted
```

4.9.2 Создание через kubectl для Deployment

Создать сервис для deployment, replicaset, replicationcontroller, других service или pods можно через команду из kubectl.

Для примера, создаём deployments и сервис для доступа к его подам:

```
kubectl create deployment my-deployment --namespace my-namespace --image=nginx:1.29.1 --replicas=3

deployment.apps/my-deployment created
```

Проверяем, что deployment он создался:

```
kubectl get deployments.apps -n my-namespace my-deployment -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
my-deployment	3/3	3	3	40s	nginx	nginx:1.29.1	app=my-deployment

Создаём сервис для доступа к подам этого deployment:

```
kubectl expose deployment -n my-namespace my-deployment --port=8082 --target-port=80

service/my-deployment exposed
```

Проверяем, что сервис создался и его описание:

```
kubectl get service -n my-namespace my-deployment -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
my-deployment	ClusterIP	10.109.93.215	<none>	8082/TCP	105s	app=my-deployment

Сервис называется так же как и Deployments — my-deployment:

```
kubectl describe services -n my-namespace my-deployment
```

```
Name:                my-deployment
Namespace:           my-namespace
Labels:               app=my-deployment
Annotations:          <none>
Selector:             app=my-deployment
Type:                 ClusterIP
IP Family Policy:     SingleStack
IP Families:          IPv4
IP:                   10.109.93.215
IPs:                  10.109.93.215
Port:                 <unset> 8082/TCP
TargetPort:           80/TCP
Endpoints:            10.244.0.123:80,10.244.2.57:80,10.244.1.55:80
Session Affinity:     None
Internal Traffic Policy: Cluster
Events:               <none>
```

Проверяем доступ к нему через временный под:

```
kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
```

...уже внутри контейнера, проверяем получение данных через сервис:

```
curl http://my-deployment:8082

<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color:scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
```

```
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

...выходим из контейнера:

```
exit

pod "my-temp" deleted
```

Поды по умолчанию создались на каждой ноде по одному поду. Заходим на первую ноду:

```
ssh combo-1.dev.lan
```

Смотрим в таблице NAT правила маршрутизации для сервисов KUBE-SERVICES:

```
sudo iptables -t nat -n --line-numbers -L KUBE-SERVICES

Chain KUBE-SERVICES (2 references)
num target prot opt source destination
1 KUBE-SVC-FTA7HQTXLRK6JXFI 6 -- 0.0.0.0/0 10.104.153.81 /* kube-system/my-kube-ops-view:http cluster IP */ tcp dpt:80
2 KUBE-SVC-UMU065MPL4DLOUBN 6 -- 0.0.0.0/0 10.109.93.215 /* my-namespace/my-deployment cluster IP */ tcp dpt:8082
3 KUBE-SVC-NPX46M4PTMTKRNGY 6 -- 0.0.0.0/0 10.96.0.1 /* default/kubernetes:https cluster IP */ tcp dpt:443
4 KUBE-SVC-TCOU7JCQKEZGVUNU 17 -- 0.0.0.0/0 10.96.0.10 /* kube-system/kube-dns:dns cluster IP */ udp dpt:53
5 KUBE-SVC-ERIFXISQEP7F7OF4 6 -- 0.0.0.0/0 10.96.0.10 /* kube-system/kube-dns:dns-tcp cluster IP */ tcp dpt:53
6 KUBE-SVC-JD5MR3NA4I4DYORP 6 -- 0.0.0.0/0 10.96.0.10 /* kube-system/kube-dns:metrics cluster IP */ tcp dpt:9153
7 KUBE-NODEPORTS 0 -- 0.0.0.0/0 0.0.0.0/0 /* kubernetes service nodeports; NOTE: this must be the last rule in this chain */ ADDRTYPE match dst-type LOCAL
```

Цепочки KUBE-SVC-* используются как маршрутизаторы и балансировщики нагрузки. Цепочка, отвечающая за сервис имеет в комментарии /* my-namespace/my-deployment cluster IP */ tcp dpt:8082. Просматриваем эту цепочку:

```
sudo iptables -t nat -n --line-numbers -L KUBE-SVC-UMU065MPL4DLOUBN

Chain KUBE-SVC-UMU065MPL4DLOUBN (1 references)
num target prot opt source destination
1 KUBE-MARK-MASQ 6 -- !10.244.0.0/16 10.109.93.215 /* my-namespace/my-deployment cluster IP */ tcp dpt:8082
2 KUBE-SEP-VHFDLAZ02BPRLG6 0 -- 0.0.0.0/0 0.0.0.0/0 /* my-namespace/my-deployment -> 10.244.0.123:80 */ statistic mode random probability 0.3333333349
3 KUBE-SEP-574P5RZFRXGBQYWL 0 -- 0.0.0.0/0 0.0.0.0/0 /* my-namespace/my-deployment -> 10.244.1.55:80 */ statistic mode random probability 0.5000000000
4 KUBE-SEP-B5FT3EDP4WD6CQV6 0 -- 0.0.0.0/0 0.0.0.0/0 /* my-namespace/my-deployment -> 10.244.2.57:80 */
```

Здесь видим, что сервис на этой ноде балансирует трафик между 3-мя Pod-ами. Если random не будет активирован на 1-ых двух нодах, он перейдёт на 3-ю ноду. Т.е. каждая нода имеет 1/3 шанса, что на него пойдёт трафик. Точно такое же правило есть на всех остальных нодах.

Заходим в крайнюю цепочку:

```
sudo iptables -t nat -n --line-numbers -L KUBE-SEP-VHFDLAZ02BPRLG6

Chain KUBE-SEP-574P5RZFRXGBQYWL (1 references)
num target prot opt source destination
1 KUBE-MARK-MASQ 0 -- 10.244.1.55 0.0.0.0/0 /* my-namespace/my-deployment */
2 DNAT 6 -- 0.0.0.0/0 0.0.0.0/0 /* my-namespace/my-deployment */ tcp to:10.244.1.55:80
```

Здесь происходит обычный DNAT, т.е. адрес назначения меняется с IP-адреса сервера на IP-адрес Pod-а.

Выходим с SSH:

```
exit
```

Завершаем работу Deployment и его Service:

```
kubectl delete -n my-namespace deployments.apps/my-deployment services/my-deployment

deployment.apps "my-deployment" deleted
service "my-deployment" deleted
```

4.9.3 Kubeclt port-forward

Доступ к подам через сервисы и трансляция в общую сеть через kubeclt port-forward. Создаём Deployment и Service:

```
kubectl create deployment my-deployment --namespace my-namespace --image=nginx:1.29.1 --replicas=3
kubectl expose deployment -n my-namespace my-deployment --port=8082 --target-port=80
```

Делаем проксирование через kubeclt port-forward, указывая Namespace, имя сервиса, то, чтобы порт был доступен для всех хостов (а не только localhost) и переадресуем его на порт 8084:

```
kubectl port-forward -n my-namespace services/my-deployment --address=0.0.0.0 8084:8082
```

После этого, можно зайти на Pod через Service в браузере хоста:

- http://gateway.dev.lan:8084

Останавливаем Forwarding через Ctrl + C.

Завершаем работы Deployment и Service:

```
kubectl delete -n my-namespace deployments.apps/my-deployment services/my-deployment
```

4.10 Задачи на выполнение

4.10.1 Job

Job — задания, которые должны запуститься один раз и успешно завершить свою работу. Если задача не выполнена, job постарается перезагрузиться в другом поде.

Получаем текущую версию API для спецификации job:

```
kubectl explain job | sed '/^${Q;}'
```

```
GROUP:      batch
KIND:       Job
VERSION:    v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-job
```

4.10.1.1 Простой job

Создаём проект для спецификации:

```
cd ~/kubernetes/example-job
vim job.yaml

apiVersion: batch/v1
kind: Job
metadata:
  name: my-job
  namespace: my-namespace
spec:
  completions: 5 # Количество экземпляров выполняемых job
  parallelism: 5 # Сколько подов с задачей может запуститься одновременно
  backoffLimit: 4 # В случае ошибки, job будет 4 раза перезагружена (6 по умолчанию)
  template:
    spec:
      restartPolicy: Never
      activeDeadlineSeconds: 100 # Тайм аут job, после которого она будет убита
      containers:
        - name: my-app
          image: busybox:1.37.0
          command: ["date"] # Задача - вывести текущую дату
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f job.yaml

job.batch/my-job created (dry run)
```

Запускаем job на исполнение:

```
kubectl apply -f job.yaml

job.batch/my-job created
```

Проверяем состояние jobs и подов, которые он запускал:

```
kubectl get -n my-namespace jobs.batch,pods -o wide
```

NAME	STATUS	COMPLETIONS	DURATION	AGE	CONTAINERS	IMAGES	SELECTOR
job.batch/my-job	Complete	5/5	10s	2m53s	my-app	busybox:1.37.0	batch.kubernetes.io/controller-uid=a51ba0a1-ce3b-4452-a425-5165c50778b4

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod/my-job-hjcmk	0/1	Completed	0	2m53s	10.244.2.39	combo-3	<none>		<none>	
pod/my-job-nzq4k	0/1	Completed	0	2m53s	10.244.0.71	combo-1	<none>		<none>	
pod/my-job-vprnf	0/1	Completed	0	2m53s	10.244.0.72	combo-1	<none>		<none>	
pod/my-job-wkg77	0/1	Completed	0	2m53s	10.244.0.70	combo-1	<none>		<none>	
pod/my-job-xxqz	0/1	Completed	0	2m53s	10.244.1.46	combo-2	<none>		<none>	

Все задачи выполнились. Они не удаляются по умолчанию, чтобы можно было посмотреть логи. Смотрим логи одной из задач:

```
kubectl logs -n my-namespace pods/my-job-hjcmk

Sat Oct 18 09:40:09 UTC 2025
```

Удаляем объект job и все созданные им поды:

```
kubectl delete -f job.yaml

job.batch "my-job" deleted
```

4.10.2 CronJob

CronJob — запуск Job по расписанию. Через указанный интервал будет создаваться спецификация Job, которая будет выполнять указанные Pod. Отработанные Job с их Pod и их логами будут храниться определённое время (по умолчанию 3 последних задачи).

Получаем текущую версию API для спецификации CronJob:

```
kubectl explain cronjob | sed '/^$/Q;}'

GROUP:      batch
KIND:        CronJob
VERSION:     v1
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-cronjob
```

4.10.2.1 Простой CronJob

Создаём проект для спецификации:

```
cd ~/kubernetes/example-cronjob
vim cronjob.yaml

apiVersion: batch/v1
kind: CronJob
metadata:
  namespace: my-namespace
  name: my-cronjob
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: my-app
              image: busybox:1.37.0
              command: ["date"] # Задача - вывести текущую дату
```

Проверяем спецификацию на корректность:

```
kubectl apply --dry-run=client -f cronjob.yaml

cronjob.batch/my-cronjob created (dry run)
```

Запускаем спецификацию:

```
kubectl apply -f cronjob.yaml

cronjob.batch/my-cronjob created
```

Проверяем состояние задачи:

```
kubectl get -n my-namespace cronjobs.batch, jobs.batch, pods -o wide
```

NAME	SCHEDULE	TIMEZONE	SUSPEND	ACTIVE	LAST	SCHEDULE	AGE	CONTAINERS	IMAGES	SELECTOR
cronjob.batch/my-cronjob	*/* * * * *	<none>	False	0	15s		2m27s	my-app	busybox:1.37.0	<none>
NAME	STATUS	COMPLETIONS	DURATION	AGE	CONTAINERS	IMAGES	SELECTOR			
job.batch/my-cronjob-29346670	Complete	1/1	4s	2m15s	my-app	busybox:1.37.0	batch.kubernetes.io/controller-uid=4c0683db-cd5a-...			
job.batch/my-cronjob-29346671	Complete	1/1	2s	75s	my-app	busybox:1.37.0	batch.kubernetes.io/controller-uid=5bc99ca8-0df0-...			
job.batch/my-cronjob-29346672	Complete	1/1	3s	15s	my-app	busybox:1.37.0	batch.kubernetes.io/controller-uid=0114b100-2632-...			
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod/my-cronjob-29346670-zwthk	0/1	Completed	0	2m14s	10.244.0.73	combo-1	<none>		<none>	
pod/my-cronjob-29346671-jmn9t	0/1	Completed	0	75s	10.244.0.74	combo-1	<none>		<none>	
pod/my-cronjob-29346672-szvcv	0/1	Completed	0	15s	10.244.0.75	combo-1	<none>		<none>	

Смотрим лог одного из отработанных Pod:

```
kubectl logs -n my-namespace pods/my-cronjob-29346670-zwthk

Sat Oct 18 15:17:00 UTC 2025
```

По завершении, удаляем задачу. После удаления основного CronJob удаляются все отработанные под ним Job и Pod объекты:

```
kubectl delete -f cronjob.yaml

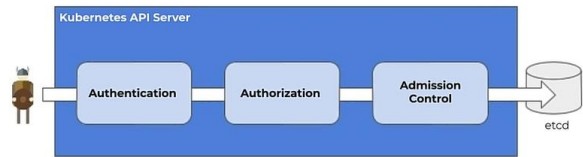
cronjob.batch "my-cronjob" deleted
```

4.11 Admission Controller

При входе пользователя, он аутентифицируется (имеет ли пользователя доступ) и авторизуется (имеет ли пользователь право) на API-сервере. После этого вступает в работу Admission Controller.

Admission Controller — часть API-сервера, которые выполняет какие либо действия после авторизации пользователя (более тонкие права на использование кластера) и является последним оплотом контроля перед доступом к ETCD.

Admission Control



Вот примеры таких контроллеров:

- InitialResources — устанавливает лимиты по умолчанию для ресурсов контейнера
- LimitRanger — устанавливаем значения по умолчанию для запросов и лимитов контейнеров
- ResourcesQuota — считает количество объектов и общие потребляемые ресурсы и предотвращает их превышение
- Специальные контроллеры запросов на сторонние сервисы (приложения в кластере, на которые делаются POST запросы) для валидации и модификации. Работают только по HTTPS:
 - MutatingAdmissionWebhook — модифицирует запрос (выполняется первым)
 - ValidatingAdmissionWebhook — валидирует запрос (выполняется вторым)

Эти контроллеры могут быть не включены по умолчанию.

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-admission
```

4.11.1 ValidatingAdmissionWebhook

Переходим в папку проекта:

```
cd ~/kubernetes/example-admission
```

Генерируем через OpenSSL самоподписанный TLS сертификат (т.к. общение между компонентами будет по HTTPS связи). Для этого создаём конфигурационный файл для OpenSSL:

```
vim openssl.cnf

# Начало секции [req] — основные параметры запроса на сертификат
[req]
# Указывает, что для формирования distinguished_name (DN) используется секция req_distinguished_name
distinguished_name = req_distinguished_name
# Указывает, что для расширений сертификата X.509 используется секция v3_ext
x509_extensions = v3_ext
# Отключает интерактивные запросы (например, ввод данных вручную)
prompt = no

# Начало секции [req_distinguished_name] — параметры для distinguished_name
[req_distinguished_name]
# Устанавливает Common Name (CN) для сертификата — имя сервиса admission-webhook.
# CN (Common Name) — основное имя, по которому идентифицируется сертификат.
CN = my-service.my-namespace.svc

# Начало секции [v3_ext] — расширения для сертификата X.509 версии 3
[v3_ext]
# Указывает, что сертификат не является сертификатом Центра Сертификации (CA)
basicConstraints = CA:FALSE
# Определяет разрешенные способы использования ключа: цифровая подпись и шифрование ключей
```

```
keyUsage = digitalSignature, keyEncipherment
# Определяет расширенные способы использования ключа: аутентификация сервера
extendedKeyUsage = serverAuth
# Указывает, что альтернативные имена субъекта (SAN) берутся из секции alt_names
subjectAltName = @alt_names

# Начало секции [alt_names] – альтернативные имена субъекта (Subject Alternative Names)
[alt_names]
# Первое альтернативное DNS-имя для сертификата (будет вызвано Web Hook-ом)
DNS.1 = my-service.my-namespace.svc
# Второе, полное альтернативное DNS-имя для сертификата (полный адрес сервиса)
DNS.2 = my-service.my-namespace.svc.cluster.local
```

Генерируем ключ `tls.key` и сертификат `tls.crt`:

```
# Генерируем приватный ключ в файл tls.key
openssl genrsa -out tls.key 2048

# Генерируем самоподписанный сертификат в файл tls.crt
openssl req -x509 -new -nodes \
-key tls.key \
-days 3650 \
-out tls.crt \
-config openssl.cnf \
-extensions v3_ext
```

Просматриваем описание созданного сертификата `tls.crt`:

```
openssl x509 -in tls.crt -text -noout

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            23:f9:f0:4a:8f:a8:82:7a:a8:44:51:8e:e6:08:08:82:09:6f:a0:1c
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = my-service.my-namespace.svc
        Validity
            Not Before: Oct 19 08:03:48 2025 GMT
            Not After : Oct 17 08:03:48 2035 GMT
        Subject: CN = my-service.my-namespace.svc
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:be:2a:ae:a2:17:09:a2:93:86:2e:18:2f:2f:24:
                ..
                cd:17:0e:be:af:b6:87:77:9a:98:a5:f4:13:99:e7:
                6e:63
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Key Usage:
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Alternative Name:
                DNS:my-service.my-namespace.svc, DNS:my-service.my-namespace.svc.cluster.local
            X509v3 Subject Key Identifier:
                AA:91:20:55:6E:41:73:E5:8A:03:8D:3D:1C:38:C6:70:3C:40:FA:AF
        Signature Algorithm: sha256WithRSAEncryption
        Signature Value:
            37:09:9d:67:d3:ae:a2:2c:28:b7:66:dd:4c:a2:6a:19:d2:86:
            ...
            46:2e:fc:9f:fc:11:f5:78:fd:6a:41:72:50:15:b5:cf:c4:08:
            9f:8d:bc:6a
```

Просматриваем описание созданного приватного ключа `tls.key`:

```
openssl rsa -in tls.key -text -noout

Private-Key: (2048 bit, 2 primes)
modulus:
    00:be:2a:ae:a2:17:09:a2:93:86:2e:18:2f:2f:24:
    ...
```

Создаём программу для валидации, которая возвращает в JSON ответ `true`, если лейбл `owner` указан и `false`, если нет:

```
vim vaw.py

from flask import Flask, request, jsonify

admission_controller = Flask(__name__)

@admission_controller.route("/validate/deployments", methods=["POST"])
def deployment_webhook():
    request_info = request.get_json()
    if request_info["request"]["object"]["metadata"]["labels"].get("owner"):
        return admission_response(request_info, True, "Owner label exist")
    return admission_response(request_info, False, "Not allowed without owner label")

def admission_response(request_info, allowed, message):
    return jsonify({
        "apiVersion": request_info["apiVersion"], "kind": request_info["kind"],
        "response": {"uid": request_info["request"]["uid"], "allowed": allowed, "status": {"message": message}},
    })

if __name__ == '__main__':
    admission_controller.run(host='0.0.0.0', port=8001, ssl_context=("/app/tls.crt", "/app/tls.key"))
```

Создаём файлы зависимостей для Python:

```
vim requirements.txt

Flask
jsonify
```

Создаём Dockerfile для запуска контейнера:

```
vim Dockerfile
```

```
FROM python:3.12.11-alpine3.22

# Устанавливаем зависимости системы и настраиваем кэш pip
ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=off \
    PIP_DEFAULT_TIMEOUT=100 \
    PIP_DISABLE_PIP_VERSION_CHECK=on

# Устанавливаем рабочую директорию
WORKDIR /app

# Копируем только requirements.txt и устанавливаем зависимости
# Это позволяет использовать кэш Docker на этом этапе, если requirements.txt не изменился
COPY requirements.txt .

# Обновляем pip и устанавливаем зависимости в один слой
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt && \
    rm -rf /tmp/* /var/tmp/* /usr/local/lib/python*/__pycache__

# Копируем остальные файлы
COPY tls.crt tls.key .
COPY vaw.py .

# Запускаем приложение
CMD ["python3", "vaw.py"]
```

Собираем контейнер:

```
| docker buildx build -t server-vaw:1.0 .
```

Проверяем, что контейнер создан:

```
| docker images | grep server-vaw
```

server-vaw	1.0	e13664c7be93	14 seconds ago	63.9MB
------------	-----	--------------	----------------	--------

Проверяем, что контейнер запускается (и выходим из него по Ctrl+C):

```
| docker run --rm -it server-vaw:1.0
```

```
* Serving Flask app 'vaw'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on https://127.0.0.1:8001
* Running on https://172.17.0.2:8001
Press CTRL+C to quit
```

Переносим созданный образ с gateway в локальные Docker нод combo-1,2,3:

```
# Экспортируем контейнер в сжатый файл:
docker save server-vaw:1.0 | gzip > server-vaw-1.0.tar.gz

for host in combo-1.dev.lan combo-2.dev.lan combo-3.dev.lan; do
    # Удаляем старый образ, если он есть
    ssh $host "docker rmi server-vaw:1.0 2>/dev/null || true"
    # Копируем сжатый контейнер на ноды в папку /tmp
    scp server-vaw-1.0.tar.gz $host:/tmp/
    # Запускаем процесс импорта образа в локальный Docker
    ssh $host "gunzip -c /tmp/server-vaw-1.0.tar.gz | docker load"
    # Удаляем старый файл
    ssh $host "rm /tmp/server-vaw-1.0.tar.gz"
done

rm server-vaw-1.0.tar.gz
```

Создаём сервис аккаунты для того, чтобы запустить созданный образ с возможностью взаимодействия с API-сервером. Для этого смотрим версии манифестов:

```
| kubectl explain serviceaccount | sed '/^$/{Q;}'
```

```
KIND:      ServiceAccount
VERSION:   v1
```

```
| kubectl explain clusterrole | sed '/^$/{Q;}'
```

```
GROUP:     rbac.authorization.k8s.io
KIND:      ClusterRole
VERSION:   v1
```

```
| kubectl explain clusterrolebinding | sed '/^$/{Q;}'
```

```
GROUP:     rbac.authorization.k8s.io
KIND:      ClusterRoleBinding
VERSION:   v1
```

Собираем манифест:

```
| vim rbac.yaml
```

```
# ServiceAccount для webhook-контроллера – идентичность в кластере
apiVersion: v1
kind: ServiceAccount
metadata:
  namespace: my-namespace           # Пространство имён, где работает контроллер
  name: my-admission-vaw           # Имя SA, на которое будут ссылаться Pod и RBAC
---

# ClusterRole – глобальные права на чтение узлов и подов (не привязан к namespace)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: my-node-pod-reader           # Уникальное имя роли в кластере
rules:
  - apiGroups: [""]
    resources: ["nodes", "nodes/status", "nodes/metrics"] # "" = core API group (Pods, Nodes и т.д.) # Доступ к информации об узлах
    verbs: ["get", "list", "watch"] # Только чтение
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"] # Доступ к подам во всём кластере # Только чтение
```

```
---

# ClusterRoleBinding — привязка SA к ClusterRole на уровне всего кластера
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-get-nodes # Уникальное имя привязки
subjects:
- kind: ServiceAccount
  name: my-admission-vaw # Кому даём права
  namespace: my-namespace # Где находится этот ServiceAccount
roleRef:
  kind: ClusterRole
  name: my-node-pod-reader # Какую роль назначаем
  apiGroup: rbac.authorization.k8s.io # Группа API для RBAC-ресурсов
```

Применяем спецификацию:

```
kubectl apply -f rbac.yaml

serviceaccount/my-admission-vaw created
clusterrole.rbac.authorization.k8s.io/my-node-pod-reader created
clusterrolebinding.rbac.authorization.k8s.io/my-get-nodes created
```

Проверяем, что все объекты созданы:

```
kubectl get -n my-namespace serviceaccounts/my-admission-vaw clusterrole/my-node-pod-reader clusterrolebindings/my-get-nodes -o wide
```

NAME	SECRETS	AGE
serviceaccount/my-admission-vaw	0	15h

NAME	CREATED AT
clusterrole.rbac.authorization.k8s.io/my-node-pod-reader	2025-10-18T17:24:13Z

NAME	ROLE	AGE	USERS	GROUPS	SERVICEACCOUNTS
clusterrolebinding.rbac.authorization.k8s.io/my-get-nodes	ClusterRole/my-node-pod-reader	15h			my-namespace/my-admission-vaw

Создаём спецификацию Deployment:

```
vim vaw-deployment.yaml

# Deployment — управляет жизненным циклом Pod'ов (реплики, обновления и т.д.)
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: my-namespace # Пространство имён, где будет запущен контроллер
  name: my-deployment # Имя Deployment'a
spec:
  replicas: 1 # Запускать ровно 1 экземпляр Pod'a (обычно для webhook — 1)
  selector:
    matchLabels:
      app: my-app # Как Deployment находит свои Pod'ы (должно совпадать с template.labels)
  template:
    metadata:
      labels:
        app: my-app # Метка на Pod'e — для селектора и Service
    spec:
      serviceAccountName: my-admission-vaw # Использовать этот ServiceAccount (для RBAC-доступа)
      containers:
      - name: my-app # Имя контейнера внутри Pod'a
        image: server-vaw:1.0 # Локальный образ (должен быть доступен в ноде или загружен в кластер)
        ports:
        - containerPort: 8001 # Порт, который слушает приложение (для документации и Service)
```

Применяем спецификацию:

```
kubectl apply -f vaw-deployment.yaml

deployment.apps/my-deployment created
```

Проверяем, что Deployment и его Pod работают:

```
kubectl get -n my-namespace deployments.apps,pods -o wide
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
deployment.apps/my-deployment	1/1	1	1	5s	my-app	server-vaw:1.0	app=my-app

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod/my-deployment-7fd9bc55c4-wr5ff	1/1	Running	0	5s	10.244.0.87	combo-1	<none>		<none>	

Для того, чтобы Pod был доступен другим объектам kubernetes, для него создаём сервис:

```
vim vaw-service.yaml

# Service — обеспечивает стабильный сетевой доступ к Pod'ам (DNS + балансировка)
apiVersion: v1
kind: Service
metadata:
  namespace: my-namespace # Пространство имён, где создаётся сервис
  name: my-service # Имя сервиса (используется в DNS: my-service.my-namespace.svc)
spec:
  selector:
    app: my-app # Выбирает Pod'ы с такой меткой (должна совпадать с labels в Deployment)
  ports:
  - port: 8001 # Порт сервиса (внутри кластера: my-service:8001)
    targetPort: 8001 # Порт в контейнере Pod'a, куда перенаправлять трафик
```

Применяем спецификацию:

```
kubectl apply -f vaw-service.yaml

service/my-service created
```

Проверяем работу Service. Мы должны увидеть IP-адрес сервиса и что Endpoints сервиса указывает на IP-адрес Pod-a:

```
kubectl get -n my-namespace pods -o wide && echo && kubectl get -n my-namespace endpoints -o wide && echo && kubectl get -n my-namespace services
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-deployment-7fd9bc55c4-wr5ff	1/1	Running	0	24m	10.244.0.87	combo-1	<none>		<none>	

NAME	ENDPOINTS	AGE
my-service	10.244.0.87:8001	5m5s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
------	------	------------	-------------	---------	-----

my-service	ClusterIP	10.108.144.222	<none>	8001/TCP	5m5s
------------	-----------	----------------	--------	----------	------

Запускаем тестовый образ в среде Kubernetes, чтобы проверить, что сертификаты созданы правильно и по ним можно обращаться к внутренним объектам. Для этого запускаем контейнер curl в фоновом режиме (в конце знак &):

```
kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh &
```

Возможно появится надпись If you don't see a command prompt, try pressing enter. Просто нажать ENTER и не обращать внимание на надпись [1]+ Stopped kubectl run my-temp -it --rm --restart=Never ...

Копируем в запущенный Pod сертификат TLS в папку /tmp, для проверки его валидности:

```
kubectl cp ./tls.crt my-namespace/my-temp:/tmp
```

Переходим в фоновую задачу командой:

```
fg %1
```

Оказавшись внутри временного Pod-a, запускаем команду на проверку корректности сертификата:

```
curl --cacert /tmp/tls.crt https://my-service.my-namespace.svc:8001/validate/deployments
```

```
<!doctype html>
<html lang=en>
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

Хоть и вернулась ошибка 405, но то, что она показала говорит о том, что сертификат подтверждён временным Pod-от от тестируемого Pod-a. Выходим из временного Pod-a:

```
exit

pod "my-temp" deleted
```

Конфигурируем Kubernetes, чтобы все валидационные данные проходили через созданный сервис. Для этого создаём объект ValidatingWebhookConfiguration:

```
kubectl explain validatingwebhookconfiguration | sed '/^$/;Q;'
```

```
GROUP:      admissionregistration.k8s.io
KIND:       ValidatingWebhookConfiguration
VERSION:    v1
```

Обрабатываем через Base64 сертификат tls.crt. Его нужно будет вставить в параметр caBundle манифеста:

```
base64 -w 0 tls.crt && echo
# -w 0 - сделать всё одной строкой
```

Создаём манифест ValidatingWebhookConfiguration:

```
vim vwc.yaml

apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata: # Не привязан к namespace, т.к. это кластерный ресурс
  name: my-validating-webhook # Уникальное имя webhook-конфигурации в кластере
webhooks:
- name: my-webhook.cluster.loc # Должно быть валидным доменом (обычно третий уровень, например: webhook.example.com)
  failurePolicy: Fail # Если webhook недоступен – отклонить запрос (альтернатива: Ignore)
  sideEffects: None # Указывает, что webhook не имеет побочных эффектов (важно для dry-run)
  admissionReviewVersions: # Поддерживаемые версии AdmissionReview
    - "v1" # Используется в современных кластерах (1.22+)
  clientConfig:
    # Kubernetes будет отправлять POST-запрос на:
    # https://my-service.my-namespace.svc:8001/validate/deployments?timeout=10s
    service:
      name: my-service # Имя Service, через который доступен webhook
      namespace: my-namespace # Namespace, где находится Service
      path: /validate/deployments # HTTP-путь в webhook-сервере
      port: 8001 # Порт Service (должен совпадать с портом в Service)
      # Base64-кодированный CA-сертификат для проверки TLS (обычно = tls.crt при самоподписи)
    caBundle:
LS0tLS1CRUdJTiBDRVJUSUJZQ0FURS0tLS0tCk1JSURmVENDQW1XZ0F3SUJBZ0lVR2RTd0s0SU5KWm9jWEZmMUNsdKhsSTA3eLRrd0RRWUpLb1pJaHZjTkFRRUwKQlFBd0pqrWtNQ0lHQTFV
VRUF3ZDJiWGt0YzJwZWRTbGpauZV0ZmVxdVlXMWxjM0JoTjVdZWZmZmpNQjRyRFRJMQpNVEF4T1RFek16Wxp0bG9YRFRNMU1UQXh0ekV6T0pZek5sb3dkakVrTUNJR0ExVUVBd3diYlhrdG
MyVnIkbWxqcCltNXRlUzF1WVcxbGMzQmhZMlV1YzNaak1JSUJJaF0QmdrcWlhraUc5dzBCQVFFRkFBT0NBUThBTUJlJ0kNnS0MKQVFFQXA5c25lZUM5MXYydhZZTVlZSxcxMudKS2IyeEtnZ
zlpaw9aMGczCtLeEJwazQ4U2VqYkxBd0dMV2lVVQ0bUlldmtMWE0QVBNWE84b1pra3ZKT0NsS3oySFA3ekRwLzhKb3phQVBPZLzQ3TXdIc1Y0b1ZhTzlsawtNQXB0CkloaFFVNUeZcvll
em42SKwy0EZ1K1FPwFVpaXhkZkt5ZkUry1cVwGU3R2YrZ2t0SGpdTEYVvZjJWNNVZUxNdLUKWERQNKdNSEY5RGhQTzhCQmZHaGVOegtTawtOQkQ2aVVIYmRMQ1VZaTREeVBQNUK0N2VsUlp
uT2doTz13TzV0MAppMk1tMjBTWC9hRELra0RrVE9mMUNrZnRMY2dwbWZrM1huVkf3eEVIwm90dGp0WkLMNmprQVZUcmZkaCtjcEMvCmdjbjVVKQXYxbFkwYmgxMUJTamRNNDBejBRSURBUU
FCbzRHaU1JR2ZmZjZEV3ZEV3UUNQUF3Q3dZRFZSMFAKQkFRREFnV2dNQk1HQTFVZEpRUU1NQW9HQ0NzR0FRVUZCd01CTUZFR0ExVWRFVUVJLTUVpQ0cyMTVMWE5sY25acApZMlV1Ylhrd
GJTrnRawE53WdV0bExuTjJZNElWYlhrdGMyVnIkbWxqWlM1dGVMTXVZVzFsYzNCaFkyVXVjM1pqCkxtTnNkWE4wWlhJdWJH0WpZV3d3SFFZRFZSMSE9CQlFRKJLRnVTRw1CU1VXMMwQ5UTV1
UUpwTDlwMC9rcK1BMECKQ1NXR1NJYjNEUUVcQ3dVQUE0SUJBUUJQSFlSb3J4aTBLEZSb0xNckJmS1JhaXh4eXZZaDR2akFBUDZ6VDIwIwBwPjRwTnTCtZrNF3T0g1dEY1VTN6cwY2TetzaEh
uRDZuUG5HUW5JLys2YVFeC9WNFJWeVcxMS90ZlRSBwLWGEs5CLmwaUvKam91aVFLM3lGQXR3R2lUQ245eEx3WlVtZGJTdUREbHU5L2dWSpTL2NrNFFYWHJjbWZhaTN2ZlYvMncKQ2NSbF
FVT1dwdU5KRE9kVHZIdG3UedhTWfjbE8aXlyk2hLZG12Y1o3eE1oSU5qbs9YVRL0XlWYVBaLZFiYgpjNG5XumdvdwLiN1I5Uzh4SHBXR1dDWHdHMGpQcUv0TVVJdmxDOEJzV2xQakFFZ
EdESLVZdi85WVQzY0J6d2R1Cm43CXJNUFc0M1dpQU0Umgxb09IVGsvWUVWSXVWM04ZMG91U1Nda2I3bFVICi0tLS0tRU5EIEFNUlRJRkldQVRFSL0tLS0t
rules:
- apiGroups: # Группа API, к которой применяется правило
  - "apps" # "apps" – группа для Deployment, StatefulSet и т.д.
  resources: # Ресурсы, на которые срабатывает webhook
  - "deployments"
  apiVersions: # Версии ресурсов (звездочка = любая)
  - "*"
  operations: # Операции, которые триггерят webhook
  - CREATE # Только при создании Deployment'a
```

Применяем манифест:

```
kubectl apply -f vwc.yaml

validatingwebhookconfiguration.admissionregistration.k8s.io/my-validating-webhook created
```

Проверяем созданный объект:

```
kubectl get -n my-namespace validatingwebhookconfigurations/my-validating-webhook
```

NAME	WEBHOOKS	AGE
my-validating-webhook	1	41s

Теперь создадим тестовый Deployment, который не будет запустятся, т.е. у него нет метки owner:

```
vim vaw-test-deployment.yaml
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: my-namespace
  name: my-test-deployment
  labels:
    app: my-test-deployment
spec:
  selector:
    matchLabels:
      app: my-test-deployment
  template:
    metadata:
      labels:
        app: my-test-deployment
    spec:
      containers:
        - name: my-nginx
          image: nginx:1.28.0-alpine3.21
          ports:
            - name: http
              containerPort: 80

```

Создаём объект (объект не будет создан, т.к. отсутствует лейбл `owner`):

```

| kubectl apply -f vaw-test-deployment.yaml

```

```

Error from server: error when creating "vaw-test-deployment.yaml": admission webhook "my-webhook.cluster.loc" denied the request: Not allowed without owner label

```

Добавляем лейбл `owner` в тот же манифест:

```

| vim vaw-test-deployment.yaml

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: my-namespace
  name: my-test-deployment
  labels:
    app: my-test-deployment
    owner: "Ivan" # <-- добавленный лейбл
spec:
  # ... осталось неизменным ...

```

Повторно пробуем создать объект (объект успешно создан):

```

| kubectl apply -f vaw-test-deployment.yaml

```

```

deployment.apps/my-test-deployment created

```

Всё работает.

После работы, удаляем все созданные ресурсы.

Удаляем тестовый Deployment:

```

| kubectl delete -f vaw-test-deployment.yaml
| kubectl delete -f vwc.yaml
| kubectl delete -f vaw-service.yaml
| kubectl delete -f vaw-deployment.yaml
| kubectl delete -f rbac.yaml

```

Ожидаем, пока все Pod-ы не будут полностью уничтожены (иначе не удастся удалить образы контейнера `server-vaw:1.0` на них):

```

| kubectl get -n my-namespace pods

```

```

No resources found in my-namespace namespace.

```

Удаляем созданный Docker образ на всех нодах:

```

| for host in combo-1.dev.lan combo-2.dev.lan combo-3.dev.lan; do
  # Удаляем старый образ, если он есть
  ssh $host "docker rmi server-vaw:1.0 2>/dev/null || true"
done

```

4.12 Передача данных в Pod

Подготавливаем директорию для проекта:

```

| mkdir -p ~/kubernetes/example-data

```

4.12.1 ENV

Проверяем версию спецификации для Pod и Service:

```

| kubectl explain pod | grep -iE '^(group|version)'

```

```

VERSION:    v1

```

```

| kubectl explain service | grep -iE '^(group|version)'

```

```

VERSION:    v1

```

Создаём проект для спецификации:

```

| cd ~/kubernetes/example-data
| vim env.yaml

```

```

apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: hashicorp/http-echo:1.0.0

```

```
args: ["-text=DEMO_GREETING: ${DEMO_GREETING}"]
# передача данных в под через переменную окружения:
env:
  - name: DEMO_GREETING
    value: "Hy all!"
ports:
  - containerPort: 5678

---

apiVersion: v1
kind: Service
metadata:
  namespace: my-namespace
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678
```

Применяем спецификацию:

```
kubectl apply -f env.yaml
```

```
pod/my-pod created
service/my-service created
```

Проверяем, что Pod и Service запустились:

```
kubectl get -n my-namespace pods,Endpoints,Service -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
pod/my-pod	1/1	Running	0	64s	10.244.0.102	combo-1	<none>		<none>	
NAME	ENDPOINTS		AGE							
Endpoints/my-service	10.244.0.102:5678		64s							
NAME	TYPE		CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR			
Service/my-service	ClusterIP		10.96.49.218	<none>	80/TCP	64s	app=my-app			

Запускаем внутри kubernetes тестовый Pod, с которого запросим информацию через Service с Pod-a:

```
kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
```

```
If you don't see a command prompt, try pressing enter.
/ #
```

Внутри контейнера запускаем:

```
curl http://my-service
```

```
DEMO_GREETING: Hy all!
```

Таким образом, через ENV мы передали данные в контейнер.

Завершаем работу всех объектов:

```
kubectl delete -f env.yaml
```

```
pod "my-pod" deleted
service "my-service" deleted
```

4.12.2 ConfigMap

Проверяем версию спецификации:

```
kubectl explain configmap | grep -iE '^(group|version)'
```

```
VERSION:      v1
```

4.12.2.1 Простейший ConfigMap

Создаём простейшую спецификацию:

```
vim configmap-sample.yaml
```

```
---
# ConfigMap, который будет подключён к Pod
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: my-namespace
  name: my-configmap
data:
  MY_DATA_1: "This MY_DATA_1"
  MY_DATA_2: "This MY_DATA_2"

---
# Pod, который получает данные из ConfigMap
apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: hashicorp/http-echo:1.0.0
      args: ["-text=MY_DATA_1: ${MY_DATA_1}, MY_DATA_2: ${MY_DATA_2}, MY_DATA_LOADED: ${MY_DATA_LOADED}"]
      # Получаем весь ConfigMap
      envFrom:
        - configMapRef:
            name: my-configmap
      # Получаем единичным способом переменную с её переименованием:
      env:
```

```

- name: MY_DATA_LOADED # Имя переменной, как она будет видна в Pod
  valueFrom:
    configMapKeyRef:
      name: my-configmap
      key: MY_DATA_1 # Имя переменной из ConfigMap
ports:
- containerPort: 5678
---
# Service, через который будем подключаться к Pod
apiVersion: v1
kind: Service
metadata:
  namespace: my-namespace
  name: my-service
spec:
  selector:
    app: my-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 5678

```

Применяем спецификацию:

```

| kubectl apply -f configmap-sample.yaml
-----
configmap/my-configmap created
pod/my-pod created
service/my-service created

```

Проверяем, что объекты загружены:

```

| kubectl get -n my-namespace configmaps,pods,endpoints,services
-----
NAME                                DATA  AGE
configmap/my-configmap              2      48s

NAME    READY   STATUS    RESTARTS   AGE
pod/my-pod  1/1     Running    0          48s

NAME                                ENDPOINTS              AGE
endpoints/my-service              10.244.0.104:5678      47s

NAME    TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
service/my-service  ClusterIP    10.100.205.102 <none>       80/TCP     47s

```

Запускаем внутри Kubernetes тестовый Pod, с которого запросим информацию через Service с Pod-a:

```

| kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
-----
If you don't see a command prompt, try pressing enter.
/ #

```

Внутри контейнера запускаем:

```

| curl http://my-service
-----
MY_DATA_1: This MY_DATA_1, MY_DATA_2: This MY_DATA_2, MY_DATA_LOADED: This MY_DATA_1

```

Таким образом, через `envFrom` и `env` (с `configMapKeyRef`) мы передали данные в контейнер.

Завершаем работу всех объектов:

```

| kubectl delete -f configmap-sample.yaml

```

4.12.3 Secret

Secret — используется для хранения секретных данных.

Версия манифеста:

```

| kubectl explain secret | grep -iE '^(group|version)'
-----
VERSION:      v1

```

4.12.3.1 Простейший Secret

Похож на ConfigMap, но данные должны быть закодированы в Base64. Может быть нескольких типов (<https://kubernetes.io/docs/concepts/configuration/secret/#secret-types>):

- `opaque` — произвольные пользовательские данные
- `kubernetes.io/service-account-token` — ServiceAccount token
- `kubernetes.io/dockercfg` — serialized `~/.dockercfg` file, логин и пароль доступа к Docker Registry
- `kubernetes.io/dockerconfigjson` — serialized `~/.docker/config.json` file
- `kubernetes.io/basic-auth` — учетные данные для базовой аутентификации
- `kubernetes.io/ssh-auth` — учетные данные для аутентификации SSH
- `kubernetes.io/tls` — данные для клиента или сервера TLS
- `bootstrap.kubernetes.io/token` — данные токена начальной загрузки

Кодируем данные и подставляем их в манифест:

```

| echo "This MY_DATA_1" | base64 && \
  echo "This MY_DATA_2" | base64
-----
VGhpY3YBNWV9EQVRBZXZEk
VGhpY3YBNWV9EQVRBZXZIK

```

Создаём манифест:

```

| vim secret-sample.yaml
-----
---

```

```

# Secret, который будет подключён к Pod
apiVersion: v1
kind: Secret
metadata:
  namespace: my-namespace
  name: my-secret
type: Opaque
data:
  MY_DATA_1: VGhpcyBNWV9EQVRBXzEK # Значение закодировано Base64
  MY_DATA_2: VGhpcyBNWV9EQVRBXzIK # Значение закодировано Base64
---
# Pod, который получает данные из ConfigMap
apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
  labels:
    app: my-app
spec:
  containers:
    - name: my-container
      image: hashicorp/http-echo:1.0.0
      args: ["-text=MY_DATA_1: ${MY_DATA_1}, MY_DATA_2: ${MY_DATA_2}, MY_DATA_LOADED: ${MY_DATA_LOADED}"]
      # Получаем весь ConfigMap
      envFrom:
        - secretRef:
            name: my-secret
      # Получаем единичным способом переменную с её переименованием:
      env:
        - name: MY_DATA_LOADED # Имя переменной, как она будет видна в Pod
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: MY_DATA_1 # Имя переменной из ConfigMap
      ports:
        - containerPort: 5678
---
# Service, через который будем подключаться к Pod
apiVersion: v1
kind: Service
metadata:
  namespace: my-namespace
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5678

```

Применяем спецификацию:

```
| kubectl apply -f secret-sample.yaml
```

```

secret/my-secret created
pod/my-pod created
service/my-service created

```

Проверяем, что объекты загружены:

```
| kubectl get -n my-namespace secrets,pods,endpoints,services
```

NAME	DATA	AGE
configmap/my-configmap	2	48s

NAME	READY	STATUS	RESTARTS	AGE
pod/my-pod	1/1	Running	0	48s

NAME	ENDPOINTS	AGE
endpoints/my-service	10.244.0.104:5678	47s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/my-service	ClusterIP	10.100.205.102	<none>	80/TCP	47s

Запускаем внутри Kubernetes тестовый Pod, с которого запросим информацию через Service с Pod-a:

```
| kubectl run my-temp -it --rm --restart=Never -n my-namespace --image=alpine/curl:8.14.1 -- /bin/sh
```

```

If you don't see a command prompt, try pressing enter.
/ #

```

Внутри контейнера запускаем:

```
| curl http://my-service
```

```

MY_DATA_1: This MY_DATA_1
, MY_DATA_2: This MY_DATA_2
, MY_DATA_LOADED: This MY_DATA_1

```

Таким образом, через `envFrom.secretRef` и `env` (с `secretKeyRef`) мы передали данные в контейнер.

Завершаем работу всех объектов:

```
| kubectl delete -f secret-sample.yaml
```

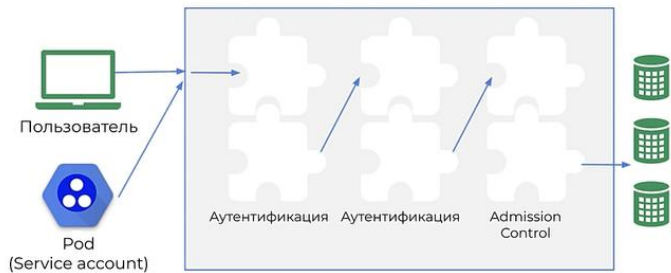
```

secret "my-secret" deleted
pod "my-pod" deleted
service "my-service" deleted

```

4.13 Аутентификация и авторизация

- Аутентификация — кто ты?
- Авторизация — что ты можешь делать?
- Admission Control — могут изменить или отклонить запрос



4.13.1 Аутентификация

Когда сервер API получает запрос, запрос проходит через список плагинов аутентификации. 1-й плагин возвращает обратно в ядро сервера API **имя пользователя, идентификатор пользователя и группу**, которой принадлежит пользователь. Сервер API перестаёт вызывать оставшиеся плагины аутентификации и переходит к этапу авторизации.

Виды пользователей

- обычные пользователи (люди), работающие через kubectl;
- Service accounts
 - используются процессами внутри кластера (приложениями)
 - локальны в namespace
 - создаются и управляются из API

Пример Service accounts: мы разворачиваем кластер RabbitMQ. После поднятия ноды RabbitMQ, она хочет узнать об остальных участниках и обращается к API-серверу. API-сервер возвращает информацию об остальных нодах и все ноды собираются в кластер. Для этого приложению нужен Service account.

Создаём рабочую директорию примеров:

```
| mkdir -p ~/kubernetes/example-auth
```

4.13.1.1 Плагины аутентификации

Их несколько:

- **Пользовательские сертификаты X.509** — каждому пользователю выписываем сертификат. Пользоваться неудобно, т.к. Kubernetes не поддерживает механизм отзыва сертификата;
- **Файл с токенами** — в обычный текстовый файл записываются все пользователи и их группы и токенами. Подкидываем этот файл к API-серверу и запускаем параметром authfile. После этого пользователи разрешено впускать по токenu. Пользоваться неудобно, если API-серверов несколько;
- **Файл с пользователями/паролями** — устаревший способ и уже удалён;
- **Service accounts** — применим для внутренних пользователей. Утентификация идёт по ключу, который генерирует сам Kubernetes;
- **Authentication Proxy** — ставится перед API-сервером и добавляет свои заголовки. Заголовки до него поставить не получится, т.к. после Proxy запросы приходят с ключом;
- **OpenID Connect Provider** — токен доступа даёт внешний сервис OAuth.

4.13.1.2 Получение токена ServiceAccount и использование в прямом запросе к API

При создании ServiceAccount для Pod, к нему автоматически генерируется токен, сертификат и название Namespace, в которое он включён. В Pod-е эти данные монтируются в папку `/var/run/secrets/kubernetes.io/serviceaccount`.

Как это выглядит?

Смотрим существующие Pod с ServiceAccount в Namespace kube-system:

```
| kubectl get -n kube-system pods
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-668d6bf9bc-75lwt	1/1	Running	9 (3h ago)	37d
coredns-668d6bf9bc-894pz	1/1	Running	9 (3h ago)	37d
etcd-combo-1	1/1	Running	9 (3h ago)	37d
etcd-combo-2	1/1	Running	9 (3h ago)	37d
etcd-combo-3	1/1	Running	9 (3h ago)	37d
kube-apiserver-combo-1	1/1	Running	9 (3h ago)	37d
kube-apiserver-combo-2	1/1	Running	9 (3h ago)	37d
kube-apiserver-combo-3	1/1	Running	9 (3h ago)	37d
kube-controller-manager-combo-1	1/1	Running	9 (3h ago)	37d
kube-controller-manager-combo-2	1/1	Running	9 (3h ago)	37d
kube-controller-manager-combo-3	1/1	Running	9 (3h ago)	37d
kube-proxy-dndt7	1/1	Running	9 (3h ago)	37d
kube-proxy-jg2wb	1/1	Running	9 (3h ago)	37d
kube-proxy-zbdgj	1/1	Running	9 (3h ago)	37d
kube-scheduler-combo-1	1/1	Running	9 (3h ago)	37d
kube-scheduler-combo-2	1/1	Running	9 (3h ago)	37d
kube-scheduler-combo-3	1/1	Running	9 (3h ago)	37d
my-kube-ops-view-6868df6684-k5fkv	1/1	Running	8 (3h ago)	35d

Выбираем под с высокими привилегиями, например — coredns. Во внутрь Pod-а зайти нельзя (он создан в минимальном варианте, без обвязок в идт bash, ls, cd и т.п.). Поэтому посмотрим примонтированные к Pod-у **coredns-668d6bf9bc-75lwt** тома:

```
| kubectl -n kube-system get pod coredns-668d6bf9bc-75lwt -o json \
| jq -r '.spec.containers[0].volumeMounts[] | select(.mountPath | contains("serviceaccount"))'
```

```
{
  "mountPath": "/var/run/secrets/kubernetes.io/serviceaccount",
  "name": "kube-api-access-lm4cn",
  "readOnly": true
}
```

Смотрим, что промониторено конкретно, а это файлы, которые содержат — токен, сертификат и имя Namespace :

```
kubectl -n kube-system get pod coredns-668d6bf9bc-75lwt -o json | jq -r '.spec.volumes[] | select(.name | contains("kube-api-access"))'
```

```
{
  "name": "kubernetes-api-access-lm4cn",
  "projected": {
    "defaultMode": 420,
    "sources": [
      {
        "serviceAccountToken": {
          "expirationSeconds": 3607,
          "path": "token"
        }
      },
      {
        "configMap": {
          "items": [
            {
              "key": "ca.crt",
              "path": "ca.crt"
            }
          ],
          "name": "kube-root-ca.crt"
        }
      },
      {
        "downwardAPI": {
          "items": [
            {
              "fieldRef": {
                "apiVersion": "v1",
                "fieldPath": "metadata.namespace"
              },
              "path": "namespace"
            }
          ]
        }
      }
    ]
  }
}
```

Чтобы увидеть все содержимое всех примонтированных через ServiceAccount данных, создадим свой Pod с таким же ServiceAccount coredns (но со всеми обязанками типа bash, ls, cd и т.п.) и войдём в него:

```
cd ~/kubernetes/example-auth
vim serviceaccount-coredns.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  namespace: kube-system # Необходимо, чтобы Pod был в системном Namespace
  name: my-pod
spec:
  serviceAccountName: coredns # Указываем такое же имя ServiceAccount, как и в CoreDNS
  containers:
  - name: my-app
    image: busybox:1.37.0-musl
    command: ["sleep", "3600"] # Чтобы образ заредрился на выполнение
  restartPolicy: Never # Никогда не перезагружался после завершения работы
```

Запускаем манифест:

```
kubectl apply -f serviceaccount-coredns.yaml
```

```
pod/my-pod created
```

Читаем список файлов из Pod-а, примонтированных к нему в специальную директорию `/var/run/secrets/kubernetes.io/serviceaccount`:

```
kubectl exec -it -n kube-system pods/my-pod -- ls -l /var/run/secrets/kubernetes.io/serviceaccount
```

```
total 0
lrwxrwxrwx 1 root root 13 Oct 27 16:29 ca.crt -> ../data/ca.crt
lrwxrwxrwx 1 root root 16 Oct 27 16:29 namespace -> ../data/namespace
lrwxrwxrwx 1 root root 12 Oct 27 16:29 token -> ../data/token
```

Читаем каждый файл по отдельности. Namespace, в котором запущен Pod:

```
kubectl exec -it -n kube-system pods/my-pod -- cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

kube-system

Корневой сертификат для доверия к API кластера:

```
kubectl exec -it -n kube-system pods/my-pod -- cat /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
```

```
-----BEGIN CERTIFICATE-----
MIIDBTCCAe2gAwIBAgIIIdZ0C5iCfrG0wDQYJKoZIhvcNAQELBQAwFTETMBEGA1UE
...
```

JWT токен, который даёт доступ и привелегии в API-сервере:

```
kubectl exec -it -n kube-system pods/my-pod -- cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

[illegible]

Т.к. это стандартный JWT-токен, его можно расшифровать, например на сайте <https://www.jwt.io/>. После расшифровки конкретно этого

токена получаем JSON:

```
Header :
{
  "alg": "RS256",
  "kid": "854shz16EZWMHESJQM-dk-sHBsgiy3acSb2TSXaWoGM"
}

Payload:
{
  "aud": [
    "https://kubernetes.default.svc.cluster.local"
  ],
  "exp": 1793206311,
  "iat": 1761670311,
  "iss": "https://kubernetes.default.svc.cluster.local",
  "jti": "a9fe9215-ab59-49b3-b5b1-64524d6acfef",
  "kubernetes.io": {
    "namespace": "kube-system",
    "node": {
      "name": "combo-1",
      "uid": "75294c09-03aa-46df-924f-181f25c394f5"
    },
    "pod": {
      "name": "my-pod",
      "uid": "6f2e484e-1b1d-480a-a5ee-b31f61ed4753"
    },
    "serviceaccount": {
      "name": "coredns",
      "uid": "d854c61f-7882-458f-b7f0-02e1d25533bd"
    },
    "warnafter": 1761673918
  },
  "nbf": 1761670311,
  "sub": "system:serviceaccount:kube-system:coredns"
}
```

По токену видна вся информация о поде, которому он выдал со всеми вспомогательными данными.

Делаем запрос напрямую к API-серверу через Curl с этим токеном. Для этого узнаём IP-адрес API-с сервера:

```
| kubectl config view | grep server
server: https://192.168.20.101:7443
```

Делаем аналог запроса `kubectl get -n kube-system services`:

```
| curl -ks https://192.168.20.101:7443/api/v1/services -H \
  "Authorization: Bearer $(kubectl exec -it -n kube-system pods/my-pod -- cat /var/run/secrets/kubernetes.io/serviceaccount/token)" \
  | jq -r '.items[].metadata.name'

kubernetes
kube-dns
```

Токен сработал и дал аналогичный список.

Удаляем созданные объекты:

```
| kubectl delete -f serviceaccount-coreDNS.yaml
```

4.13.1.3 Создание сертификата X.509 для входа пользователем

Создаём свой сертификат, подписываем его в кластере Kubernetes и заходим по нему. Единственно, его нельзя отозвать в случае компрометации.

Создаём приватный ключ `my-key.key`:

```
| cd ~/kubernetes/example-auth
openssl genrsa -out my-key.key 2048
```

На основе приватного ключа создаём запрос CSR (Certificate Signing Request), где:

- Common Name (CN) = имя пользователя — здесь **my-user**;
- Organization (O) = группа — здесь группа администраторов кластера **system:masters** (имеет полный доступ, только для администраторов). Если нужна своя группа, например `O=my-team`, нужно будет настроить RBAC.

```
| openssl req -new -key my-key.key -out my-csr.csr -subj "/CN=my-user/O=system:masters"
```

Для подписи понадобятся:

- `ca.crt` — публичный сертификат CA кластера
- `ca.key` — приватный ключ CA (он находится только у администратора)

Подпишем ключ на первой ноде Kubernetes, где эти ключи и сертификаты есть и вернём обратно:

```
# Копируем запрос на подпись на сервер combo-1:
scp my-csr.csr administrator@combo-1.dev.lan:~

# Подписываем его на нём сертификатом и ключём Kubernetes:
ssh administrator@combo-1.dev.lan 'sudo openssl x509 -req -in my-csr.csr -CA /etc/kubernetes/pki/ca.crt -CAkey /etc/kubernetes/pki/ca.key -CAcreateserial -out my-crt.crt -days 365'

# Копируем готовый сертификат обратно:
scp administrator@combo-1.dev.lan:~/my-crt.crt .

# Копируем сертификат кластера:
scp administrator@combo-1.dev.lan:/etc/kubernetes/pki/ca.crt .

# Удаляем на combo-1 уже не нужные файлы:
ssh administrator@combo-1.dev.lan 'sudo rm my-{csr,crt}.**'
```

Теперь у нас есть:

- `my-key.key` — приватный ключ;
- `my-crt.crt` — клиентский сертификат;
- `ca.crt` — сертификат кластера.

Определяем IP и порт кластера:

```
| kubectl cluster-info
```

```
Kubernetes control plane is running at https://192.168.20.101:7443
CoreDNS is running at https://192.168.20.101:7443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Делаем запрос к кластеру через API на возврат списка Namespaces:

```
curl -s -cacert ca.crt --cert my.crt --key my-key.key https://192.168.20.101:7443/api/v1/namespaces | jq -r '.items[].metadata.name'
```

```
default
kube-flannel
kube-node-lease
kube-public
kube-system
my-namespace
```

Доступ по указанным сертификатам подтверждён.

4.13.1.4 ServiceAccount

Версия манифеста:

```
kubectl explain serviceaccount | grep -iE '^(\group|version)'
```

```
VERSION:      v1
```

Создаём манифест:

```
cd ~/kubernetes/example-auth
vim serviceaccount.yaml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-serviceaccount
```

Это только информация без отработки.

4.13.2 Авторизация

Раздача прав на действия с объектами.

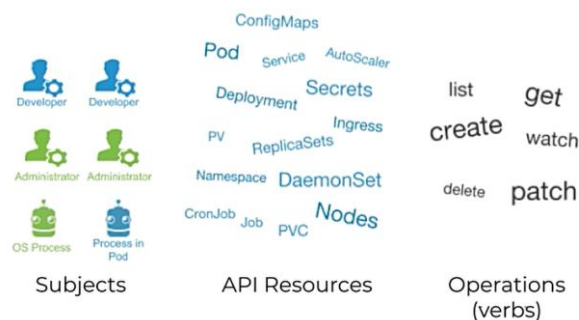
Модули авторизации:

- Node — разрешение kubelet ходить в API. Разрешает работать только на своей ноде;
- ABAC — Attribute Base Access Control — доступ, базирующийся на атрибутах. Гибкий но сложный в понимании метод. Считается устаревшим.
- RBAC — Role Based Access Control — доступ на основе ролей. Актуальный и гибкий метод.
- Webhook — HTTP вызов на сторонний REST-сервис для определения пользовательских привелегий

4.13.2.1 RBAC

Состоит из:

- subject (субъект) — совокупность пользователей и объектов, которые хотят иметь доступ к Kubernetes API;
- resources (ресурсы) — совокупность объектов, доступных в кластере: Pod, Deployment и т.п.
- verbs (действия) — совокупность операций, которые могут быть произведены над ресурсами: Get, Create, Delete и т.п.



Role и **ClusterRole** — Роли описывают действия над ресурсами. Role действуют в пределах одного Namespace. ClusterRole действует на все Namespace (на весь кластер).

RoleBinding и **ClusterRoleBinding** — Соединяют роли с субъектами. RoleBinding действует в пределах Namespace, ClusterRoleBinding действует на весь кластер.

Создаём директорию для проекта:

```
mkdir -p ~/kubernetes/example-rbac
```

4.13.2.2 Определение текущего пользователя и контекста kubectl

Текущего пользователя, под которым работает kubectl можно командой:

```
kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.20.101:7443
    name: kubernetes
contexts:
- context:
```

```
cluster: kubernetes
user: kubernetes-admin
name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: DATA+OMITTED
    client-key-data: DATA+OMITTED
```

Смотрим пользователя в секции `users`.

Т.к. может быть несколько пользователей в `kubectl`, они переключаются по контексту. Посмотреть текущий контекст (помечен `*`) и их список:

```
kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	kubernetes-admin@kubernetes	kubernetes	kubernetes-admin	

4.13.2.3 Проверка прав для пользователя

Проверка, может ли текущий пользователь получить список Pod в Namespace `my-namespace`:

```
kubectl auth can-i get pods -n my-namespace
```

```
yes
```

Проверить от имени конкретного пользователя через `--as`:

```
kubectl auth can-i get pods -n my-namespace --as my-user
```

```
no
```

4.13.2.4 Простой RBAC

Создаём сертификат для пользователя `my-user` в группе `my-group`. Создаём приватный ключ `my-user.key`:

```
cd ~/kubernetes/example-rbac
openssl genrsa -out my-user.key 2048
```

На основе приватного ключа создаём запрос CSR (Certificate Signing Request), где:

- Common Name (CN) = имя пользователя — здесь **my-user**;
- Organization (O) = группа — здесь пользователя **my-group**.

```
openssl req -new -key my-user.key -out my-user.csr -subj "/CN=my-user/O=my-group"
```

Для подписи понадобятся:

- `ca.crt` — публичный сертификат CA кластера
- `ca.key` — приватный ключ CA (он находится только у администратора)

Подпишем ключ на первой ноде Kubernetes, где эти ключи и сертификаты есть и вернём обратно:

```
# Копируем запрос на подпись на сервер combo-1:
scp my-user.csr administrator@combo-1.dev.lan:~

# Подписываем его на нём сертификатом и ключём Kubernetes:
ssh administrator@combo-1.dev.lan 'sudo openssl x509 -req -in my-user.csr -CA /etc/kubernetes/pki/ca.crt -CAkey /etc/kubernetes/pki/ca.key -CAcreateserial -out my-user.crt -days 365'

# Копируем готовый сертификат обратно:
scp administrator@combo-1.dev.lan:~/my-user.crt .

# Копируем сертификат кластера:
scp administrator@combo-1.dev.lan:/etc/kubernetes/pki/ca.crt .

# Удаляем на combo-1 уже не нужные файлы:
ssh administrator@combo-1.dev.lan 'sudo rm my-user.*'
```

Просматриваем созданный сертификат:

```
openssl x509 -in my-user.crt -text -noout
```

```
Certificate:
Data:
  Version: 1 (0x0)
  Serial Number:
    45:1e:f6:1d:9c:47:5f:8c:1d:99:64:52:7f:7c:4a:8d:15:09:19:c9
  Signature Algorithm: sha256WithRSAEncryption
  Issuer: CN = kubernetes
  Validity
    Not Before: Oct 29 15:48:41 2025 GMT
    Not After : Oct 29 15:48:41 2026 GMT
  Subject: CN = my-user, O = my-group
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    Public-Key: (2048 bit)
    Modulus:
      00:d3:ae:05:fc:75:0a:f6:b3:83:be:ab:99:75:69:
      24:67:95:75:ba:46:2d:75:7f:52:96:c1:e2:ba:e8:
      b9:dc:64:82:2a:0a:3f:fc:da:5a:b7:e9:63:39:d2:
      80:27:e0:5d:10:b5:fc:22:8a:fd:24:b2:9a:f2:db:
      9a:39:f3:25:60:86:15:05:2a:ee:b9:0a:51:41:c9:
      5d:1d:e2:fc:9c:3c:e2:4f:cd:1d:74:11:0e:fd:03:
      f5:67:66:29:d7:a8:0b:ec:e3:54:e4:4e:7e:12:da:
      5c:67:8b:2a:73:33:98:ed:2b:2c:30:92:15:6b:c9:
      ae:71:09:fa:40:39:07:4b:35:d8:e3:24:83:f1:ce:
      bf:8a:dd:07:25:f5:01:20:2e:b9:4a:27:39:a5:1f:
      5f:f3:e9:95:0a:34:23:c4:d1:77:2b:3f:4a:fd:8d:
      2c:ca:ea:bb:49:54:25:e7:88:10:11:49:a9:01:fa:
      73:a9:a9:11:ce:95:19:ac:6d:58:e0:11:ef:05:6b:
      2b:aa:54:67:94:5c:ba:52:01:a2:15:39:8d:db:c1:
      23:7d:97:47:91:7a:82:47:32:af:38:fe:d6:66:6c:
      29:1c:b9:1c:36:84:61:7e:4f:d5:0a:80:45:bc:d6:
      00:1f:e9:45:80:a5:3b:8c:b9:70:58:d3:24:3e:ec:
      b0:cf
    Exponent: 65537 (0x10001)
  Signature Algorithm: sha256WithRSAEncryption
  Signature Value:
```

```
7b:32:e7:1f:eb:da:f3:31:6a:73:2f:55:89:b9:69:74:9c:ac:
ab:0e:f6:e9:e4:db:98:08:df:9a:11:a6:e4:02:b0:81:b5:87:
34:05:1b:6b:2a:91:72:f5:c7:db:aa:f3:46:8e:f4:ae:9d:81:
25:f2:dd:27:4e:14:2a:67:8d:84:1b:c5:d8:4c:87:80:0a:15:
18:9b:cb:d7:e8:62:ee:91:2d:f0:ac:c3:e6:8f:a5:32:b8:54:
ca:5d:32:59:a7:f1:1e:7e:c0:0d:b9:c0:c6:4d:d5:ae:00:b2:
54:a0:e6:64:11:9c:21:66:cf:70:51:8f:36:d8:d7:a9:51:7c:
2e:c9:99:9d:eb:f1:b2:79:a5:43:ff:8b:3b:93:e9:be:3b:0b:
7a:a6:5b:c3:44:f9:ba:47:5b:0d:c1:06:76:43:19:96:de:84:
e3:da:68:18:3d:d8:73:bd:f5:d4:14:36:e0:d2:49:e4:2b:1d:
9d:1e:18:53:96:62:84:a4:7d:7a:e9:c0:e3:d7:64:75:0d:eb:
56:e8:aa:6b:d8:cf:16:84:80:9f:25:40:fa:01:2b:a5:36:a6:
ca:5c:11:69:55:fc:a8:5c:cd:c3:9a:74:3f:78:0c:3f:60:1a:
89:cd:b1:da:78:25:b2:ad:06:86:0e:81:ef:06:22:85:b7:9a:
54:5f:60:6a
```

Теперь у нас есть:

- `my-user.key` — приватный ключ;
- `my-user.crt` — клиентский сертификат, подписанный Kubernetes;
- `ca.crt` — сертификат кластера.

Определяем IP и порт кластера:

```
| kubectl cluster-info

Kubernetes control plane is running at https://192.168.20.101:7443
CoreDNS is running at https://192.168.20.101:7443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Делаем запрос к кластеру через API на возврат списка Namespaces:

```
| curl -s --cacert ca.crt --cert my-user.crt --key my-user.key https://192.168.20.101:7443/api/v1/namespaces

{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "namespaces is forbidden: User \"my-user\" cannot list resource \"namespaces\" in API group \"\" at the cluster scope",
  "reason": "Forbidden",
  "details": {
    "kind": "namespaces"
  },
  "code": 403
}
```

Но Kubernetes запретил доступ, т.к. действительно, нет описания разрешений для пользователя `my-user`.

Определяем версии API для создаваемых спецификаций. Т.е. в запросе мы запрашиваем Namespaces, то это будут кластерные роли, а не простые. Если бы мы запрашивали, например Pod из какого либо Namespace, то нужно было бы использовать Role и RoleBinding с указанием namespace в котором будут действовать разрешения:

```
| kubectl explain clusterrole | grep -Ei '^(group|version)'
```

```
GROUP:      rbac.authorization.k8s.io
VERSION:    v1
```

```
| kubectl explain clusterrolebinding | grep -Ei '^(group|version)'
```

```
GROUP:      rbac.authorization.k8s.io
VERSION:    v1
```

Создаем спецификацию прав на весь кластер для **пользователя**:

```
| vim my-role-user.yaml

---
# Разрешения
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: my-role
rules:
- apiGroups: [""]
  resources: ["namespaces"] # Позволяет работать с namespaces
  verbs: ["get", "list", "watch"] # Позволяет получать, читать и следить

---
# Привязка пользователя к разрешению
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: my-rolebinding
subjects:
- kind: User # Привязываем роль к пользователю
  name: my-user # Имя пользователя из CN сертификата
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole # Ссылаемся на роль, к которой привязываем
  name: my-role # Имя роли, к которой привязываем
  apiGroup: rbac.authorization.k8s.io
```

Применяем спецификацию:

```
| kubectl apply -f my-role-user.yaml

role.rbac.authorization.k8s.io/my-role created
rolebinding.rbac.authorization.k8s.io/my-rolebinding created
```

Снова проверяем запрос с получением из JSON имён Namespace:

```
| curl -s --cacert ca.crt --cert my-user.crt --key my-user.key https://192.168.20.101:7443/api/v1/namespaces | jq -r '.items[].metadata.name'
```

```
default
kube-flannel
kube-node-lease
kube-public
```

```
kube-system
my-namespace
```

Удаляем разрешение для пользователя:

```
| kubectl delete -f my-role-user.yaml
```

Проверяем, что прав теперь нет:

```
| curl -s --cacert ca.crt --cert my-user.crt --key my-user.key https://192.168.20.101:7443/api/v1/namespaces
```

```
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "namespaces is forbidden: User \"my-user\" cannot list resource \"namespaces\" in API group \"\" at the cluster scope",
  "reason": "Forbidden",
  "details": {
    "kind": "namespaces"
  },
  "code": 403
}
```

Создадим права на Namespace my-namespace для группы (из О сертификата):

```
| vim my-role-group.yaml
```

```
---
# Разрешения
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: my-namespace # Ограничиваем конкретным namespace
  name: my-role
rules:
- apiGroups: [""]
  resources: ["pods"] # Позволяет работать с namespaces
  verbs: ["get", "list", "watch"] # Позволяет получать, читать и следить

---
# Привязка пользователя к разрешению
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: my-namespace # Ограничиваем конкретным namespace
  name: my-rolebinding
subjects:
- kind: Group # Привязываем роль к группе
  name: my-group # Имя группы из CN сертификата
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role # Ссылаемся на роль, к которой привязываем
  name: my-role # Имя роли, к которой привязываем
  apiGroup: rbac.authorization.k8s.io
```

Применяем спецификацию:

```
| kubectl apply -f my-role-group.yaml
```

```
role.rbac.authorization.k8s.io/my-role created
rolebinding.rbac.authorization.k8s.io/my-rolebinding created
```

Делаем запрос на получение списка Pods в Namespace my-namespace:

```
| curl -s --cacert ca.crt --cert my-user.crt --key my-user.key https://192.168.20.101:7443/api/v1/namespaces/my-namespace/pods
```

```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "641992"
  },
  "items": []
}
```

Всё работает, просто нет рабочих подов сейчас в Namespace my-namespace.

Удаляем права:

```
| kubectl delete -f my-role-group.yaml
```

4.13.2.5 Добавление сертификата пользователя в контекст kubectl

Т.к. сертификаты для пользователя были сделаны, можно из-под него работать не через curl, а через kubectl.

Добавляем нового пользователя и новый контекст для него в kubectl:

```
| kubectl config set-credentials my-user --client-certificate=my-user.crt --client-key=my-user.key
```

```
User "my-user" set.
```

Проверяем список пользователей:

```
| kubectl config get-users
```

```
NAME
kubernetes-admin
my-user
```

Создаём контекст для добавленного пользователя:

```
| kubectl config set-context my-user-context --user=my-user
```

```
Context "my-user-context" created.
```

Проверяем, что контекст добавлен:

```
| kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	kubernetes-admin@kubernetes	kubernetes	kubernetes-admin	
	my-user-context		my-user	

Активируем контекст my-user-context:

```
| kubectl config use-context my-user-context
```

```
Switched to context "my-user-context".
```

Проверяем, что контекст переключен:

```
| kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	kubernetes-admin@kubernetes my-user-context	kubernetes	kubernetes-admin my-user	

Т.к. для пользователя my-user сейчас не установлены никакие Role, то любой запрос будет возвращать ошибку.

Переключаем контекст обратно на администратора:

```
| kubectl config use-context kubernetes-admin@kubernetes
```

4.13.2.6 Встроенные кластерные роли

Смотрим все встроенные кластерные роли:

```
| kubectl get clusterrole
```

NAME	CREATED AT
admin	2025-09-20T13:19:10Z
cluster-admin	2025-09-20T13:19:10Z
edit	2025-09-20T13:19:10Z
flannel	2025-09-20T14:08:22Z
kubeadm:get-nodes	2025-09-20T13:19:11Z
my-kube-ops-view	2025-09-21T19:51:42Z
system:aggregate-to-admin	2025-09-20T13:19:10Z
system:aggregate-to-edit	2025-09-20T13:19:10Z
system:aggregate-to-view	2025-09-20T13:19:10Z
system:auth-delegator	2025-09-20T13:19:10Z
system:basic-user	2025-09-20T13:19:10Z
system:certificates.k8s.io:certificatesigningrequests:nodeclient	2025-09-20T13:19:10Z
system:certificates.k8s.io:certificatesigningrequests:selfnodeclient	2025-09-20T13:19:10Z
system:certificates.k8s.io:kube-apiserver-client-approver	2025-09-20T13:19:10Z
system:certificates.k8s.io:kube-apiserver-client-kubelet-approver	2025-09-20T13:19:10Z
system:certificates.k8s.io:kubelet-serving-approver	2025-09-20T13:19:10Z
system:certificates.k8s.io:legacy-unknown-approver	2025-09-20T13:19:10Z
system:controller:attachdetach-controller	2025-09-20T13:19:10Z
system:controller:certificate-controller	2025-09-20T13:19:10Z
system:controller:clusterrole-aggregation-controller	2025-09-20T13:19:10Z
system:controller:cronjob-controller	2025-09-20T13:19:10Z
system:controller:daemon-set-controller	2025-09-20T13:19:10Z
system:controller:deployment-controller	2025-09-20T13:19:10Z
system:controller:disruption-controller	2025-09-20T13:19:10Z
system:controller:endpoint-controller	2025-09-20T13:19:10Z
system:controller:endpointslice-controller	2025-09-20T13:19:10Z
system:controller:endpointslicemirroring-controller	2025-09-20T13:19:10Z
system:controller:ephemeral-volume-controller	2025-09-20T13:19:10Z
system:controller:expand-controller	2025-09-20T13:19:10Z
system:controller:generic-garbage-collector	2025-09-20T13:19:10Z
system:controller:horizontal-pod-autoscaler	2025-09-20T13:19:10Z
system:controller:job-controller	2025-09-20T13:19:10Z
system:controller:legacy-service-account-token-cleaner	2025-09-20T13:19:10Z
system:controller:namespace-controller	2025-09-20T13:19:10Z
system:controller:node-controller	2025-09-20T13:19:10Z
system:controller:persistent-volume-binder	2025-09-20T13:19:10Z
system:controller:pod-garbage-collector	2025-09-20T13:19:10Z
system:controller:pvc-protection-controller	2025-09-20T13:19:10Z
system:controller:pvc-protection-controller	2025-09-20T13:19:10Z
system:controller:replicaset-controller	2025-09-20T13:19:10Z
system:controller:replication-controller	2025-09-20T13:19:10Z
system:controller:resourcequota-controller	2025-09-20T13:19:10Z
system:controller:root-ca-cert-publisher	2025-09-20T13:19:10Z
system:controller:route-controller	2025-09-20T13:19:10Z
system:controller:service-account-controller	2025-09-20T13:19:10Z
system:controller:service-controller	2025-09-20T13:19:10Z
system:controller:statefulset-controller	2025-09-20T13:19:10Z
system:controller:ttl-after-finished-controller	2025-09-20T13:19:10Z
system:controller:ttl-controller	2025-09-20T13:19:10Z
system:controller:validatingadmissionpolicy-status-controller	2025-09-20T13:19:10Z
system:coredns	2025-09-20T13:19:12Z
system:discovery	2025-09-20T13:19:10Z
system:heapster	2025-09-20T13:19:10Z
system:kube-aggregator	2025-09-20T13:19:10Z
system:kube-controller-manager	2025-09-20T13:19:10Z
system:kube-dns	2025-09-20T13:19:10Z
system:kube-scheduler	2025-09-20T13:19:10Z
system:kubelet-api-admin	2025-09-20T13:19:10Z
system:monitoring	2025-09-20T13:19:10Z
system:node	2025-09-20T13:19:10Z
system:node-bootstrap	2025-09-20T13:19:10Z
system:node-problem-detector	2025-09-20T13:19:10Z
system:node-proxier	2025-09-20T13:19:10Z
system:persistent-volume-provisioner	2025-09-20T13:19:10Z
system:public-info-viewer	2025-09-20T13:19:10Z
system:service-account-issuer-discovery	2025-09-20T13:19:10Z
system:volume-scheduler	2025-09-20T13:19:10Z
view	2025-09-20T13:19:10Z

Обращаем внимание на роли:

- cluster-admin — роль самого главного админа кластера;
- admin — роль самого главного админа в конкретном Namespace. Он не может настраивать квоты Namespace и изменять его параметры;
- view — роль позволяет только просматривать объекты (но далеко не все).

4.14 Linux Capabilities

Кусочки привелений root для пользователей:

- SETPCAP
- SYS_ADMIN
- SYS_NICE
- SYS_TIME
- NEW_ADMIN
- NET_RAW

Подготавливаем директорию для проекта:

```
| mkdir -p ~/kubernetes/example-capabilities
```

4.14.1 Пример Capabilities

По умолчанию, программе запрещено занимать порты ниже 1024-го в режиме пользователя. Например Nginx нельзя запустить на 80-м порту под пользователем. Но в Docker уже установлены соответствующие права, поэтому контейнер Nginx запустить на этом порту можно:

```
| docker run --network=host nginx:1.28.0-alpine3.21
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/10/30 15:44:44 [notice] 1#1: using the "epoll" event method
2025/10/30 15:44:44 [notice] 1#1: nginx/1.28.0
2025/10/30 15:44:44 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2025/10/30 15:44:44 [notice] 1#1: OS: Linux 6.8.0-83-generic
2025/10/30 15:44:44 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2025/10/30 15:44:44 [notice] 1#1: start worker processes
2025/10/30 15:44:44 [notice] 1#1: start worker process 30
2025/10/30 15:44:44 [notice] 1#1: start worker process 31
2025/10/30 15:44:44 [notice] 1#1: start worker process 32
2025/10/30 15:44:44 [notice] 1#1: start worker process 33
```

Но если мы отберём эту привилегию (добавив опцию `--cap-drop CAP_NET_BIND_SERVICE`) у Docker, то Nginx не сможет запуститься, т.к. не сможет занять порт 80:

```
| docker run --network=host --cap-drop CAP_NET_BIND_SERVICE nginx:1.28.0-alpine3.21
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/10/30 15:47:04 [emerg] 1#1: bind() to 0.0.0.0:80 failed (13: Permission denied)
nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)
```

4.14.2 SecurityContext

SecurityContext — опция в спецификации, которая определяет привилегии и доступы которые будет иметь под и его контейнер. Создаём пример такого пода:

```
| cd ~/kubernetes/example-capabilities
| vim securitycontext.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  namespace: my-namespace
  name: my-pod
spec:
  # Определяем правила для всех контейнеров пода:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
  containers:
    - name: my-app
      image: busybox:1.37.0-musl
      command: ["sh", "-c", "sleep 1h"]
      securityContext:
        capabilities:
          # Определяем правила для конкретного контейнера пода
          add: ["NET_ADMIN", "SYS_TIME"]
```

Запускаем Pod:

```
| kubectl apply -f securitycontext.yaml
```

Заходим во внутрь пода:

```
| kubectl exec -it -n my-namespace pods/my-pod -- /bin/sh
```

Удаляем Pod:

```
| kubectl delete -f securitycontext.yaml
```

4.14.3 Хакерский Pod

В Pod монтируем хостовую директорию во внутрь контейнера и смотрим, что там. Создаём Pod:

```
| vim securitycontext-hacker.yaml
```

```
apiVersion: v1
kind: Pod
```

```

metadata:
  namespace: my-namespace
  name: my-pod
spec:
  containers:
    - name: my-app
      image: nginx:1.28.0-alpine3.21
      # Монитруем корень хоста в директорию /master
      volumeMounts:
        - mountPath: /master
          name: master
  volumes:
    - name: master
      # Указываем корень хоста:
      hostPath:
        path: /
        type: Directory
      # На стенде уже удалён этот параметр, в примене он не нужен,
      # только в реальном кластере.
      #tolerations:
      # - effect: NoShedule
      #   operator: Exists
      # Указываем запуститься только на контрольной ноде:
      #nodeSelector:
      # node-role.kubernetes.io/master: ""

```

Запускаем спецификацию:

```
| kubectl apply -f securitycontext-hacker.yaml
```

Заходим в созданный Pod:

```
| kubectl exec -it -n my-namespace pods/my-pod -- bash
```

Заходим в директорию Kubernetes, где хранятся все его ключи (считай, получили полный доступ к кластеру):

```

| cd /master/etc/kubernetes/pki
| ls -l

-rw-r--r-- 1 root root 1123 Sep 20 13:19 apiserver-etcd-client.crt
-rw----- 1 root root 1675 Sep 20 13:19 apiserver-etcd-client.key
-rw-r--r-- 1 root root 1176 Sep 20 13:19 apiserver-kubelet-client.crt
-rw----- 1 root root 1679 Sep 20 13:19 apiserver-kubelet-client.key
-rw-r--r-- 1 root root 1294 Sep 20 13:19 apiserver.crt
-rw----- 1 root root 1679 Sep 20 13:19 apiserver.key
-rw-r--r-- 1 root root 1107 Sep 20 13:19 ca.crt
-rw----- 1 root root 1679 Sep 20 13:19 ca.key
-rw-r--r-- 1 root root 41 Oct 29 15:48 ca.srl
drwxr-xr-x 2 root root 4096 Sep 20 13:19 etcd
-rw-r--r-- 1 root root 1123 Sep 20 13:19 front-proxy-ca.crt
-rw----- 1 root root 1675 Sep 20 13:19 front-proxy-ca.key
-rw-r--r-- 1 root root 1119 Sep 20 13:19 front-proxy-client.crt
-rw----- 1 root root 1679 Sep 20 13:19 front-proxy-client.key
-rw----- 1 root root 1679 Sep 20 13:19 sa.key
-rw----- 1 root root 451 Sep 20 13:19 sa.pub

```

Получаем содержимое admin.conf:

```

| cd /master/etc/kubernetes
| cat admin.conf

apiVersion: v1
clusters:
- cluster:
# ...

```

Копируем содержимое в буфер обмена и выходим из Pod. На хостовой машине создаём файл config-hacked.conf и сохраняем скопированное содержимое туда:

```
| vim config-hacked.conf
```

Проверяем, сможем ли мы работать со всеми правами с таким конфигом:

```

| kubectl --kubeconfig=config-hacked.conf auth can-i ***
yes

```

Завершаем работу и очищаем все рабочее пространство:

```
| kubectl delete -f securitycontext-hacker.yaml
```

4.15 CNI

Container Network Interface (CNI) — стандарт (спецификация) конфигурирования сети. Плагин должен иметь определённые методы с определёнными параметрами, например:

- ADD — вызывается при создании нового пода на хосте. При его вызове, плагин должен создать новый интерфейс veth с одним концом, подключенным к namespace Pod-a, а другим к хосту.
- DEL
- CHECK
- VERSION

Существуют встроенные плагины и сторонние. Сторонние:

- Flannel — один из старейших плагинов. Самый простой в установке и настройке. Есть 3 режима — оверлейная сеть по верх хостовой, host gateway, UTP (в целях debug);
- Calico — навороченный open source проект. Продвинутые функции сетевой безопасности. Является одним из самых быстрых плагинов;
- Weave Net — очень быстрый. Работает на втором уровне модели OSI;
- Cilium — работает на уровне BPF.

Создаём рабочую директорию:

```
mkdir -p ~/kubernetes/example-cni
cd ~/kubernetes/example-cni
```

4.15.1 Исследование Flannel

Flannel работает как DaemonSet. Смотрим в Namespace kube-flannel. Смотрим его описание:

```
kubectl get -n kube-flannel daemonsets.apps -o wide
```

NAME	DESIRED	CURRENT	READY	UP-TO-DATE	AVAILABLE	NODE SELECTOR	AGE	CONTAINERS	IMAGES	SELECTOR
kube-flannel-ds	3	3	3	3	3	<none>	42d	kube-flannel	ghcr.io/flannel-io/flannel:v0.27.3	app=flannel,k8s-app=flannel

Он располагает свои Pod на каждой из нод. Они там расположены для того, чтобы настраивать сеть хоста для обеспечения сетевой связанности. Смотрим его Pod-ы:

```
kubectl get -n kube-flannel pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
kube-flannel-ds-4gpxm	1/1	Running	11 (4d23h ago)	42d	192.168.20.12	combo-2	<none>		<none>	
kube-flannel-ds-5rp67	1/1	Running	10 (4d23h ago)	42d	192.168.20.13	combo-3	<none>		<none>	
kube-flannel-ds-l69xz	1/1	Running	10 (4d23h ago)	42d	192.168.20.11	combo-1	<none>		<none>	

Свою конфигурацию Flannel хранит в ConfigMap-ах. Посмотрим в них, какой из 3-х режимов его работы сейчас включен. Для этого смотрим описание одного из Pod и ищем секцию с подключённым ConfigMap flannel-cfg:

```
kubectl describe -n kube-flannel pods/kube-flannel-ds-4gpxm
```

```
# ...
Volumes:
run:
  Type:          HostPath (bare host directory volume)
  Path:          /run/flannel
  HostPathType:
cni-plugin:
  Type:          HostPath (bare host directory volume)
  Path:          /opt/cni/bin
  HostPathType:
cni:
  Type:          HostPath (bare host directory volume)
  Path:          /etc/cni/net.d
  HostPathType:
flannel-cfg:
  Type:          ConfigMap (a volume populated by a ConfigMap)
  Name:          kube-flannel-cfg
  Optional:      false
xtables-lock:
  Type:          HostPath (bare host directory volume)
  Path:          /run/xtables.lock
  HostPathType:  FileOrCreate
kube-api-access-zj8b2:
  Type:          Projected (a volume that contains injected data from multiple sources)
  TokenExpirationSeconds: 3607
  ConfigMapName:  kube-root-ca.crt
  ConfigMapOptional: <nil>
  DownwardAPI:    true
# ...
```

Смотрим ConfigMap kube-flannel-cfg:

```
kubectl describe configmaps -n kube-flannel kube-flannel-cfg
```

```
Name:          kube-flannel-cfg
Namespace:     kube-flannel
Labels:        app=flannel
               k8s-app=flannel
               tier=node
Annotations:   <none>

Data
====
cni-conf.json:
----
{
  "name": "cbr0",
  "cniVersion": "0.3.1",
  "plugins": [
    {
      "type": "flannel",
      "delegate": {
        "hairpinMode": true,
        "isDefaultGateway": true
      }
    },
    {
      "type": "portmap",
      "capabilities": {
        "portMappings": true
      }
    }
  ]
}

net-conf.json:
----
{
  "Network": "10.244.0.0/16",
  "EnableNFTables": false,
  "Backend": {
    "Type": "vxlan"
  }
}

BinaryData
====

Events: <none>
```

По конфигурации в `net-conf.json` видим, что используется режим VXLAN. Для того, чтобы увидеть, как это выглядит на хосте, заходим на хост `combo-1` и смотрим сетевой интерфейс:

```
ssh administrator@combo-1.dev.lan
ip -c -br addr

lo                UNKNOWN    127.0.0.1/8 ::1/128
eth0              UP                192.168.20.11/24 192.168.20.101/32 fe80::215:5dff:fe01:c69e/64
docker0           DOWN          172.17.0.1/16
flannel.1         UNKNOWN    10.244.0.0/32 fe80::a4f4:fcff:fe3a:59cd/64
cnio              DOWN          10.244.0.1/24 fe80::401a:a2ff:fee6:db2b/64
```

Смотрим подробности про интерфейс `flannel.1`:

```
ip -c -details link show flannel.1

4: flannel.1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UNKNOWN mode DEFAULT group default
   link/ether a6:f4:fc:3a:59:cd brd ff:ff:ff:ff:ff:ff promiscuity 0  allmulti 0  minmtu 68  maxmtu 65535
info: Using default fan map value (33)
   vxlan id 1 local 192.168.20.11 dev eth0 srcport 0 0 dstport 8472 nolearning ttl auto ageing 300 udpcsum noudp6zerocsumtx noudp6zerocsumrx
   addrngenmode eui64 numtxqueues 1 numrxqueues 1 gso_max_size 62780 gso_max_segs 65535 tso_max_size 62780 tso_max_segs 65535 gro_max_size 65536
```

В описании видим, что тип устройства VXLAN.

4.15.2 NetworkPolicy

Т.е. в стандартном варианте, любой Pod имеет доступ к любому другому поду, то это небезопасно. Для ограничений используется NetworkPolicy.

```
kubectl explain networkpolicy | grep -Ei '^(group|version)'
```

```
GROUP:      networking.k8s.io
VERSION:    v1
```

Но они поддерживаются не всеми CNI плагинами. Например Flannel их не поддерживает, а Calico поддерживает.

Варианты использования

Запретить все подключения (для примера):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  namespace: my-namespace
  name: default-deny
spec:
  podSelector: {}
```

Разрешить подключения к базе (для примера). В нём Pod-ы с лейблом `app=app` могут подключаться к Pod-ам с `app=database` по порту 5432.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  namespace: my-namespace
  name: postgres-allow
spec:
  podSelector:
    matchLabels:
      app: database
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: app
  ports:
    - port: 5432
```

Настроить доступ можно и для Namespace и для целого IP блока.

Разрешить подключение к поду из Namespace `billing` (для примера):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  namespace: my-namespace
  name: ns-default-allow
spec:
  podSelector:
    matchLabels:
      app: database
  ingress:
    - from:
      # Разрешить подключение из namespace billing
      - namespaceSelector:
          matchLabels:
            ns: billing
  ports:
    - port: 5432
```

4.15.3 Сеть Docker

Хостовая машина соединяется с контейнером Docker через виртуальные сетевые интерфейсы. Внутри контейнера создаётся интерфейс `eth0`, на хосте — `veth` интерфейс.

Запускаем Docker контейнер на хосте:

```
docker run -d --name=my-temp nginx:1.28.0-alpine3.21

5d1a79a5f9fc44b416f07b3b8681fc8f357a548492a5d85f7ccda159ed7ef5e6
```

Проверяем, что контейнер запущен:

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5d1a79a5f9fc	nginx:1.28.0-alpine3.21	"/docker-entrypoint..."	28 seconds ago	Up 27 seconds	80/tcp	my-temp

Заходим во внутрь контейнера:

```
| docker exec -it my-temp /bin/sh
/ #
```

Смотрим описание сетевого интерфейса:

```
| ip addr

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether e6:e0:06:a6:d8:c7 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

Видим интерфейс `eth0@if15`, который подключён со стороны контейнера. Получаем его индекс, чтобы понять, какой `veth` интерфейс соединён с ним со стороны хоста (в принципе он уже написан в его имени `eth0@if15`):

```
| cat /sys/class/net/eth0/iflink
15
```

Выходим через `exit` из контейнера.

На хосте проходим по всем `veth` интерфейсам и ищем интерфейс с таким же индексом (в принципе он уже указан в порядковом номере интерфейса `15`):

```
| grep -l 15 /sys/class/net/veth*/ifindex
/sys/class/net/vethb28fc54/ifindex
```

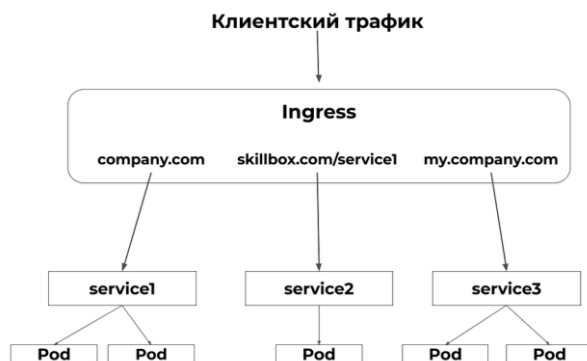
Смотрим описание найденного интерфейса:

```
| ip -c addr show vethb28fc54

15: vethb28fc54@if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether 8a:de:e6:07:8e:12 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::88de:e6ff:fe07:8e12/64 scope link
        valid_lft forever preferred_lft forever
```

4.16 Ingress

Ingress — набор правил внутри кластера Kubernetes, предназначенный для того, чтобы входящие подключения могли достичь сервисов приложения. Работает на 7-м уровне модели OSI, поэтому он понимает заголовки HTTP запросов и может по ним коммутировать трафик. Один Ingress может предоставлять доступ к десяткам служб.



Получаем текущую версию API для спецификации DaemonSet:

```
| kubectl explain ingress | grep -Ei '^ (group|version) '
GROUP:      networking.k8s.io
VERSION:    v1
```

Пример спецификации:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: my-namespace
  annotations:
    # Настройка поведения Ingress.
    # На Service придёт /, вместо /svc1:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    # Хост, на который приходит запрос:
    - host: hello-world.com
      http:
        paths:
          # Префикс пути на хосте:
          - path: /svc1
            # Prefix, Exact, ImplementationSpecific
            pathType: Prefix
            backend:
              service:
                # Имя сервиса, на который делать перенаправление:
                name: my-deployment
                # Порт для трафика на сервисе:
                port:
                  number: 8082
```

Подготавливаем директорию для проекта:

```
mkdir -p ~/kubernetes/example-ingress
cd ~/kubernetes/example-ingress
```

4.16.1 Простой Ingress

Создаём поды и сервис к ним:

```
kubectl create deployment my-deployment --namespace my-namespace --image=nginx:1.29.1 --replicas=3
kubectl expose deployment -n my-namespace my-deployment --port=8082 --target-port=80
```

Создаём спецификацию:

```
vim simple-ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: my-namespace
  name: my-ingress
  annotations:
    # Настройка поведения Ingress.
    # На Service придёт /, вместо /svc1:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - http:
        paths:
          # Префикс пути на хосте:
          - path: /
            pathType: Prefix
            backend:
              service:
                # Имя сервиса, на который делать перенаправление:
                name: my-deployment
                # Порт для трафика на сервисе:
                port:
                  number: 8082
```

Применяем спецификацию:

```
kubectl apply -f simple-ingress.yaml
```

```
ingress.networking.k8s.io/my-ingress created
```

Смотрим описание созданного Ingress:

```
kubectl describe -n my-namespace ingress/my-ingress
```

```
Name: my-ingress
Labels: <none>
Namespace: my-namespace
Address:
Ingress Class: <none>
Default backend: <default>
Rules:
  Host      Path      Backends
  ----      -
  *         /         my-deployment:8082 (10.244.1.58:80,10.244.0.127:80,10.244.2.60:80)
Annotations: nginx.ingress.kubernetes.io/rewrite-target: /
Events: <none>
```

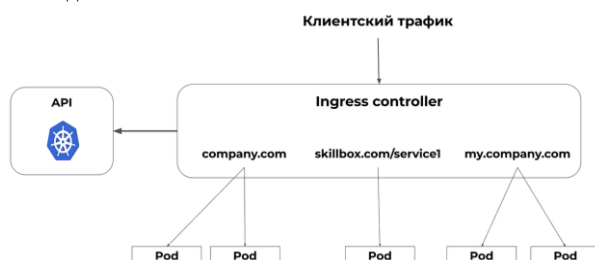
Здесь описывается, что запрос на любой хост будет переадресован в корень хоста сервиса `my-deployment:8082`.

Завершаем работы Deployment и Service:

```
kubectl delete -n my-namespace deployments.apps/my-deployment services/my-deployment
```

4.16.2 IngressController

IngressController — это приложение, которое занимается непосредственно обработкой трафика. Правила его работы описываются с помощью объектов Ingress. Он представляет собой реверс-прокси в Pod-е, который может располагаться на любом из узлов. Этот Pod запрашивает из API список Pod-ов и на основе этой информации строит конфигурацию. Далее он создаёт сервис типа PortNode и перенаправляет на себя все запросы со всех нод.



Устанавливаем Kubernetes Nginx IngressController через официальный Helm-чарт kubernetes <https://github.com/kubernetes/ingress->

[nginx/releases](#), чтобы в процессе установки его можно было настраивать.

Подключаем репозиторий:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx

"ingress-nginx" has been added to your repositories
```

Обновляем список ресурсов репозитория:

```
helm repo update

Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "ingress-nginx" chart repository
...Successfully got an update from the "christianhuth" chart repository
...Successfully got an update from the "bitnami" chart repository
Update Complete. Happy Helming!
```

Устанавливаем контроллер в namespace ingress-nginx (имеет такое название по умолчанию при ручной установке). При установке используем режим NodePort:

```
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx \
  --create-namespace \
  --set controller.service.type=NodePort \
  --set controller.service.nodePorts.http=30080 \
  --set controller.service.nodePorts.https=30443 \
  --set controller.env.TZ=Europe/Moscow

NAME: ingress-nginx
LAST DEPLOYED: Mon Nov 3 11:14:08 2025
NAMESPACE: ingress-nginx
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The ingress-nginx controller has been installed.
Get the application URL by running these commands:
  export HTTP_NODE_PORT=30080
  export HTTPS_NODE_PORT=30443
  export NODE_IP=$(kubectl get nodes --output jsonpath="{.items[0].status.addresses[1].address}")

  echo "Visit http://${NODE_IP}:${HTTP_NODE_PORT} to access your application via HTTP."
  echo "Visit https://${NODE_IP}:${HTTPS_NODE_PORT} to access your application via HTTPS."

An example Ingress that makes use of the controller:
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example
  namespace: foo
spec:
  ingressClassName: nginx
  rules:
    - host: www.example.com
      http:
        paths:
          - pathType: Prefix
            backend:
              service:
                name: exampleService
                port:
                  number: 80
            path: /
    # This section is only required if TLS is to be enabled for the Ingress
    tls:
      - hosts:
          - www.example.com
        secretName: example-tls

If TLS is enabled for the Ingress, a Secret containing the certificate and key must also be provided:

apiVersion: v1
kind: Secret
metadata:
  name: example-tls
  namespace: foo
data:
  tls.crt: <base64 encoded cert>
  tls.key: <base64 encoded key>
type: kubernetes.io/tls
```

где:

- `--namespace ingress-nginx` и `--create-namespace` — создают Namespace;
- `--set controller.service.*` — объявляют порты NodePort.

Теперь Ingress Controller будет слушать на всех узлах: combo-1:30080, combo-2:30080 и combo-1:30080.

Смотрим, какого класса (нужно будет указать в Ingress) контроллер был добавлен:

```
kubectl get ingressclass

NAME      CONTROLLER      PARAMETERS      AGE
nginx     k8s.io/ingress-nginx  <none>          91m
```

Здесь класс nginx, который нужно будет указывать в описании Ingress.

Проверяем, что Deploy и Pod запущены и работают:

```
kubectl get -n ingress-nginx pods -o wide

NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE    NOMINATED NODE   READINESS GATES
ingress-nginx-controller-7d8cffd99c-qc5l6  1/1     Running   0          13m   10.244.0.134   combo-1  <none>           <none>
```

Заходим во внутрь созданного Pod-а IngressController-а, чтобы посмотреть, из чего он состоит:

```
kubectl exec -it -n ingress-nginx pods/ingress-nginx-controller-9596bd777-kx4lk -- bash

ingress-nginx-controller-9596bd777-kx4lk:/etc/nginx$
```

Смотрим список процессов:

ps auxf

```

ingress-nginx-controller-77968558-tldc7:/etc/nginx$ ps auxf
PID   USER     TIME  COMMAND
  1 www-data  0:00 /usr/bin/dumb-init -- /nginx-ingress-controller --election-id=ingress-nginx-leader --controller-class=k8s.io/ingress-nginx --ingress-class=nginx --configmap=ingress-nginx-co
 11 www-data  0:00 nginx.ingress-controller --election-id=ingress-nginx-leader --controller-class=k8s.io/ingress-nginx --ingress-class=nginx --configmap=ingress-nginx/ingress-nginx-controller --validating-
15 www-data  0:00 nginx: master process /usr/bin/nginx -c /etc/nginx/nginx.conf
 22 www-data  0:00 nginx: worker process
 23 www-data  0:00 nginx: worker process
 24 www-data  0:00 nginx: cache manager process
 90 www-data  0:00 bash
 96 www-data  0:00 ps.Auxf

```

... ИЛИ:

```
pstree -p
```

```
dumb-init(1)---nginx-ingress-c(7)+-nginx(16)-+ nginx(22)+- {nginx}(26)
                                                    |- {nginx}(28)
                                                    |- {nginx}(65)
                                                    |-nginx(23)+- {nginx}(27)
                                                    |- {nginx}(52)
                                                    -nginx(24)
                                -{nginx-ingress-c}(8)
                                -{nginx-ingress-c}(9)
```

Где:

- `/usr/bin/dumb-init` — супервайзер запущенных процессов. Его задача - запуститься первым с PID=1 и запустить следующие процессы и перезапустить их при сбое;
- `/nginx-ingress-controller` — go-приложение, которое ходит в API-сервер и смотрит, нет ли новых Ingress объектов. Если они есть, он пересоздаёт конфигурацию `/etc/nginx/nginx.conf` и перезагружает процесс;
- `nginx: worker process` — сами процесс переадресации.

Можно посмотреть результирующую конфигурацию (вывод очень длинный):

```
cat /etc/nginx/nginx.conf
```

Выходим из Pod-а через `exit`.

Смотрим описание сервиса IngressController-a:

```
kubectl describe -n ingress-nginx services/ingress-nginx-controller
```

```
Name: ingress-nginx-controller
Namespace: ingress-nginx
Labels: app.kubernetes.io/component=controller
        app.kubernetes.io/instance=ingress-nginx
        app.kubernetes.io/managed-by=Helm
        app.kubernetes.io/name=ingress-nginx
        app.kubernetes.io/part-of=ingress-nginx
        app.kubernetes.io/version=1.13.3
        helm.sh/chart=ingress-nginx-4.13.3
Annotations: meta.helm.sh/release-name: ingress-nginx
             meta.helm.sh/release-namespace: ingress-nginx
Selector: app.kubernetes.io/component=controller, app.kubernetes.io/instance=ingress-nginx, app.kubernetes.io/name=ingress-nginx
Type: NodePort
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.104.93.165
IPs: 10.104.93.165
Port: http 80/TCP
TargetPort: http/TCP
NodePort: http 30080/TCP
Endpoints: 10.244.0.141:80
Port: https 443/TCP
TargetPort: https/TCP
NodePort: https 30443/TCP
Endpoints: 10.244.0.141:443
Session Affinity: None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events: <none>
```

Теперь на каждой рабочей ноде кластера появилось правило iptables, пройдясь по цепочкам которой можно дойти то конкретной переадресаций на порт :80 и :443:

```
sudo iptables -t nat -n --line-numbers -L KUBE-NODEPORTS
```

Chain kube-NODEPORTS (1 references)									
num	target	proto	port	source	destination				
1	KUBE-EXT-C65142R83ZVWGLK	6	--	0.0.0.0/0	127.0.0.0/8	/* ingress-nginx/ingress-nginx-controller:https	/	tcp dpt:30080	nfacct-name localhost_nps_accepted_pkts
2	KUBE-EXT-C65142R83ZVWGLK	6	--	0.0.0.0/0	0.0.0.0/0	/* ingress-nginx/ingress-nginx-controller:https	/	tcp dpt:30080	
3	KUBE-EXT-EDNDUDH2C75GTR60	6	--	0.0.0.0/0	127.0.0.0/8	/* ingress-nginx/ingress-nginx-controller:https	/	tcp dpt:30443	nfacct-name localhost_nps_accepted_pkts
4	KUBE-EXT-EDNDUDH2C75GTR60	6	--	0.0.0.0/0	0.0.0.0/0	/* ingress-nginx/ingress-nginx-controller:https	/	tcp dpt:30443	

Создаём манифесты для проверки работы Ingress Controller:

```
cd ~/kubernetes/example-ingress
vim simple-ingress-controller.yaml
```

```
--
# Создаём Deployment на разворачивание 3-х подов, которые по порту :5678 возвращают информацию о себе
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: my-namespace
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
        # Для теста смены версий
        version: v1
    spec:
      containers:
        - name: my-app
          image: hashicorp/http-echo:1.0.0
          # Поля можно смотреть через `kubectl explain pod` и `kubectl describe pods my-deployment-b7c5ff48d-s7hir`
```

```

    args: ["-text='Pod: ${POD_NAME}, IP: ${POD_IP}'"]
    env:
      - {name: TZ, value: Europe/Moscow}
      - {name: POD_NAME, valueFrom: {fieldRef: {fieldPath: metadata.name}}}
      - {name: POD_IP, valueFrom: {fieldRef: {fieldPath: status.podIP}}}
    ports:
      - containerPort: 5678
---
# Создаём сервис, ведущий на Deployment
apiVersion: v1
kind: Service
metadata:
  namespace: my-namespace
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 8082
      targetPort: 5678
---
# Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  namespace: my-namespace
  name: my-ingress
  annotations:
    # Настройка поведения Ingress.
    # На Service придёт /, вместо /svc1:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx # Обязателен к указанию класса используемого IngressController
  rules:
    # Указываем правило только для хоста combo-1.dev.lan
    - host: combo-1.dev.lan
      http:
        paths:
          # Префикс пути на хосте:
          - path: /
            pathType: Prefix
            backend:
              service:
                # Имя сервиса, на который делать перенаправление:
                name: my-service
                # Порт для трафика на сервис:
                port:
                  number: 8082

```

Запускаем спецификацию:

```
| kubectl apply -f simple-ingress-controller.yaml
```

```

deployment.apps/my-deployment created
service/my-service created
ingress.networking.k8s.io/my-ingress created

```

Проверяем, что ответ для хоста <http://combo-1.dev.lan:30080> приходит от IngressController:

```
| curl http://combo-1.dev.lan:30080/
```

```
Pod: my-deployment-97bb89545-5gsdj, IP: 10.244.2.68
```

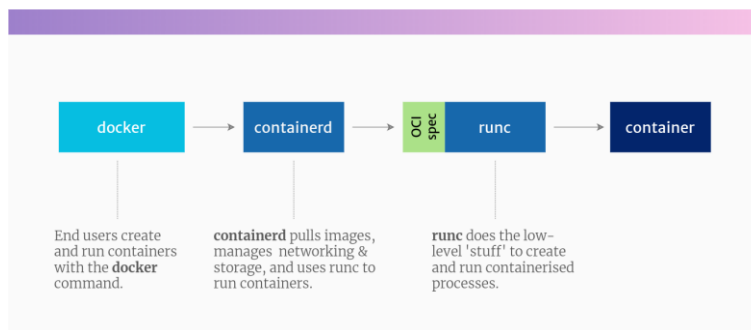
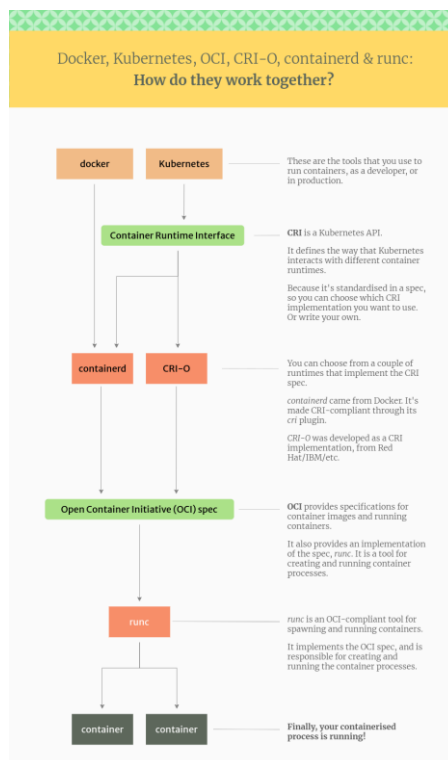
Завершаем работу тестовых подов:

```
| kubectl delete -f simple-ingress-controller.yaml
```

5 Дополнительный материал

5.1 Docker, ContainerD, CRI-O

<https://habr.com/ru/companies/domclick/articles/566224/>

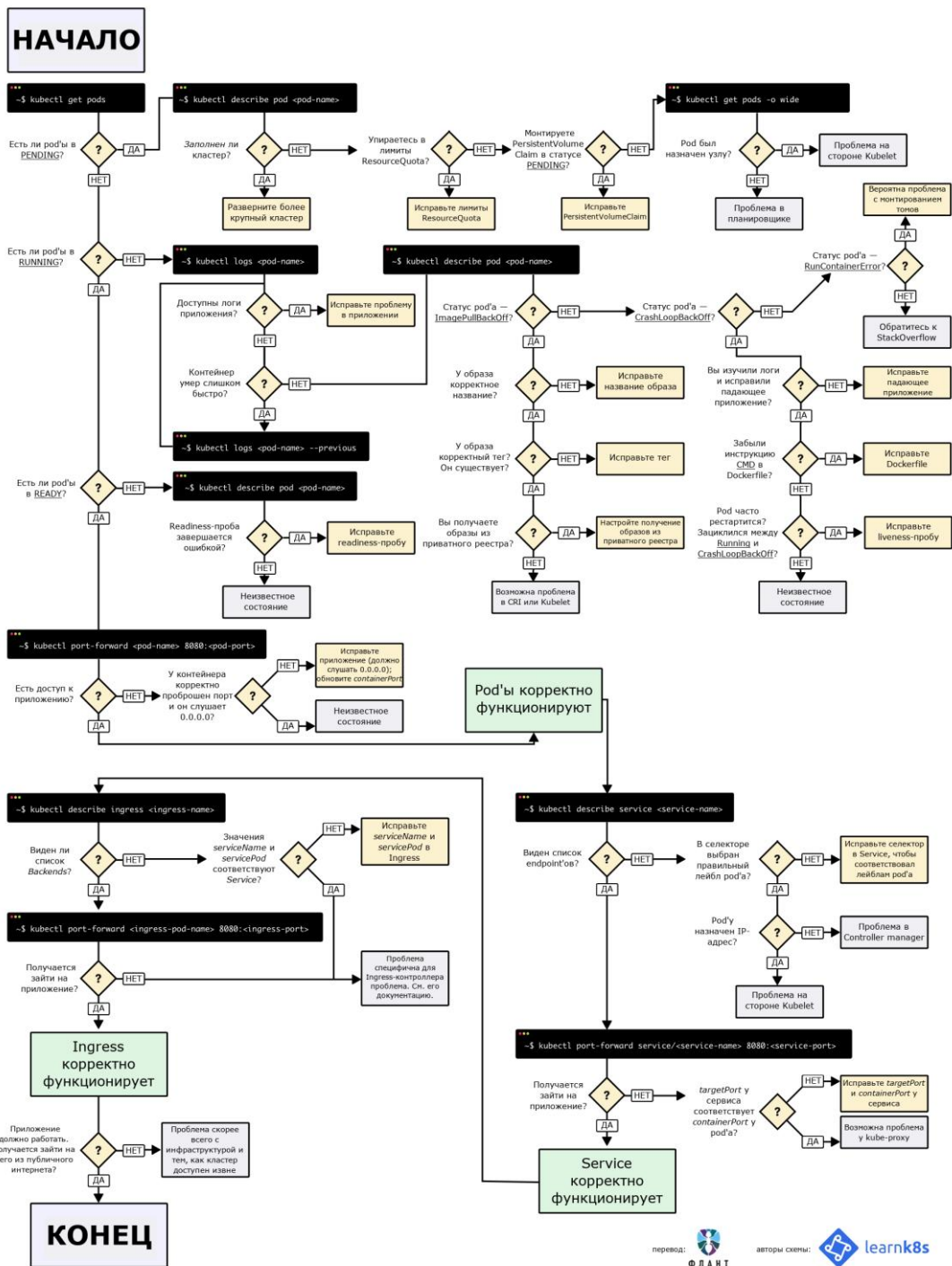


Docker состоит из трех проектов:

- **docker-cli**: утилита командной строки, с которой вы взаимодействуете с помощью команды **docker**.
- **containerd**: Linux Daemon, который управляет контейнерами и запускает их. Он загружает образы из репозитория, управляет хранилищем и сетью, а также контролирует работу контейнеров.
- **runc**: низкоуровневая среда выполнения контейнеров, которая создает и запускает контейнеры.

5.2 Диагностике неисправностей в Kubernetes

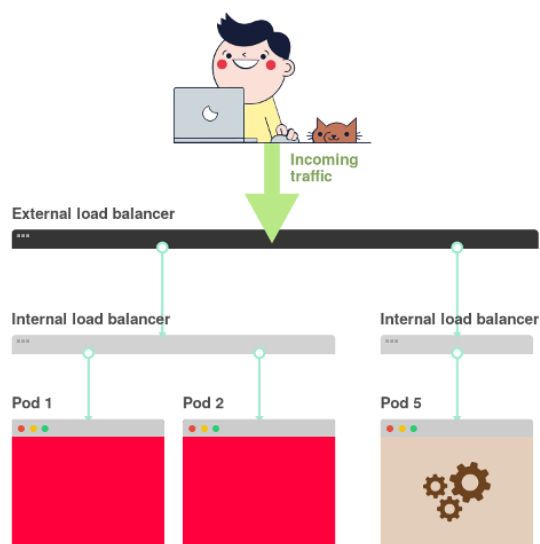
<https://habr.com/ru/companies/flant/articles/484954/>



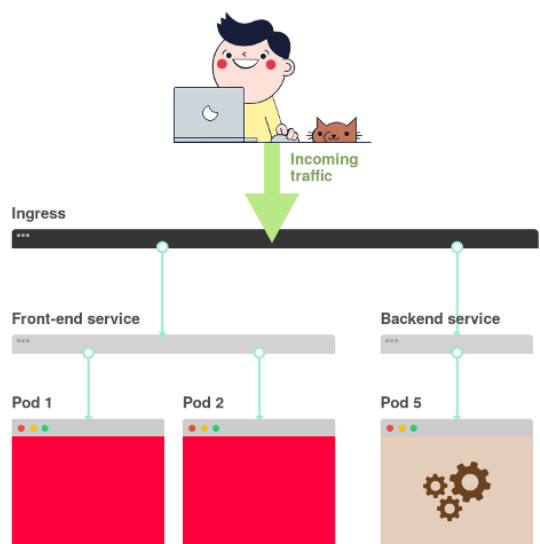
5.3 Балансировка нагрузки

<https://habr.com/ru/companies/flant/articles/484954/>

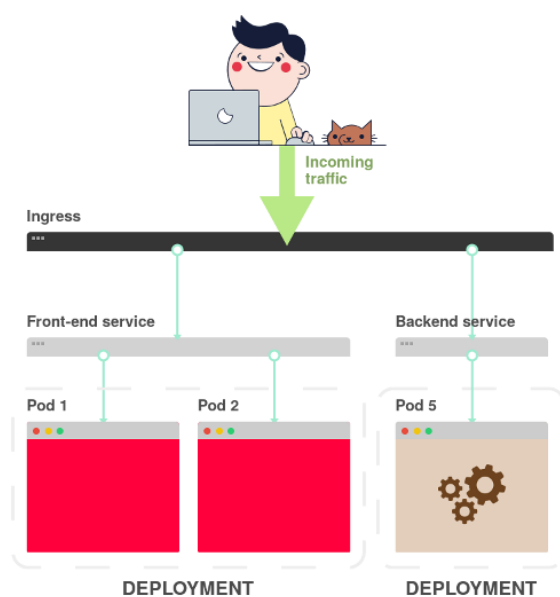
В Kubernetes приложения получают трафик из внешнего мира через два слоя балансировщиков нагрузки: внутренний и внешний:



Внутренний балансировщик называется Service, внешний – Ingress:



Deployment создает pod'ы и следит за ними (они не создаются вручную):



5.4 Helm

Обновление списка репозитория:

```
| helm repo update
```

Просмотр доступных репозиториев:

```
| helm repo list
```

Поиск чартов:

```
| helm search repo nginx
```

Установка приложения с помощью Helm:

```
| helm install my-nginx bitnami/nginx
```

где:

- `my-nginx` — имя вашего релиза (может быть любым).
- `bitnami/nginx` — имя чарта (репозиторий/чарт).

Просмотр установленных релизов:

```
| helm list
```

Обновление релиза:

```
| helm upgrade my-nginx bitnami/nginx --set replicaCount=2
```

Просмотр информации о чарте:

```
| helm show chart bitnami/nginx
```